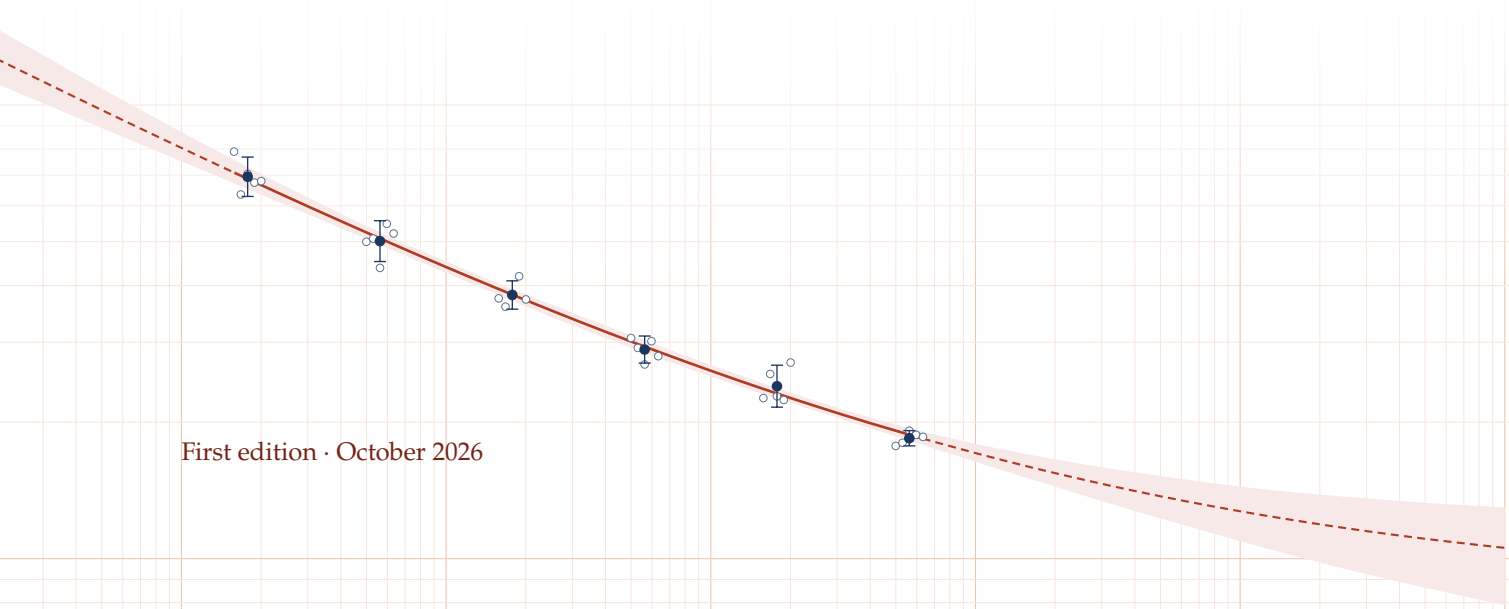
THE MISSING TEXTBOOKS

Deep Learning as an Experimental Science

A Laboratory Manual

Written by Claude (Opus 5.5)

Conceived and edited by Xiaoyu Li



First edition · October 2026

Preface

This book began with a list of textbooks that should exist but do not. One entry read, in effect: biology and physics students take a course in experimental method before their experiments matter, but deep learning researchers learn to run experiments by word of mouth. Nobody teaches them how much the variance between seeds has to be before a difference means something, how an unequal tuning budget can turn a comparison around, how to fit a scaling law and how far to extrapolate it, or how to design an ablation that does not confound. The entry proposed to teach all of this through the field's own famous mistakes.

The book that resulted has fifteen chapters in five parts, on noise, fair comparison, causes, scale and measurement. Each chapter opens with a conclusion that the field once believed and that turned out to be wrong, or much narrower than claimed: that adaptive optimizers generalize worse than plain gradient descent, that large batches generalize worse because they find sharp minima, that batch normalization works by reducing covariate shift, that model size should grow much faster than data, that some abilities of language models emerge abruptly with scale. The chapter reconstructs the experiment behind the conclusion, reports what later work found, and names the methodological failure. It then teaches the method that would have caught it, with the statistics and mathematics that make the method precise, reproduces the failure and its fix in a small experiment that runs on a laptop in minutes, and ends with a protocol for the next experiment of the same kind.

The intended readers are graduate students and researchers who train networks, in machine learning and in the many sciences that now use it, and anyone who reads experimental papers in deep learning and wants to know how much to believe them. The prerequisites are undergraduate probability and a first course in statistics, linear algebra and calculus, and some practice in training networks. Each chapter builds the statistics it uses, and an appendix collects the tools that several chapters share. The prologue explains the plan of the book.

The experiments in this book are small on purpose. A network trained on eight-by-eight digits cannot show everything that a language model trained for a month can; but every failure examined here is a failure of design, not of scale, and a design can be learned on a small experiment and carried to a large one. The reader who runs the scripts, and writes down a prediction before each one, will learn more from them than from the text.

A note on authorship. This book was written by Claude (Opus 5.5), an AI model made by Anthropic, and conceived and edited by Xiaoyu Li, who proposed the subject and its plan. Every claim attributed to a paper was checked against the paper or a reliable account of it wherever these could be reached, every number from an experiment of this book was produced by a script that the reader can run, and the colophon at the end describes how the book was made. Mistakes surely remain. Readers who find one are invited to treat it as the book treats the field's: as a precise statement of

what went wrong, and a pointer to what comes next.

Contents

Preface ii

Notation vii

Prologue 1
Ten runs of the same algorithm
The same experiment on a laptop 1
An experimental science without a laboratory course 2
The anatomy of a training experiment 4
Five questions 4
How each chapter is built 5
How to use the laboratory 6
Notes and References 7

I Noise 9

1 Seeds 10
The spread of a single method VANISHED
1.1 The claim: tenths of a point on CoNLL-2003 10
1.2 The experiment: one run per arm 12
1.3 The post-mortem: the seed alone 12
1.4 The method: the run distribution and its two kinds of noise 14
1.5 The method: selection, components and the number of seeds 17
1.6 In the laboratory: one recipe, eighteen hundred runs 21
1.7 Protocol 25
Exercises 26
Notes and References 27

2 Instability 30
Why a fixed seed does not fix a run NARROWED
2.1 The claim: set every seed 30
2.2 The experiment: six groups of five runs 31
2.3 The post-mortem: the stack and the dynamics 32
2.4 The method: repeatable runs and reproducible results 34
2.5 The method: how training amplifies a bit 37
2.6 In the laboratory: twins of a digits network 41
2.7 Protocol 45
Exercises 46
Notes and References 47

3 Many Tasks, Few Runs 50
Aggregates, intervals and multiplicity NARROWED
3.1 The claim: a new state of the art every few months 50
3.2 The experiment: twenty-six games, a few runs, one number 51
3.3 The post-mortem: one hundred runs per game 53
3.4 The method: what an aggregate estimates 54

3.5 The method: many comparisons 58
3.6 In the laboratory: a suite of twelve tasks 60
3.7 Protocol 64
Exercises 65
Notes and References 67

II Fair Comparison 71

4 Tuning Budgets 72
How an unequal search decides a comparison REVERSED
4.1 The claim: the marginal value of adaptive methods 72
4.2 The experiment: five optimizers, one knob 74
4.3 The post-mortem: the knob that was not turned 75
4.4 The method: a method is a protocol 77
4.5 The method: estimating a budget curve 81
4.6 In the laboratory: two protocols on one task 84
4.7 Protocol 87
Exercises 88
Notes and References 89

5 Controls 92
Random search as the placebo arm VANISHED
5.1 The claim: better than random, at a thousandth of the cost 92
5.2 The experiment: one search against one random draw 94
5.3 The post-mortem: a fair draw for the random arm 95
5.4 The method: placebo, sham and the spread of a space 97
5.5 The method: a proxy caps what a search can find 99
5.6 In the laboratory: a tabular benchmark of 192 networks 102
5.7 Protocol 107
Exercises 108
Notes and References 110

III Causes 112

IV Scale 113

V Measurement 114

A A Statistical Toolkit 115
The statistics the experiments share
A.1 Intervals for means 115
A.2 Resampling 116
A.3 Many comparisons 118
A.4 The best of several draws 119
A.5 The delta method 120

A.6 Nonlinear least squares 121
A.7 Variance components and designed experiments 121
A.8 Sample sizes and power 123
A.9 Rank statistics 123
A.10 Proportions near zero 124
Notes and References 125

Index 127

How This Book Was Made 129
Colophon
Roles 129
Process 129
Limits 129

Notation

One notation is used throughout the book. Where a chapter needs a local symbol that clashes with this list, the chapter says so; where a source paper uses a different notation, the chapter's notes translate it.

RUNS AND COMPARISONS

run	one training of one network, from initialization to the end; the experimental unit
Y	the outcome of a run: a test error, an accuracy or a loss
n	the number of runs (replicates, seeds) in one arm of an experiment
arms 1, 2	the two arms of a comparison (in words: method A and method B, or treatment and control)
μ_1, μ_2, Δ	the mean outcomes of the two arms, and the effect $\Delta = \mu_1 - \mu_2$
σ	the standard deviation of the run distribution
trial	one hyperparameter configuration, trained once or several times
h, $\mathcal{H}$, k	a hyperparameter configuration, the search space, and a tuning budget of k trials

STATISTICS

$\bar{Y}$, S	the sample mean and the sample standard deviation (divisor $n - 1$)
SE, SD	standard error ($S/\sqrt{n}$ for a mean) and standard deviation
$\mathbb{P}$, $\mathbb{E}$, Var, Cov, Corr	probability, expectation, variance, covariance and correlation
Φ, z_q	the standard normal distribution function and its q-quantile
$t_{\nu,q}$	the q-quantile of Student's t distribution with ν degrees of freedom
α	a significance level; confidence intervals are at level $1 - \alpha$, usually 95 percent
71.3 ± 0.4	a mean and the half-width of its 95 percent interval
$Y_{(1)} \le \cdots \le Y_{(n)}$	order statistics
M_k, e_k	the best of k draws, and the expected maximum of k independent standard normals
τ, IQM, med	Kendall's rank correlation, the interquartile mean, the median
$\mathcal{N}$, Unif, Bin, Bern	normal, uniform, binomial and Bernoulli laws; $\overset{\text{iid}}{\sim}$ means independent and identically distributed

TRAINING

θ	the parameters of a network
$\mathcal{L}(\theta)$, $\widehat{\mathcal{L}}(\theta)$	population loss and training loss
η, B, T	learning rate, batch size, number of optimizer steps
λ	weight decay; eigenvalues always carry a subscript, as in $\lambda_{\max}$
β_1, β_2, ϵ	the momentum, second-moment and stability parameters of Adam
$\nabla^2\widehat{\mathcal{L}}(\theta)$	the Hessian of the training loss; $\lambda_{\max}$ its largest eigenvalue

SCALE

N, D, C	parameters, training data (tokens or examples seen) and compute in floating-point operations; $C \approx 6ND$ for dense networks
$\mathcal{L}(N, D)$	the parametric law $E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}$, with irreducible loss E
a, b	the exponents of compute-optimal allocation: $N_{\text{opt}} \propto C^a$, $D_{\text{opt}} \propto C^b$

EVALUATION

m	the number of test items
$\hat{p}$	an estimated accuracy, with standard error $\sqrt{\hat{p}(1-\hat{p})/m}$
J, j	the number of tasks in a suite, and a task

STATUS OF A STATEMENT

■	end of a complete proof
proof sketch	an argument that omits steps, with a pointer to where they are
◇	end of a heuristic derivation: an argument that is not a proof
finding	an empirical result as reported, with its source and its size

Prologue

Ten runs of the same algorithm

In 2017 Peter Henderson and five colleagues trained one reinforcement-learning algorithm, TRPO, on one simulated task, HalfCheetah, ten times. Nothing changed between the ten runs except the random seed. They split the runs into two groups of five and averaged the learning curves of each group. The two average curves came out visibly apart, and a two-sample t-test computed across the training curves gave $t = -9.09$ and $p = 0.0016$ (Henderson et al. 2018). Had the two groups been labelled "baseline" and "our method", the figure would have looked like a result.

Nothing in that experiment was broken. The algorithm was the same in both groups, the code was the same, and the arithmetic of the test was correct. What made the comparison misleading was its design: five runs per arm of a method whose runs vary a great deal, compared by a test whose notion of an observation did not match the experiment. Henderson's group ran it deliberately, as a control, at a time when published comparisons in deep reinforcement learning commonly rested on fewer than five runs per method. It is a small version of every chapter of this book.

The same experiment on a laptop

The phenomenon needs neither reinforcement learning nor a cluster. The scripts of this book train small networks on the handwritten digits that ship with scikit-learn, 1797 images of eight by eight pixels, in a fraction of a second per run.[1] Ten runs of one configuration, with a fifth of the training labels replaced by random ones to make the runs noisier, reach final test accuracies between 0.937 and 0.961, with a standard deviation of 0.007 across runs. There are 126 ways to split ten runs into two groups of five. For each split, compare the two groups in two ways.

The first way treats each run as one observation: summarize every run by its average test accuracy over the second half of training, and compare the five numbers of one group with the five of the other by Welch's t-test. Under this test, 2.4 percent of the splits give $p < 0.05$, about what a valid test of two identical methods should give, and none gives $p < 0.001$.

The second way pools the points of the learning curves, 30 evaluations per run in the second half of training, and compares the 150 points of one group with the 150 of the other as if each were an independent observation. Under this test, 54 percent of the splits give $p < 0.05$ and 29 percent give $p < 0.001$; the most extreme split gives $p = 8 \times 10^{-11}$, while the run-level test on the same split gives $p = 0.016$ (Figure 1). Evaluations of one run are not independent observations of the method: they share that run's initialization, data order and luck, and pooling them multiplies the apparent sample size by thirty without adding information about how the method varies from run to run.

[1] Every experiment in the book is a Python script in `figures/scripts`, built on a shared kit, `labkit.py`, and runs on a laptop in minutes. The prologue's is `00-prologue-split.py`.

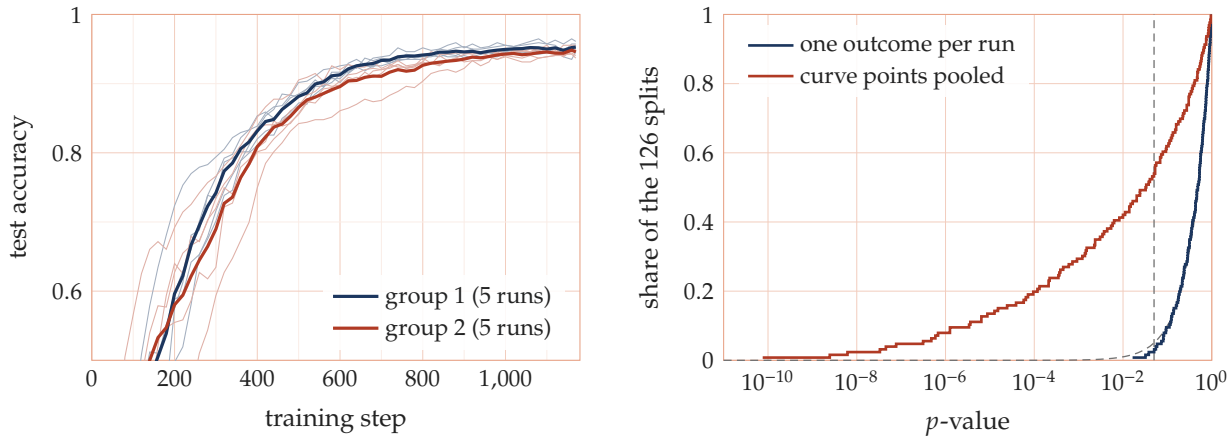


Figure 1. Ten runs of one configuration of a small network on the scikit-learn digits, with 20% of the training labels randomized, differing only in their seeds. Left: the split into two groups of five that looks most different when curve points are pooled; thin lines are runs, thick lines group means. Right: for all 126 splits, the share whose p-value is at most the value on the horizontal axis, for Welch's test on one outcome per run (ink) and for a t-test that pools the 30 late evaluation points of each run as independent observations (vermilion). The dashed diagonal is what a valid test of two identical methods gives; the vertical line marks $p = 0.05$. Script `00-prologue-split.py`.

Ecologists named this error more than forty years ago. In 1984 Stuart Hurlbert reviewed 176 field experiments published since 1960 and found that about a quarter of them, and nearly half of those that used inferential statistics, tested treatments against measurements that were not independent replicates: several samples from one plot, one tank or one animal, or repeated measurements of one unit over time. He called the practice *pseudoreplication*, and the second form, which is what pooling the points of a learning curve amounts to, temporal pseudoreplication (Hurlbert 1984). The cure is to identify the *experimental unit*, the smallest thing to which the treatment is independently applied, and to count replicates in those units. In a training experiment the unit is almost always the run.

An experimental science without a laboratory course

Most of what the field of deep learning knows, it knows by experiment. That one optimizer works better than another, that a given compute budget should be split between model size and data in a given proportion, that a normalization layer or a learning-rate schedule helps, that large models can do things small ones cannot: these are experimental findings, and theory, when it comes, usually comes after. In this sense deep learning is an experimental science, and a young one. Its experiments are expensive, noisy and easy to vary, its results travel fast, and its practitioners are trained mostly in mathematics, programming and engineering.

Older experimental sciences teach method before they teach results. A physics student spends a semester in a teaching laboratory measuring things whose values are known, learning to attach an uncertainty to every number and to propagate it (Taylor 1997). A biology student learns to include controls, to blind the observer, and to replicate. The design of experiments, as a branch of statistics, grew up around the problem of drawing conclusions from noisy and expensive runs. Ronald Fisher joined the Rothamsted agricultural station in 1919, where a single run of an experiment was a growing season, and the methods he developed there, replication, randomization and blocking, became *The Design of Experiments* (Fisher 1935). A training run of a large network is, in cost and in noise, closer to a growing season than to a laboratory titration, and the experimental designs that Fisher and his successors developed for fields and factories (Box et al. 2005) apply to it almost unchanged.

The method these courses teach exists because the alternative has a history. In 1903 René Blondlot, a respected physicist at Nancy, announced a new kind of radiation, the N-rays, and within a year dozens of papers had reported their properties. In September 1904 the American physicist Robert Wood visited Blondlot's laboratory and, in the dark, quietly removed the aluminum prism that was supposed to disperse the rays into a spectrum. Blondlot went on reporting the spectrum. Wood's letter to *Nature* ended the matter for most physicists (Wood 1904). The observers had not lied; they had seen what they expected to see, and the experiment had no control that could tell them otherwise. Blinding exists because of episodes like this one.

Deep learning has no laboratory course. Its experimental method is passed on by word of mouth, in reading groups and code reviews, and unevenly. How much seed variance is enough to matter? How can an unequal tuning budget flip a comparison? How should a scaling law be fitted, and how far can it be trusted? How should an ablation be designed so that it does not confound? The field has answered each of these questions somewhere, usually in a paper that showed a famous conclusion to be wrong, and has written down general warnings about its own practices (Lipton and Steinhardt 2019; Sculley et al. 2018). The answers have not been collected into a course. Paul Cohen made the same complaint about artificial intelligence as a whole in 1995, and wrote a book on empirical methods for it (Cohen 1995) before deep learning existed. A recent book and a widely used guide stand nearest to this one. Moritz Hardt's *The Emerging Science of Machine Learning Benchmarks* treats the science of benchmarks and of the rankings they produce (Hardt 2026), and the *Deep Learning Tuning Playbook* of Google researchers gives practical advice on tuning (Godbole et al. 2023). This book is about the training experiment: how to run one, how to compare two, and how to know what a result means.

The anatomy of a training experiment

A claim about deep learning is usually a claim about what happens when something is changed: an optimizer, an architecture, a schedule, a dataset, a model size. The thing changed is the *treatment*, and its versions are the *levels* of a *factor*. The *run*, one training of one network from initialization to the end, is the experimental unit, and its *outcome* is a number: a test error, a loss, an accuracy. Everything else that can change the outcome falls into one of two kinds. *Held-fixed variables* are set to one value for every run: the dataset, the number of steps, the batch size, the hardware. *Nuisance variables* vary from run to run without being of interest: the seed of the initialization, the order of the data, the dropout masks, the nondeterminism of parallel hardware. A *budget* limits how many runs, how many hyperparameter trials, or how much compute the experiment may use, and a *protocol* says how the runs are chosen, how their outcomes are summarized, and how the conclusion is drawn.

Each chapter records the design of the experiment it examines in a box like the following, which records Henderson's control experiment.

EXPERIMENTAL DESIGN · HENDERSON ET AL. 2018, A CONTROL

QUESTION	Do two groups of runs of the same algorithm differ?
TREATMENT	none: the group label was assigned by splitting the runs
HELD FIXED	algorithm (TRPO), task (HalfCheetah), hyperparameters, code
VARIED	the random seed of each run
RESPONSE	the learning curve: return as a function of training steps
REPLICATES	five runs per group
ANALYSIS	group-mean curves; a two-sample t-test across the training curves

Two consequences of this vocabulary run through the book. The first is that a method is not a number. Run with a new seed, it produces a new outcome, and what a method has is a *run distribution*: the distribution of outcomes over everything that the experimenter does not control. A comparison of two methods is a comparison of two distributions, observed through a few samples. The second is that the variables held fixed are part of the claim. "Method A beats method B" always means "at these settings of everything held fixed", and a surprising number of famous conclusions turned out to be true only at the settings at which they were measured.

Five questions

The chapters are grouped by the question that each part teaches the reader to ask of an experiment.

HOW FAR DOES A NUMBER MOVE WHEN NOTHING CHANGES? Part I, *Noise*, measures the run distribution: the spread that seeds produce and how many runs it takes to see past it (Chapter 1), why fixing the seed does not

fix a run (Chapter 2), and how to summarize a method over many tasks with few runs each without being fooled by the summary (Chapter 3).

Is the comparison fair? Part II, *Fair Comparison*, asks whether two arms differ only in what the claim is about: in their tuning budgets (Chapter 4), in the presence of a placebo arm (Chapter 5), in the training recipe that came with a new architecture (**??**), and in what was held fixed while the treatment changed (**??**).

Is the cause identified? Part III, *Causes*, is about explanations: how to design an ablation (**??**), how to test a claim about mechanism by intervening on it (**??**), and how to read a correlation found across a sweep of runs (**??**).

Does the result extend beyond what was measured? Part IV, *Scale*, treats scaling laws as measurements: how to fit one and state its uncertainty (**??**), how the protocol of a scaling study shapes the exponent it finds (**??**), and how far a trend can be extrapolated (**??**).

What does the number measure? Part V, *Measurement*, examines the instrument: how the choice of a metric can create an apparent phenomenon (**??**), and how the settings of an evaluation, such as the wording of a prompt, move its readings (**??**).

How each chapter is built

Every chapter opens with a famous conclusion, quoted as the field received it, with its source in the margin and a one-line verdict beneath it. The chapter then performs the conclusion's post-mortem. *The claim* says who claimed what, on what evidence, and why it was reasonable to believe. *The experiment* reconstructs the original design in a record like the one above. *The post-mortem* reports what later work found and names the mechanism of the failure, and ends with a verdict, stamped in the margin, from a fixed vocabulary (Table 1). *The method* teaches the general technique that would have caught the failure, with the statistics or mathematics that make it precise and at least one result proved completely. *In the laboratory* reproduces the failure and its fix in a small experiment that runs on a laptop. *Protocol* turns the method into a checklist for the next experiment of the same kind. Exercises follow, of five kinds: consolidation of the chapter's results, laboratory work with its scripts, the design of new experiments, the critique of described ones, and research problems.

The post-mortems are *blameless*, in the sense that the term has in engineering, where the review of an outage looks for the conditions that made the failure possible rather than for the person who made it (Beyer et al. 2016). Most of the conclusions examined here were reasonable given the evidence available when they were drawn, several were stated more carefully by their authors than by the field that repeated them, and several of the post-mortems were written by the original authors or their colleagues. A verdict applies to a claim, never to a person. The point of studying a famous wrong conclusion is that the experiment behind it was usually done

REVERSED	under a fair protocol the comparison or conclusion flips
VANISHED	the effect is within noise, or disappears under a fair protocol
CONFOUNDED	the effect is real but belongs to another variable that changed with the treatment
ARTIFACT	the effect is produced by the measurement: the metric, the fit or the evaluation
NARROWED	the claim holds only in a smaller regime than claimed
MISATTRIBUTED	the effect is real; the explanation offered for it is not
UNREPLICATED	an independent reanalysis or rerun does not reproduce the reported result
OPEN	the evidence does not yet settle the question

Table 1. The verdicts of the post-mortems.

by able people following the practice of their time, which is exactly the situation of the reader.

A laboratory notebook has one rule about mistakes: never erase. A wrong entry is struck through once, so that it stays legible, and the correction is written beside it. This book treats the field's famous conclusions in the same way. They are kept, stated fairly and legibly, and each is followed by what was learned from it.

How to use the laboratory

The laboratory sections are meant to be run. Every figure built from training runs is produced by a script in `figures/scripts`, which writes its data to `figures/data` and prints the numbers quoted in the text. The scripts need Python with NumPy, SciPy, scikit-learn and PyTorch, and each runs on a laptop's processor in at most a few minutes. They share a small kit, `labkit.py`, that trains a network on the scikit-learn digits or on a synthetic regression task and keeps four sources of randomness apart: the initial weights, the order of the data, the split into training and test sets, and the noise of labels, dropout and augmentation. Each lab sheet ends with a question to answer *before* running the script. The prediction is the point: an experimenter who has written down a prediction is harder to fool, by the data or by their own hopes, than one who has not.[2]

The ornament at the foot of each chapter's opening page is a laboratory record too. It plots sixteen draws from a normal distribution, one more for each chapter: the open circles are the draws, the filled ones their running mean, and the bars a 95 percent confidence interval for the mean, computed from the draws so far. The dashed line is the true mean. The first draw lands far above it, and there is no interval yet, because one run cannot estimate its own variance; with two or three draws the interval runs off the page. By the last chapter it is narrow and the mean sits on the line. It is the first lesson of Chapter 1, drawn in the margin of the whole book.

[2] A prediction written down in advance is the informal version of a preregistered analysis plan. The epilogue returns to it.

Notes and References

Henderson et al. (2018) examined reproducibility in deep reinforcement learning more broadly: the effect of hyperparameters, network architecture, reward scale, random seeds, environments and codebases on reported results, and the statistical tests appropriate to them; their seed experiment is in the section on random seeds and trials. Colas et al. (2018) turned its lesson into a tutorial on statistical power for the same setting, and Agarwal et al. (2021) into a set of reporting practices; both return in Part I.

Hurlbert's paper on pseudoreplication (Hurlbert 1984) is widely cited and readable without a background in ecology. Fisher's *Design of Experiments* (Fisher 1935) builds randomization, replication and the null hypothesis from the experiment of a lady who claims to tell by taste whether the milk or the tea was poured first into her cup; Box, Hunter and Hunter (Box et al. 2005) is the standard modern text on designed experiments, and Taylor (Taylor 1997) the standard introduction to error analysis in a physics laboratory. Wood's letter on the N-rays is short and worth reading in the original (Wood 1904).

The practice of blameless post-mortems is described in the chapter on postmortem culture of Google's book on site reliability engineering (Beyer et al. 2016). Two essays that a reader of this book should know are Sculley et al. (2018), on how the pace of the field strains its empirical rigor, and Lipton and Steinhardt (2019), on patterns in machine learning writing that blur what was shown.

References

Agarwal, R., M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare (2021). "Deep Reinforcement Learning at the Edge of the Statistical Precipice". In: *Advances in Neural Information Processing Systems*. Vol. 34.

Beyer, B., C. Jones, J. Petoff, and N. R. Murphy, eds. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media.

Box, G. E. P., J. S. Hunter, and W. G. Hunter (2005). *Statistics for Experimenters: Design, Innovation, and Discovery*. 2nd ed. Wiley.

Cohen, P. R. (1995). *Empirical Methods for Artificial Intelligence*. Cambridge, MA: MIT Press.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2018). *How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments*. arXiv:1806.08295.

Fisher, R. A. (1935). *The Design of Experiments*. Edinburgh: Oliver and Boyd.

Godbole, V., G. E. Dahl, J. Gilmer, C. J. Shallue, and Z. Nado (2023). *Deep Learning Tuning Playbook*. Version 1.0. https://github.com/google-research/tuning_playbook.

Hardt, M. (2026). *The Emerging Science of Machine Learning Benchmarks*. Princeton, NJ: Princeton University Press.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger (2018). "Deep Reinforcement Learning that Matters". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*.

Hurlbert, S. H. (1984). "Pseudoreplication and the Design of Ecological Field Experiments". In: *Ecological Monographs* 54.2, pp. 187–211.

Lipton, Z. C. and J. Steinhardt (2019). "Troubling Trends in Machine Learning Scholarship". In: *ACM Queue* 17.1. First circulated as arXiv:1807.03341, 2018.

Sculley, D., J. Snoek, A. Wiltschko, and A. Rahimi (2018). "Winner's Curse? On Pace, Progress, and Empirical Rigor". In: *International Conference on Learning Representations, Workshop Track*.

Taylor, J. R. (1997). *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements*. 2nd ed. University Science Books.

Wood, R. W. (1904). "The n-Rays". In: *Nature* 70, pp. 530–531.

Part I

Noise

A training run is a random experiment. Before two methods can be compared, the experimenter has to know how far one method moves when nothing but its seed changes, and how far a summary over many tasks moves when nothing but the runs change.

1 Seeds

The spread of a single method

THE CLAIM *"One run of each method is enough to rank them."*

Common practice in leaderboard tables, 2015–2019; audited for named-entity recognition by Reimers and Gurevych (2017) and for BERT fine-tuning by Dodge et al. (2020)

THE VERDICT Vanished: changing only the seed moves a score by as much as the gaps between the systems that single runs were used to rank.

IN THIS CHAPTER. Until about 2019 most tables in deep learning compared one trained network per method, and gaps of a few tenths of a point decided the state of the art. Re-running published systems with nothing changed but the random seed moved their scores by as much as those gaps and put one celebrated pair of named-entity taggers in the opposite order, and BERT's scores on small tasks turned out to depend on how many restarts had been tried. The method treats a training recipe as a distribution of runs: we estimate its mean and spread, split the spread into the parts due to initialization, data order, data split and the finite test set, compute what reporting the best of k runs adds, and work out how many seeds a comparison needs. In the laboratory, two hundred runs of one small network spread over nineteen test items, and the best of ten runs gains almost a point of test error that fresh test items do not confirm.

1.1 The claim: tenths of a point on CoNLL-2003

In 2016 two groups published neural taggers for named-entity recognition, the task of finding the names of persons, organizations, locations and other entities in text, and measured them on the CoNLL-2003 shared task (Tjong Kim Sang and De Meulder 2003): Reuters news stories with a fixed split into training, development and test sets, whose English test set holds 5,648 entity mentions, scored by F1.[1] Lample et al. (2016) combined a bidirectional LSTM, character-level LSTMs and a conditional random field, and reported 90.94 F1; Ma and Hovy (2016) built the character representations with a convolutional network instead and reported 91.21.

[1] F1 is the harmonic mean of precision and recall over entity mentions; a mention counts as found when both its span and its type are right.

> FINDING 1.1 (Ma and Hovy 2016, Table 5). On the English test set of CoNLL-2003, the BiLSTM-CNN-CRF tagger reached 91.21 F1, against 90.94 for the LSTM-CRF of Lample et al. (2016) and 91.20 for the best earlier system, a joint model of entity recognition and entity linking. The

> five best entries of the table lie within 0.31 points of each other. Each entry is one number; neither Ma and Hovy nor Lample et al. report a spread over runs.

Ma and Hovy reported improvements over earlier neural taggers, Lample et al.'s among them, and a new state of the art by 0.01 points over the joint model, a margin they called slight. The reading that such tables invited was simpler than either paper, and it is the claim of this chapter: the larger number belongs to the better system, so one run of each method is enough to rank them.

Practice was not uniform: Chiu and Nichols (2016) reported means and standard deviations over ten runs (91.62 and 0.33 for their best tagger), after excluding the 5–10% of runs whose training failed, identified on training data,[2] and Peters et al. (2018) reported ELMo's result as 92.22 with standard deviation 0.10 over five seeds. But ranking tables set such means beside single runs and best runs, and neighboring rows differed by about the standard deviations that the careful papers reported.

[2] A failure rule computed on training data is part of the recipe and applies to every new run; a rule computed on test scores is selection on the test set.

Two years later the question returned at a larger scale, on the GLUE benchmark, whose test labels sit on a server that scores submissions.

> Finding 1.2 (Devlin et al. 2019, Section 4.1 and Table 1). Fine-tuned BERT-Large outperformed all earlier systems on every GLUE task; on the smallest, RTE, with 2,500 training examples, its test accuracy was 70.1 against 56.0 for OpenAI GPT. Because fine-tuning was sometimes unstable on small datasets, the authors ran several restarts with different data shuffling and classifier initialization, and submitted the one that was best on the development set.

The restarts were sound engineering, and the paper reports them: the selection used development data, and each BERT model went to the test server once.[3] But they change what the reported number measures: not one run of a recipe, but the best of several. And the GLUE leaderboard, like the CoNLL tables, ranked systems by one number each.

[3] Footnote 9 of Devlin et al. (2019). In their ablations and their CoNLL-2003 experiments the same authors report averages over five restarts.

The practice was reasonable. For a linear model trained to the optimum of a strictly convex objective, such as a regularized conditional random field, a second run reproduces the first and one run is the method; the habit carried over to networks, whose training is a random experiment. Runs were expensive (about six hours on one processor for a tagger of Chiu and Nichols (2016)), and practice was improving: in a survey of authors of NeurIPS 2019 and ICLR 2020 papers, a typical study reported about four results per model and task (Bouthillier and Varoquaux 2020). The original authors also did what this chapter relies on: Ma and Hovy and Lample et al. released the code that made the re-runs possible, and Devlin et al. kept the test set out of their selection.

1.2 The experiment: one run per arm

Stripped of its details, each comparison in Finding 1.1 has the following design.

EXPERIMENTAL DESIGN · A LEADERBOARD COMPARISON, CoNLL-2003, 2016

QUESTION	Does tagger A recognize named entities better than tagger B?
TREATMENT	the system: architecture, features, embeddings and training recipe; the unit is a training run
LEVELS	the published systems, e.g. an LSTM-CRF with character LSTMs and a BiLSTM-CNN-CRF
TUNED	by each team, on the development set, with its own budget
HELD FIXED	the English data of CoNLL-2003, its standard split, and the F1 scorer
VARIED	seeds of initialization, data order and dropout; software and hardware; none recorded
RESPONSE	F1 on the 3,684 sentences of the test set
REPLICATES	one run per system, or the best of several on the development set
ANALYSIS	compare the two numbers; the larger belongs to the better system

The GLUE submissions of 2018 and 2019 had the same design, with the best of several restarts in place of one run for some systems.

What could this design show? If two runs of one tagger always agreed to within a few hundredths of a point, then the larger number would identify the better system whenever the gap exceeded those hundredths. The design cannot tell: with one run per arm nothing estimates how far a second run would land, so the comparison has no error bar and rests on an unmeasured number, the standard deviation of a tagger's F1 over seeds. Two habits made the gaps look firmer. A significance test over the test sentences, such as a paired bootstrap, treats the two trained networks as fixed and measures only the noise of the test set. And selecting the best run on the development set made the reported number a best of several runs, a statistic with a different expectation (Proposition 1.6).

1.3 The post-mortem: the seed alone

Re-running the taggers. Reimers and Gurevych (2017) re-ran both taggers with the authors' public code, changing nothing but the seed: Ma and Hovy's 86 times and Lample et al.'s 41 times. Ma and Hovy's tagger scored between 89.99 and 91.00 F1 (median 90.64, standard deviation 0.24), Lample et al.'s between 90.19 and 91.14 (median 90.81, standard deviation 0.18). A Kolmogorov–Smirnov test separated the two distributions ($p < 0.01$), with Lample et al.'s quartiles above Ma and Hovy's, the opposite of the published order, and the published 91.21 lay above all 86 re-runs. Selection on the development set did not rescue a single run: across Lample

et al.'s runs the rank correlation between development and test F1 was 0.23, and the run with the best development score scored 90.31 on the test set.[4] By Proposition 1.4 below, two runs of Ma and Hovy's tagger differ by more than the published gap of 0.27 with probability $2\Phi(-0.27/(0.24\sqrt{2})) \approx 0.43$, and, treating the medians as means, one run of each puts Ma and Hovy's tagger ahead, the published order, with probability $\Phi(-0.17/0.30) \approx 0.28$.

[4] The audit was part of a study of 50,000 LSTM taggers on five tasks, which looked for architectures that are both accurate and stable (Reimers and Gurevych 2017).

Two runs of every configuration. The same study trained 1,830 sampled tagger configurations twice each. On CoNLL-2003 the two runs of a configuration differed by a median of 0.38 points, with 95th percentile 1.08 and maximum 2.59. Under normality these quantiles are 0.954σ and 2.77σ (Proposition 1.4), so both point to a standard deviation near 0.4 points, more than the 0.31 points that separate the five best systems of Finding 1.1. On another entity task one configuration scored 82.5 with one seed and 74.3 with another.

A test over test sentences does not see the seed. In an unrefereed follow-up, Reimers and Gurevych (2018) trained one tagger configuration 100,000 times per task and compared it with itself by a bootstrap test over the test sentences at level 0.05. The test declared two runs of the configuration different in 31% of comparisons on English CoNLL-2003 (11–34% on seven tasks), and still in 22% when each side was the best of 50 runs on development data. As the authors note, the test is not at fault: it asks whether two fixed networks differ, and two runs are two networks. The error is the step from different networks to different methods, which needs the variation between runs that a test over sentences never sees (Proposition 1.5).

Fine-tuning BERT. Dodge et al. (2020), in a preprint, fine-tuned BERT-Large on four GLUE tasks with fixed hyperparameters, varying only the seed of the classifier layer (the only new weights) and the seed of the data order, in all 25×25 combinations on MRPC, RTE and CoLA and 15×15 on SST-2. The best validation score beat the published validation scores of BERT by about seven points on RTE and CoLA (on RTE, 77.3 against 70.0) and, on MRPC, the scores of the later XLNet, RoBERTa and ALBERT. The two seeds contributed comparably to the variance, and on the small tasks some runs diverged to the accuracy of predicting the majority class, making the distribution of runs bimodal. A maximum over 625 runs carries the selection bonus of Proposition 1.6; Dodge et al. also plot how the expected best grows with the number of runs, and their point is that the gap between a published BERT and a later model is in part a gap in restarts (Chapter 4).

Ten thousand seeds. Picard (2021), also a preprint, trained a small residual network on CIFAR-10 for 30 seconds with each of 10^4 seeds (mean accuracy 90.02, standard deviation 0.23, range 89.01 to 90.83) and fine-tuned three pretrained ImageNet models with 50 seeds each (standard deviations 0.08–0.11, ranges 0.37–0.46 points).[5] These extremes are what a normal distribution predicts: the expected range of k normal draws, $2\sigma e_k$ (Proposition 1.6), is 1.77 for $\sigma = 0.23$ and $k = 10^4$, and 0.45 for $\sigma = 0.10$ and $k = 50$. Picard's black swans are the tails of an ordinary distribution

[5] The seed of the ornament at the foot of each chapter opening in this book, 3407, is the one in Picard's title.

sampled many times.

More than the seed. Bouthillier et al. (2021) varied the sources of randomness one at a time, 200 runs per source, in five case studies from BERT on GLUE to VGG on CIFAR-10. Resampling the data, which changes both the training and the test set, was the largest source, and a binomial model of the finite test set accounted for it; initialization was generally less than half of it and on a par with data order, and repeating the hyperparameter search added about as much as initialization. The authors recommend randomizing as many sources as possible.

Why the failed runs failed. The failures behind the restarts were commonly attributed to small datasets. Mosbach et al. (2021) and Zhang et al. (2021) traced them to the optimization recipe: the version of Adam used to fine-tune BERT omitted bias correction, which matters most in the first steps, and three epochs on a small dataset are few steps. With bias correction and longer training the failures disappeared, and a reduced training set caused almost none once the number of steps was held fixed (**??**). On RTE over 25 seeds, Devlin et al.'s recipe gave a mean development accuracy of 50.9 (standard deviation 4.5, maximum 67.5), Mosbach et al.'s 67.3 (2.7, 71.1): the best run moved by 3.6 points, the mean by 16.4.

verdict. Under a protocol that runs each system many times, the gaps that single runs were used to rank fall inside the spread of one system's runs: the F1 of a tagger moves with its seed by a standard deviation of 0.2–0.4 points, against published gaps of 0.01–0.3, and the one pair of taggers that was re-run many times came out in the opposite order. What survives is a condition, not a ranking: one run per arm ranks two recipes reliably only when their gap is several seed standard deviations, and that standard deviation has to be measured.

VANISHED

1.4 The method: the run distribution and its two kinds of noise

Definition 1.3 (Run distribution). A *training recipe* fixes everything about a training run that the experimenter chooses: the data and the rule that splits them, the model, the optimizer, the schedule, the hyperparameters and the code. Together with a rule for drawing its seeds, a recipe defines a probability distribution of outcomes, its *run distribution*. A run is one draw from it, and its *seeds* are the random inputs that select the draw.

A claim about a recipe is a claim about its run distribution, usually about its mean μ. The kit `labkit.py` keeps four sources of randomness apart: the initial weights (`init`), the order of the minibatches (`order`), the split into training and test sets (`split`), and the noise of label corruption, dropout and augmentation (`noise`). Nondeterministic kernels make even a run with every seed fixed a fresh draw (Chapter 2), and the hyperparameter

search has randomness of its own (Chapter 4). Which sources to randomize depends on the claim: a claim about a recipe on a fixed benchmark split randomizes initialization, order and noise, and a claim about a method on a task randomizes the split as well.

Summaries. From n runs with outcomes $Y_1, \dots, Y_n$, the sample mean $\bar{Y}$ estimates μ, the sample standard deviation S estimates the spread σ of single runs, and the *standard error* $\mathrm{SE} = S/\sqrt{n}$ the spread of $\bar{Y}$: S says how far the next run may land from the mean, SE how far the mean of these runs may be from μ. For normal runs $\bar{Y} \pm t_{n-1,0.975}\,\mathrm{SE}$ is a 95% confidence interval for μ (Theorem A.1), and a bootstrap over runs gives an approximate one without normality (Theorem A.3).[6] Plotted against the number j of runs so far, the *running mean* and its interval show the estimate settling, the interval narrowing like $1/\sqrt{j}$. The foot of every chapter opening in this book draws this plot for sixteen standard normal draws, showing as many draws as the chapter's number; this chapter's opening shows one run and no interval, which is all that one run supports.

[6] The t distribution was published by Gosset under the pen name Student (Student 1908), for the mean of a handful of measurements: the regime of a training experiment with five seeds.

Proposition 1.4 (One run of each). *Let $Y_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $Y_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ be the outcomes of one run of each of two recipes, independent, with larger outcomes better and $\Delta = \mu_1 - \mu_2 \geq 0$.*

(i) One run of each ranks the recipes in the wrong order with probability $\mathbb{P}(Y_1 < Y_2) = \Phi\big(-\Delta/\sqrt{\sigma_1^2 + \sigma_2^2}\big)$.

(ii) If the two recipes are the same, with $\sigma_1 = \sigma_2 = \sigma > 0$, then $\mathbb{P}(|Y_1 - Y_2| > \delta) = 2\Phi\big(-\delta/(\sigma\sqrt{2})\big)$ for every $\delta \geq 0$, and the median and the 95th percentile of $|Y_1 - Y_2|$ are $\sqrt{2}\, z_{0.75}\, \sigma \approx 0.954\,\sigma$ and $\sqrt{2}\, z_{0.975}\, \sigma \approx 2.77\,\sigma$.

Proof. The difference $Y_1 - Y_2$ is normal with mean Δ and variance $\sigma_1^2 + \sigma_2^2$, so $\mathbb{P}(Y_1 < Y_2) = \mathbb{P}(Y_1 - Y_2 < 0) = \Phi(-\Delta/\sqrt{\sigma_1^2 + \sigma_2^2})$. In part (ii) the variable $Z = (Y_1 - Y_2)/(\sigma\sqrt{2})$ is standard normal, so $\mathbb{P}(|Z| > x) = 2\Phi(-x)$ by symmetry, which gives the tail probability with $x = \delta/(\sigma\sqrt{2})$. The q-quantile of $|Z|$ is $z_{(1+q)/2}$, and the quantiles of $|Y_1 - Y_2|$ are $\sigma\sqrt{2}$ times those of $|Z|$. ∎

With $\delta = \sigma$ the tail probability is $2\Phi(-1/\sqrt{2}) \approx 0.48$: two runs of one recipe differ by more than a standard deviation about half the time. With $\Delta = \sigma$, one run of each ranks two recipes wrongly with probability $\Phi(-1/\sqrt{2}) \approx 0.24$. A comparison by single runs is informative only when the gap is several times σ. The outcome of a classifier is moreover a measurement on a finite test set, which adds a second source of noise.

Proposition 1.5 (Two sources of noise in a measured error). *Let a run R be drawn from the run distribution and test items $\mathbf{Z} = (Z_1, \dots, Z_m)$ be drawn from the data distribution, independently of each other and of R. Write $e_R(z) \in \{0, 1\}$ for the indicator that the network of run R errs on item z, $p(R) = \mathbb{E}[e_R(Z_1) \mid R]$ for its population error, $q(z) = \mathbb{E}\, e_R(z)$ for the*

probability that a fresh run errs on item z, and $\bar{p} = \mathbb{E}\, p(R) = \mathbb{E}\, q(Z_1)$. The measured error $\hat{p} = \frac{1}{m}\sum_{i=1}^{m} e_R(Z_i)$ satisfies the following.

(i) $\mathbb{E}\hat{p} = \bar{p}$ *and* $\operatorname{Var}\hat{p} = \operatorname{Var} p(R) + \mathbb{E}[p(R)(1-p(R))]/m$.

(ii) For a test set held fixed while the runs vary, the variance over runs has mean

$$\mathbb{E}\big[\operatorname{Var}(\hat{p} \mid \mathbf{Z})\big] = \frac{m-1}{m}\operatorname{Var} p(R) + \frac{1}{m}\mathbb{E}\big[q(Z_1)\big(1-q(Z_1)\big)\big]. \quad (1.1)$$

(iii) $\bar{p}(1-\bar{p}) = \mathbb{E}[q(Z_1)(1-q(Z_1))] + \operatorname{Var} q(Z_1)$*; in particular* $\mathbb{E}[q(Z_1)(1-q(Z_1))] \le \bar{p}(1-\bar{p})$*, with equality if and only if* $q(Z_1) = \bar{p}$ *almost surely.*

Proof. The plan is to condition on the run for part (i), to derive part (iii) by algebra, and to obtain part (ii) from both by conditioning on the test items.

Part (i). Given R, the indicators $e_R(Z_1), \dots, e_R(Z_m)$ are independent Bernoulli variables with mean $p(R)$, because the items are independent of each other and of R. Hence $\mathbb{E}[\hat{p} \mid R] = p(R)$ and $\operatorname{Var}(\hat{p} \mid R) = p(R)(1-p(R))/m$. Taking expectations gives $\mathbb{E}\hat{p} = \bar{p}$, and the law of total variance gives the formula for $\operatorname{Var}\hat{p}$.

Part (iii). Since $q(1-q) = q - q^2$, we have $\mathbb{E}[q(Z_1)(1-q(Z_1))] = \bar{p} - \mathbb{E}\, q(Z_1)^2 = \bar{p} - \bar{p}^2 - \operatorname{Var} q(Z_1)$, which is the identity; the inequality and its case of equality follow because $\operatorname{Var} q(Z_1) \ge 0$, with equality exactly when $q(Z_1)$ is almost surely constant.

Part (ii). Given the items, $\hat{p}$ has mean $\frac{1}{m}\sum_i q(Z_i)$, an average of m independent copies of $q(Z_1)$, whose variance is $\operatorname{Var} q(Z_1)/m$. The law of total variance, now conditioning on the items, gives $\mathbb{E}[\operatorname{Var}(\hat{p} \mid \mathbf{Z})] = \operatorname{Var}\hat{p} - \operatorname{Var} q(Z_1)/m$. Inserting part (i), in which $\mathbb{E}[p(R)(1-p(R))] = \bar{p} - \mathbb{E}\, p(R)^2 = \bar{p}(1-\bar{p}) - \operatorname{Var} p(R)$, gives

$$\mathbb{E}\big[\operatorname{Var}(\hat{p} \mid \mathbf{Z})\big] = \operatorname{Var} p(R) + \frac{\bar{p}(1-\bar{p}) - \operatorname{Var} p(R) - \operatorname{Var} q(Z_1)}{m}.$$

By part (iii), $\bar{p}(1-\bar{p}) - \operatorname{Var} q(Z_1) = \mathbb{E}[q(Z_1)(1-q(Z_1))]$, and collecting the two terms in $\operatorname{Var} p(R)$ gives (1.1). ∎

Part (i) is the noise of the test set: even a fixed network measures with standard error $\sqrt{\bar{p}(1-\bar{p})/m}$, about 0.7 points for an error of 3% on 539 items. Part (ii) says what a spread over seeds on a fixed benchmark measures: the variance of the population error plus the *item churn* $\mathbb{E}[q(1-q)]/m$, the variance due to runs that err on different items. By part (iii) the churn lies between zero, when every run errs on the same items, and the full binomial variance, when every item is equally hard; the rest of the binomial variance, $\operatorname{Var} q(Z_1)/m$, is *item difficulty*, which moves all runs together when the test set changes. A seed standard deviation measured on a small test set can therefore be mostly a property of the test set, as Lab 1.3 finds.

1.5 The method: selection, components and the number of seeds

The best of k. A reported number is often the best of several runs: of a few restarts, of the seeds tried during development, of a few submissions. Its expectation is not the mean of the recipe.

Proposition 1.6 (The best of k runs). *Let $k \geq 1$, and let $e_k = \mathbb{E} \max_{i \leq k} Z_i$ for independent standard normals $Z_1, \dots, Z_k$.*

(i) If $Y_1, \dots, Y_k \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$, then $\mathbb{E} \max_i Y_i = \mu + \sigma e_k$ and $\mathbb{E} \min_i Y_i = \mu - \sigma e_k$. Moreover $e_1 = 0$, the sequence e_k increases strictly, and $e_k \leq \sqrt{2 \ln k}$.

(ii) If $(D_1, T_1), \dots, (D_k, T_k)$ are independent pairs, each bivariate normal with means μ_D, μ_T, standard deviations $\sigma_D, \sigma_T > 0$ and correlation ρ, and ι is the index of the largest D_i, then $\mathbb{E} T_\iota = \mu_T + \rho \, \sigma_T e_k$.

Proof. The plan is to reduce part (i) to standard normals and bound e_k through the moment generating function, and then to derive part (ii) from part (i) by conditioning on the D_i.

Part (i). Write $Y_i = \mu + \sigma Z_i$ with Z_i independent standard normals. Since $\sigma \geq 0$, $\max_i Y_i = \mu + \sigma \max_i Z_i$, which gives the first formula; and $\min_i Y_i = \mu - \sigma \max_i(-Z_i)$, where the $-Z_i$ are again independent standard normals, which gives the second. For $k = 1$, $e_1 = \mathbb{E} Z_1 = 0$. The maximum of $Z_1, \dots, Z_{k+1}$ is at least that of $Z_1, \dots, Z_k$, and strictly larger on the event that Z_{k+1} exceeds the others, which has probability $1/(k+1) > 0$; so $e_{k+1} > e_k$. For the bound, let $k \geq 2$ and $t > 0$. Jensen's inequality for the convex exponential, followed by bounding the largest of k positive numbers by their sum, gives

$$e^{t e_k} \leq \mathbb{E} \, e^{t \max_i Z_i} \leq \sum_{i=1}^{k} \mathbb{E} \, e^{t Z_i} = k \, e^{t^2/2}.$$

Taking logarithms, $e_k \leq (\ln k)/t + t/2$, and the choice $t = \sqrt{2 \ln k}$, which minimizes the right side, gives $e_k \leq \sqrt{2 \ln k}$. For $k = 1$ the bound reads $0 \leq 0$.

Part (ii). The pairs are independent, so given $D_1, \dots, D_k$ the variable T_i depends only on D_i, and the regression of a bivariate normal gives $\mathbb{E}[T_i \mid D_1, \dots, D_k] = \mu_T + \rho(\sigma_T/\sigma_D)(D_i - \mu_D)$. The index ι is a function of the D_i, unique with probability one, so $\mathbb{E}[T_\iota \mid D_1, \dots, D_k] = \mu_T + \rho(\sigma_T/\sigma_D)(\max_i D_i - \mu_D)$. Taking expectations and applying part (i) to the D_i gives $\mathbb{E} T_\iota = \mu_T + \rho(\sigma_T/\sigma_D)\sigma_D e_k = \mu_T + \rho \sigma_T e_k$. ∎

Table 1.1 lists e_k, and Exercise 1.1 derives $e_2 = 1/\sqrt{\pi}$ and $e_3 = 3/(2\sqrt{\pi})$. The constant grows slowly, like $\sqrt{2 \ln k}$ (Exercise 1.2), but it grows: the best of 5 runs sits 1.16σ above the mean, the best of 625 runs 3.10σ. Part (i) is the bonus of reporting the best run on the items that score it, and the bonus is luck. Part (ii) is the honest procedure, choosing on development data

k	2	3	4	5	6	7	8	9	10
e_k	0.564	0.846	1.029	1.163	1.267	1.352	1.424	1.485	1.539
$\sqrt{2\ln k}$	1.177	1.482	1.665	1.794	1.893	1.973	2.039	2.096	2.146

Table 1.1. The expected maximum e_k of k independent standard normals (numerical integration in `01-seeds-bestof.py`) and the bound of Proposition 1.6; further, $e_{20} = 1.867$, $e_{50} = 2.249$, $e_{100} = 2.508$, $e_{10^4} = 3.852$.

and scoring once on test data: if the two scores share a real component, then the selected run is truly better, by $\rho\sigma_T e_k$, the fraction ρ of the apparent gain; Reimers and Gurevych's correlation of 0.23 between development and test scores says that most of what their best development run gained was luck. The same order statistic governs the best of k hyperparameter trials (Chapter 4) and selection by a cheap proxy (Chapter 5); its asymptotics are in Proposition A.9.

Failed runs. The normal model breaks when some runs fail. If a run fails with probability π, landing near the score of a trivial predictor, then the mean mixes failures with working runs, the standard deviation is inflated by the gap between them, and the median remains the score of a working run as long as $\pi < 1/2$. The best of k runs fails only with probability π^k, which is why restarts worked for BERT, and why a best-of-k number hides the failure rate, as the numbers for RTE in Section 1.3 show. Report the failure rate with an exact binomial interval (Proposition A.17), summarize the working runs separately, and show every run.

Variance components. Which seeds produce the spread? A *crossed design* answers this: draw I initialization seeds and J data-order seeds, and train one run for every combination. Write Y_{ij} for the outcome of the run with the ith initialization and the jth order seed, $\bar{Y}_{i\cdot} = \frac{1}{J}\sum_j Y_{ij}$ and $\bar{Y}_{\cdot j} = \frac{1}{I}\sum_i Y_{ij}$ for the row and column means, and $\bar{Y}_{\cdot\cdot}$ for the grand mean. The mean squares of the table are

$$\begin{aligned} \mathrm{MS}_A &= \frac{J}{I-1}\sum_{i=1}^{I}(\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot})^2, \qquad \mathrm{MS}_B = \frac{I}{J-1}\sum_{j=1}^{J}(\bar{Y}_{\cdot j} - \bar{Y}_{\cdot\cdot})^2, \\ \mathrm{MS}_E &= \frac{1}{(I-1)(J-1)}\sum_{i=1}^{I}\sum_{j=1}^{J}(Y_{ij} - \bar{Y}_{i\cdot} - \bar{Y}_{\cdot j} + \bar{Y}_{\cdot\cdot})^2. \end{aligned} \tag{1.2}$$

The textbook model behind them, $Y_{ij} = \mu + a_i + b_j + e_{ij}$ with independent normal effects, is an assumption. For seeds it is not needed: a deterministic training pipeline is a function of its seeds, and every such function has an exact decomposition of the same form.

Lemma 1.7 (Centred sums of squares). *If $X_1, \dots, X_n$, with $n \geq 2$, are pairwise uncorrelated with a common mean and a common variance τ^2, then $\mathbb{E}\sum_{i=1}^{n}(X_i - \bar{X})^2 = (n-1)\tau^2$.*

Proof. Let ν be the common mean. Expanding around ν gives $\sum_i (X_i - \bar{X})^2 = \sum_i (X_i - \nu)^2 - n(\bar{X} - \nu)^2$. The first sum has mean $n\tau^2$, and $\bar{X} - \nu$ has variance τ^2/n because the X_i are pairwise uncorrelated, so the expectation is $n\tau^2 - \tau^2$. ∎

Theorem 1.8 (Variance components of a crossed design). *Let $U_1, \dots, U_I$ and $V_1, \dots, V_J$, with $I, J \geq 2$, be independent random seeds, the U_i identically distributed and the V_j identically distributed, and let $Y_{ij} = f(U_i, V_j)$ for a function f with $\mathbb{E} f(U_1, V_1)^2 < \infty$. Let $\mu = \mathbb{E} Y_{11}$, and define the main effects $a(u) = \mathbb{E} f(u, V_1) - \mu$ and $b(v) = \mathbb{E} f(U_1, v) - \mu$, the interaction $c(u, v) = f(u, v) - \mu - a(u) - b(v)$, and*

$$\sigma_a^2 = \operatorname{Var} a(U_1), \qquad \sigma_b^2 = \operatorname{Var} b(V_1), \qquad \sigma_e^2 = \operatorname{Var} c(U_1, V_1).$$

Then $\operatorname{Var} Y_{11} = \sigma_a^2 + \sigma_b^2 + \sigma_e^2$, and the mean squares (1.2) satisfy

$$\mathbb{E}\mathrm{MS}_A = \sigma_e^2 + J\sigma_a^2, \qquad \mathbb{E}\mathrm{MS}_B = \sigma_e^2 + I\sigma_b^2, \qquad \mathbb{E}\mathrm{MS}_E = \sigma_e^2.$$

Consequently $\hat{\sigma}_a^2 = (\mathrm{MS}_A - \mathrm{MS}_E)/J$, $\hat{\sigma}_b^2 = (\mathrm{MS}_B - \mathrm{MS}_E)/I$ and $\hat{\sigma}_e^2 = \mathrm{MS}_E$ are unbiased estimators of σ_a^2, σ_b^2 and σ_e^2.

Proof. The plan is to show that the pieces $a(U_i)$, $b(V_j)$ and $c(U_i, V_j)$ are pairwise uncorrelated, and then to compute each mean square from them with Lemma 1.7. Write $a_i = a(U_i)$, $b_j = b(V_j)$ and $c_{ij} = c(U_i, V_j)$, and use dots for averages over an index, as in $\bar{c}_{i\cdot} = \frac{1}{J}\sum_j c_{ij}$.

The pieces are orthogonal. Because the seeds are independent, fixing $U_i = u$ leaves the law of the other seeds unchanged. Averaging c over either seed gives zero: for each u, $\mathbb{E}\, c(u, V_1) = \mathbb{E} f(u, V_1) - \mu - a(u) - \mathbb{E}\, b(V_1) = 0$, since $\mathbb{E}\, b(V_1) = \mathbb{E} f(U_1, V_1) - \mu = 0$; symmetrically $\mathbb{E}\, c(U_1, v) = 0$ for each v. Hence a_i, b_j and c_{ij} have mean zero, and they are square integrable because conditional expectations do not increase second moments. Two of them that involve disjoint sets of seeds are independent, hence uncorrelated. Otherwise the two share exactly one seed, say U_i (a shared V_j is symmetric), and at least one of them is a c. Given $U_i = u$ they are independent, and the c among them, $c(u, V_j)$, has mean zero; so the conditional mean of their product is zero, and so is its expectation. In particular $\operatorname{Var} Y_{11} = \operatorname{Var}(a_1 + b_1 + c_{11}) = \sigma_a^2 + \sigma_b^2 + \sigma_e^2$.

Rows and columns. The row mean is $\bar{Y}_{i\cdot} = \mu + a_i + \bar{b}_\cdot + \bar{c}_{i\cdot}$, and $\bar{b}_\cdot$ is the same for every row; so $\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot} = X_i - \bar{X}$ with $X_i = a_i + \bar{c}_{i\cdot}$. By orthogonality the X_i are pairwise uncorrelated, with mean zero and variance $\sigma_a^2 + \sigma_e^2/J$, so Lemma 1.7 gives $\mathbb{E}\sum_i (\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot})^2 = (I-1)(\sigma_a^2 + \sigma_e^2/J)$, and multiplying by $J/(I-1)$ gives $\mathbb{E}\mathrm{MS}_A = J\sigma_a^2 + \sigma_e^2$. Exchanging the roles of rows and columns gives $\mathbb{E}\mathrm{MS}_B = I\sigma_b^2 + \sigma_e^2$.

Residuals. Substituting the decomposition into the residual, the terms in μ, a and b cancel:

$$r_{ij} = Y_{ij} - \bar{Y}_{i\cdot} - \bar{Y}_{\cdot j} + \bar{Y}_{\cdot\cdot} = c_{ij} - \bar{c}_{i\cdot} - \bar{c}_{\cdot j} + \bar{c}_{\cdot\cdot}.$$

The table $c_{ij} - \bar{c}_{\cdot\cdot}$ is the sum of three tables, with entries $\bar{c}_{i\cdot} - \bar{c}_{\cdot\cdot}$, $\bar{c}_{\cdot j} - \bar{c}_{\cdot\cdot}$ and r_{ij}, which are orthogonal in $\mathbb{R}^{I\times J}$: the residuals sum to zero along every row

and every column, the first table depends only on the row and the second only on the column, and each of these two sums to zero. By Pythagoras,

$$\sum_{i,j}(c_{ij}-\bar{c}_{\cdot\cdot})^2 = J\sum_i(\bar{c}_{i\cdot}-\bar{c}_{\cdot\cdot})^2 + I\sum_j(\bar{c}_{\cdot j}-\bar{c}_{\cdot\cdot})^2 + \sum_{i,j} r_{ij}^2.$$

We apply Lemma 1.7 three times: to the IJ variables c_{ij}, of variance σ_e^2; to the I row means $\bar{c}_{i\cdot}$, of variance σ_e^2/J; and to the J column means, of variance σ_e^2/I, all pairwise uncorrelated by orthogonality. Taking expectations,

$$\mathbb{E}\sum_{i,j} r_{ij}^2 = (IJ-1)\sigma_e^2 - (I-1)\sigma_e^2 - (J-1)\sigma_e^2 = (I-1)(J-1)\sigma_e^2,$$

which is $\mathbb{E}\mathrm{MS}_E = \sigma_e^2$. The three estimators are linear in the mean squares, so their unbiasedness follows. ∎

Three remarks make the theorem usable. *Nothing about training is assumed* beyond independent seeds; independence and normality of the effects are needed only for F-tests and exact intervals (Theorem A.12). *One run per cell cannot separate* the interaction from noise that is fresh in every run, such as that of nondeterministic kernels (Chapter 2); separating them takes two runs per cell. *Estimates can be negative*, when $\mathrm{MS}_A < \mathrm{MS}_E$; report them as computed, since rounding them up to zero biases them. Fixing a seed leaves a conditional spread: runs that share the initialization seed spread with standard deviation $\sqrt{\sigma_b^2+\sigma_e^2}$, runs that share the order seed with $\sqrt{\sigma_a^2+\sigma_e^2}$.[7] With splits as rows and runs within a split as replicates, the same analysis gives the split component.

[7] Dodge et al. summarized their crossed design by such conditional spreads, averaged over seeds: their "standard deviation aggregated over weight initializations" estimates roughly $\sqrt{\sigma_b^2+\sigma_e^2}$.

How many seeds. The last question is the one an experimenter faces first: how many runs does a comparison need?

Proposition 1.9 (Power of a comparison of means). *Let each of two recipes be run n times, all runs independent, with outcomes normal with means μ_1, μ_2 and a common known standard deviation $\sigma > 0$, and let $\Delta = \mu_1 - \mu_2$. The two-sided test that rejects $\Delta = 0$ when $|\bar{Y}_1 - \bar{Y}_2| > z_{1-\alpha/2}\,\sigma\sqrt{2/n}$ has level α and power*

$$\Phi\Big(\frac{\sqrt{n}\,|\Delta|}{\sigma\sqrt{2}} - z_{1-\alpha/2}\Big) + \Phi\Big(-\frac{\sqrt{n}\,|\Delta|}{\sigma\sqrt{2}} - z_{1-\alpha/2}\Big).$$

If instead the runs are paired, the ith runs of the two recipes sharing their seeds and split, with the pairs independent and bivariate normal and $\sigma_D^2 = \mathrm{Var}(Y_{1i} - Y_{2i}) > 0$, then the same holds with $2\sigma^2$ replaced by σ_D^2, for the test that rejects when $|\bar{Y}_1 - \bar{Y}_2| > z_{1-\alpha/2}\,\sigma_D/\sqrt{n}$.

Proof. Write $s = \sigma\sqrt{2/n}$. The difference $\bar{Y}_1 - \bar{Y}_2$ is normal with mean Δ and standard deviation s, so the rejection probability is

$$\mathbb{P}(\bar{Y}_1-\bar{Y}_2 > z_{1-\alpha/2}s)+\mathbb{P}(\bar{Y}_1-\bar{Y}_2 < -z_{1-\alpha/2}s) = \Phi\Big(\frac{\Delta}{s}-z_{1-\alpha/2}\Big)+\Phi\Big(-\frac{\Delta}{s}-z_{1-\alpha/2}\Big),$$

which is symmetric in the sign of Δ and equals the displayed power. At $\Delta = 0$ it is $2\Phi(-z_{1-\alpha/2}) = \alpha$. In the paired case $\bar{Y}_1 - \bar{Y}_2$ is the mean of n independent normal differences of variance σ_D^2, and the same computation applies with $s = \sigma_D/\sqrt{n}$. ■

Corollary 1.10 (Number of seeds). *In the setting of Proposition 1.9, the power is at least* 0.8 *whenever*

$$n \geq \frac{2\sigma^2 \, (z_{1-\alpha/2} + z_{0.8})^2}{\Delta^2},$$

which for $\alpha = 0.05$ *is* $n \geq 15.7\,\sigma^2/\Delta^2$ *runs per arm. For paired runs,* $2\sigma^2$ *is replaced by* σ_D^2.

Proof. The second term of the power is positive, and the first is at least 0.8 exactly when $\sqrt{n}|\Delta|/(\sigma\sqrt{2}) - z_{1-\alpha/2} \geq z_{0.8}$; squaring and rearranging gives the condition on n. With $z_{0.975} = 1.960$ and $z_{0.8} = 0.842$, the constant $2(z_{0.975} + z_{0.8})^2$ is 15.70. ■

To detect a gain of one seed standard deviation takes 16 runs per arm, of half a standard deviation 63. A gain of 0.3 points at a standard deviation of 0.2 takes 7 runs per arm by the corollary and 9 by an exact t-test (Exercise 1.3, Proposition A.14); the taggers of Section 1.3, with a gap of 0.27 and a standard deviation of 0.24, needed 13 each. *Pairing* helps as much as the arms share their noise, since $\sigma_D^2 = \sigma_1^2 + \sigma_2^2 - 2\,\mathrm{Cov}(Y_1, Y_2)$: a shared split drops out of the difference, and so does the difficulty of shared test items, since only items on which two networks disagree contribute to it (Proposition A.5).

1.6 In the laboratory: one recipe, eighteen hundred runs

All three labs use one recipe: a two-layer perceptron of width 64 (8,970 parameters) on the scikit-learn digits, trained with SGD and momentum 0.9 at learning rate 0.05, batch size 64, for 20 epochs. A split puts 1,258 images in the training set and 539 in the test set, so that one misclassified digit is 0.19 points of test error.

lab 1.1 · Two hundred runs of one recipe

script
`01-seeds-spread.py`

Setup. Two hundred runs; run r uses seed r for initialization, data order and split. About 15 seconds.

Predict first. How many test items will separate the best and the worst run? After how many runs will the 95% interval be narrower than one test item?

Look for. The shape of the distribution and its extremes; the running mean inside an interval that narrows like $1/\sqrt{j}$; the fraction of pairs of runs more than one standard deviation apart (0.48 by

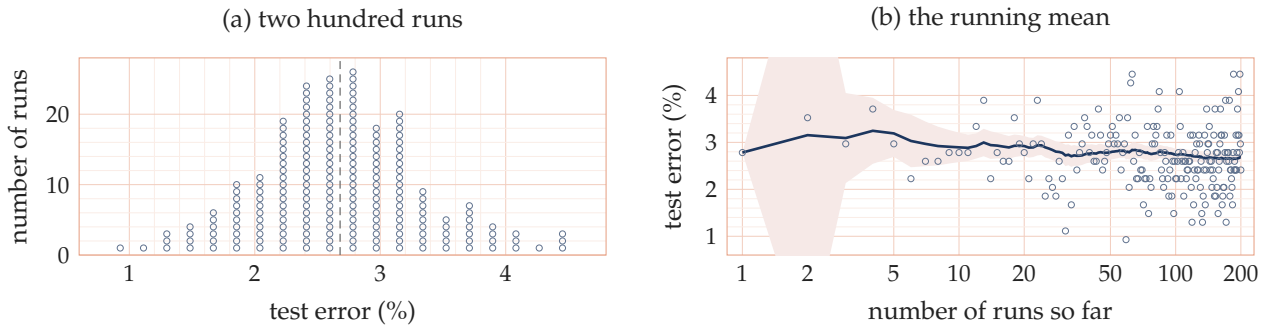


Figure 1.1. Lab 1.1: 200 runs of one recipe (two-layer MLP on the digits, 539 test images), each with new seeds for initialization, data order and split (seeds 0–199). (a) Each hollow mark is one run, runs with equal error stacked; the dashed line is the mean, 2.68%. (b) The runs in the order they were made (hollow marks), the running mean (line) and its 95% t-interval (band); the band for two runs, $[-1.6, 7.9]$, is cut off by the frame.

Proposition 1.4).

WHAT HAPPENED. The 200 runs (Figure 1.1a) have mean test error 2.68 ± 0.09 (95% t-interval), standard deviation 0.65 points (95% interval 0.60–0.73) and range 0.93% to 4.45%: nineteen test items separate the best and the worst run of one recipe. These are ordinary extremes; for 200 normal draws, Proposition 1.6 with $e_{200} = 2.75$ predicts 0.88% and 4.48%. Of 100 disjoint pairs of runs, 50 differ by more than one standard deviation (exact 95% interval 40–60) against 48 predicted by Proposition 1.4, with median difference 0.65 points against 0.62: two copies of this recipe, one run each, would show a gap of more than half a point about half the time.

Panel (b) is the ornament of the chapter openings with real data: after two runs the 95% interval is $[-1.6, 7.9]$; after sixteen, all the draws of the ornament, 2.91 ± 0.26; after 43 runs its half-width drops below one test item; after 200, 2.68 ± 0.09, narrower than at sixteen by a factor of 2.8 rather than $\sqrt{200/16} = 3.5$, because the first sixteen runs happened to spread less (standard deviation 0.48). By Proposition 1.5(i), a fixed network scored on a random set of 539 items measures with standard deviation 0.70 points at this error, or 0.58 for test sets drawn without replacement from the 1,797 digits: most of the spread of these runs belongs to the test set.

LAB 1.2 · WHERE THE SPREAD COMES FROM

SCRIPT
`01-seeds-components.py`

SETUP. Crossed design: split 0, initialization seeds 0–19 crossed with data-order seeds 0–19 (400 runs). Nested design: 50 further splits, 8 runs on each with new initialization and order seeds (400 runs). Estimators of Theorem 1.8 and their one-way analogue for the split; intervals from a parametric bootstrap of the fitted normal random-effects model. About a minute.

PREDICT FIRST. Which moves the test error more in this network, the

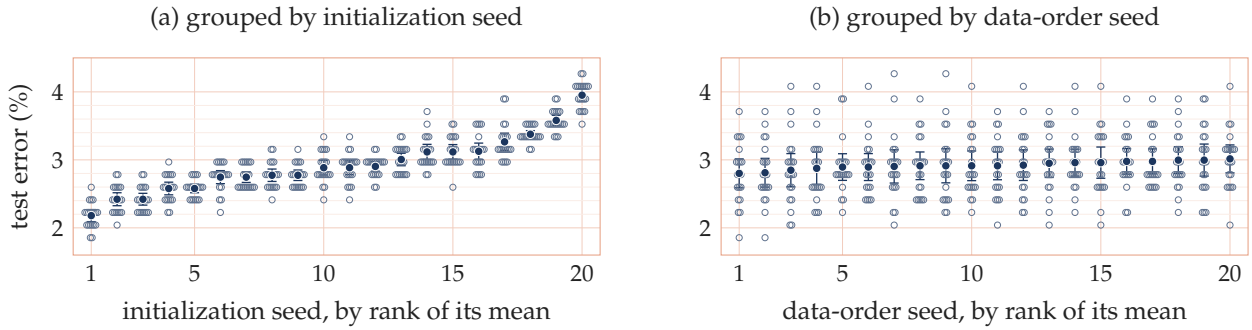


Figure 1.2. Lab 1.2, crossed design: 400 runs on split 0, every combination of 20 initialization seeds and 20 data-order seeds. The same runs (hollow marks; equal errors side by side) grouped (a) by initialization seed and (b) by data-order seed, each group sorted by its mean; filled marks are group means with 95% t-intervals over the 20 runs of the group.

initial weights or the order of the minibatches? Will a new split move it more or less than new seeds?

Look for. Whether the means of the initialization seeds spread out more than those of the order seeds; the interaction; the split component against the binomial noise of 539 items and its part due to item difficulty.

What happened. The initialization seed carries 81% of the seed variance on this split: $\hat{\sigma}_a = 0.42$ points (95% interval 0.29–0.55), against $\hat{\sigma}_b = 0.04$ (0.00–0.06) for the data order and $\hat{\sigma}_e = 0.20$ (0.18–0.21) for their interaction. The means of the initialization seeds range from 2.18% to 3.95%, those of the order seeds only from 2.80% to 3.02% (Figure 1.2), and the F-test finds the first effect beyond doubt ($p \approx 10^{-125}$) and the second barely ($p = 0.02$). The order matters only in combination with an initialization: runs that share the initialization seed spread with standard deviation 0.20, runs that share the order seed with 0.46. This differs from BERT fine-tuning, where Dodge et al. (2020) found the two seeds comparable, but there the initialization seed set only the 2,048 weights of a new classifier layer, and here it sets all 8,970; the ordering is also consistent with the finding of Frankle et al. (2020) that small networks on MNIST are stable to the noise of SGD from initialization on, an explanation we have not tested.[8]

Is the initialization effect a property of the network or of these 539 items? On two halves of the test items, the means of the 20 initialization seeds correlate at only 0.15 (95% interval −0.32 to 0.55); Lab 1.3 settles the question. A new split moves the error about as much as new seeds do: 0.44 points (0.35–0.53) between splits against 0.38 (0.35–0.41) within them. The binomial noise of 539 items is 0.73 points at this error (0.61 for the finite pool), and by Proposition 1.5(iii) its part that a new split moves and new seeds do not, item difficulty, is 0.52 points after the finite-pool correction, inside the interval for the split: a new split acts almost entirely through which digits land in the test set, as Bouthillier et al. (2021) found for data sampling at

[8] Stability to SGD noise means that two runs from the same weights with different data orders end in the same linearly connected region of weights; Chapter 2 takes it up.

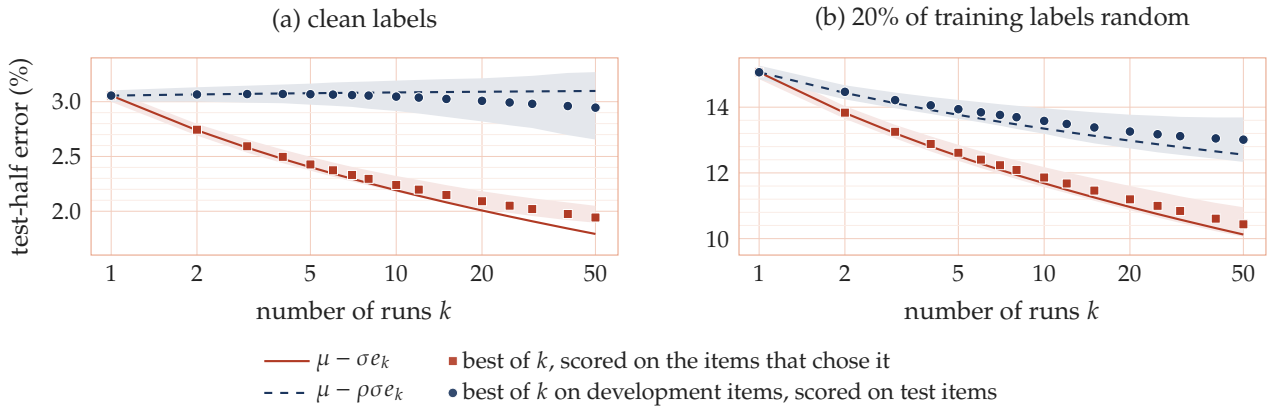


Figure 1.3. Lab 1.3: the best of k runs, 400 runs per panel (split 0, seeds 0–399), errors on a test half of 269 images. Squares: expected lowest test-half error among k runs; circles: expected test-half error of the run with the lowest error on the other 270 images; both exact averages over all k-subsets, with 95% bootstrap bands over runs. Lines: the predictions of Proposition 1.6 from the sample mean μ, standard deviation σ and development–test correlation ρ of each panel.

larger scale.

lab 1.3 · The best of k

script
`01-seeds-bestof.py`

Setup. Split 0, 400 runs with new initialization and order seeds, once on clean labels and once with 20% of the training labels replaced by random labels. The 539 test images are divided at random into a development half (270) and a test half (269). For each $k \le 50$, exact averages over all k-subsets of the 400 runs of the best test-half error, and of the test-half error of the run with the best development-half error (ties broken at random). About 70 seconds.

Predict first. On clean labels, how much lower is the best of 10 runs than a typical run? How much of that survives when the best run is chosen on the development half and scored on the test half? And with noisy labels?

Look for. The best-of-k curve against $\mu - \sigma e_k$, and where it departs; the honest curve against $\mu - \rho\sigma e_k$; the correlation ρ between development and test errors in the two settings.

What happened. On clean labels (Figure 1.3a) the test-half error has mean 3.06 ± 0.06 and standard deviation 0.56 points. The best of k runs scored on the test half follows $\mu - \sigma e_k$: for $k = 10$ it is 2.24% (95% band 2.18–2.32) against 2.19% predicted, 0.82 points below a typical run; at $k = 50$ it falls short of the prediction (1.94% against 1.79%), because the errors are skewed to the right (skewness 0.23) and cannot fall below the few digits every run gets wrong. Chosen on the development half, the best of 10 runs scores 3.05% (2.92–3.19) on the test half: nothing of the 0.82 points survives, because a run's errors on the two halves are uncorrelated ($\rho = -0.03$, 95%

interval −0.13 to 0.07). Item churn makes up 91% of the seed variance on all 539 items (80–104%): on clean digits the seed decides which borderline digits a network gets wrong, not how many it would get wrong on new digits, and the initialization effect of Lab 1.2 is an effect on which items fail.

With 20% of the training labels random (Figure 1.3b) the runs differ for real: mean 15.06 ± 0.22, standard deviation 2.20 points, $\rho = 0.51$ (0.43–0.58), and item churn only 37% of the seed variance (32–43%). The best of 10 runs scores 11.86% on the items that chose it (11.69% predicted) and 13.58% (13.19–13.97) when chosen on the development half (13.35% predicted by $\mu - \rho\sigma e_k$). At every k the honest gain is 44–48% of the apparent gain, close to ρ. Restarts selected on held-out data buy a real improvement when runs differ in what they learned, as they did for BERT; scored on the items that selected them, they report that improvement plus a bonus of luck.

The miniature reproduced the post-mortems, and differed from them in one respect: here the initialization seed outweighs the data-order seed.

1.7 Protocol

PROTOCOL 1.1 · MEASURING ONE RECIPE

- ☐ Decide which sources of randomness the claim must hold across (initialization, data order, augmentation and dropout, split, hardware); randomize those, fix the rest, and record every seed.
- ☐ Fix the number of runs before looking, from a pilot standard deviation and the smallest difference worth detecting (Corollary 1.10); never fewer than five for a description.
- ☐ Report the mean with a 95% interval, the standard deviation, the number of runs and every run, in a dot plot or a table; report failed runs and the rule that defines them separately.
- ☐ Never report the best of k runs without k and the mean; when a run must be selected, select it on development data and score it once on test data.
- ☐ Compare recipes on the same splits, seeds and test items, and test the difference across runs, not only across test items.
- ☐ Compute the binomial noise of the test set and, from per-item predictions, its churn; if they are comparable to the effect under study, then the test set is too small for the claim.
- ☐ When the spread matters, estimate where it comes from with a crossed design before choosing which seeds to vary.

The protocol makes the measurement of one recipe honest; it does not make a comparison fair. Two well-measured recipes can still differ in their tuning budgets (Chapter 4) or in more than the component a claim is about (**??**), be compared on many tasks at once (Chapter 3), or behave differently on other hardware (Chapter 2). And at 16 runs per arm to detect a gain of

one standard deviation, the protocol is out of reach for the largest models. Then measure the spread where runs are affordable (a smaller model, fewer steps, a subset of the data) and say so, use paired designs, and when only one run is possible, report it as one run beside the seed standard deviation of a comparable setting, with a provisional conclusion.

Exercises

1.1 consolidation. (Expected maxima.) (a) Show that $e_2 = 1/\sqrt{\pi}$, using $\max(a,b) = \frac{1}{2}(a+b) + \frac{1}{2}|a-b|$, and that $e_3 = 3/(2\sqrt{\pi})$. (b) Show that $e_k = \int z k \varphi(z) \Phi(z)^{k-1}\,\mathrm{d}z$, and check Table 1.1 numerically. (c) Show that the expected range of k independent $\mathcal{N}(\mu,\sigma^2)$ draws is $2\sigma e_k$, and check the CIFAR-10 range of Picard (2021). (d) For runs uniform on $[\mu-h, \mu+h]$, show that the best of k has mean $\mu + h(k-1)/(k+1)$, and compare its growth with that of e_k; which run distributions make best-of-k reports most misleading? *(Hint: for e_3, integrate $3\int z\varphi\Phi^2\,\mathrm{d}z$ by parts, then use $\Phi(z)+\Phi(-z)=1$ and $\int \varphi^2 = 1/(2\sqrt{\pi})$.)*

1.2 consolidation. ★ (The growth of e_k.) Prove that $e_k/\sqrt{2\ln k} \to 1$. *(Hint: for the lower bound, with $M = \max_i Z_i$ and $t = (1-\varepsilon)\sqrt{2\ln k}$, show $e_k \ge t\,\mathbb{P}(M \ge t) - \mathbb{E}|Z_1|$, and use $1-\Phi(t) \ge \varphi(t)\,t/(1+t^2)$ to show $\Phi(t)^k \to 0$.)*

1.3 consolidation. (How many seeds.) A method is expected to gain 0.3 points over a baseline whose seed standard deviation is 0.2 points. (a) By Corollary 1.10, how many runs per arm give power 0.8 at level 0.05? (b) With the noncentral t distribution, show that a two-sample t-test needs 9 runs per arm. (c) Suppose the runs are paired by seed and split, and paired outcomes have correlation 0.6. How many pairs suffice by the corollary?

1.4 lab. (A smaller test set.) Modify `01-seeds-spread.py` to score every run on only the first 100 test images of its split. Predict the new standard deviation from Proposition 1.5(i) and Lab 1.1 before running it, then compare.

1.5 lab. (When does the order matter?) Rerun the crossed design of `01-seeds-components.py` with learning rate 0.2, then with batch size 16, then with Adam at learning rate 10^{-3}. Observe how $\hat{\sigma}_b$ and $\hat{\sigma}_e$ change relative to $\hat{\sigma}_a$, and relate the change to the stability of training to the noise of SGD (Chapter 2).

1.6 lab. (The size of the development set.) In `01-seeds-bestof.py` with noisy labels, use development sets of $d = 50$, 270 and 489 of the 539 test images and test sets of the rest. Predict how ρ changes with d, check it, and observe the honest best-of-10 error. When is selecting the best of k runs on development data worth its cost?

1.7 design. (Components of a fine-tuning pipeline.) A team fine-tunes a pretrained language model and can vary the pretraining checkpoint

(five are available, differing only in their pretraining seed), the initialization of the new head and the data order, with a budget of 300 runs. Design a study of the three main components: which interactions are estimable and which are confounded with what, how the runs are allocated, and how precisely five checkpoints determine their component.

1.8 DESIGN. (Paired or unpaired.) Two recipes are to be compared with 60 runs in all. A pilot gives a seed standard deviation of 0.5 points for each and a correlation of 0.7 between runs of the two recipes that share split, seeds and test items. Compare the power of paired and unpaired designs to detect a gain of 0.2 points, and list what the code must do for the pairing to be real.

1.9 CRITIQUE. (Best of five against single runs.) A paper's main table lists its method as "best of 5 seeds" next to baseline scores copied from the original papers, each a single run. The method leads by 0.4 points, and its own runs have a standard deviation of 0.3. What does the lead measure? Estimate the part due to selection with Proposition 1.6, and describe the table a reader needs.

1.10 CRITIQUE. (The best and the worst seed.) A study crosses 25 initialization seeds with 25 data orders, picks the initialization seeds with the highest and the lowest mean, and reports a two-sample t-test between their 25 runs each, $p < 0.01$, as evidence that some initializations are better than others. Suppose that all runs are independent with standard deviation σ, as when neither seed has an effect. Show that the expected gap between the best and the worst seed mean is $2e_{25}\sigma/5 \approx 0.79\sigma$ ($e_{25} = 1.965$), while the standard error of the test is $\sigma\sqrt{2/25} \approx 0.28\sigma$. Which test answers the question?

1.11 RESEARCH. (Intervals for bimodal run distributions.) When some runs fail, the mean is a poor summary and the t-interval loses its coverage. Open: a procedure that reports the failure rate and the mean of working runs jointly, with a failure rule that does not look at test data and with guaranteed coverage when failures are rare, at rates near $1/n$. Also open: a theory that predicts which seed dominates the spread (the initialization in Lab 1.2, neither for BERT fine-tuning) from the width, the learning rate and the fraction of weights a seed initializes.

Notes and References

The problem of this chapter is older than deep learning. Student (1908) derived the t distribution for the mean of a few measurements, and Tippett (1925) studied the extremes and the range of normal samples, whose mean is $2e_k$ in our notation. Theorem 1.8 is the classical result on expected mean squares in the balanced two-way layout (Searle et al. 1992), stated here for a function of independent seeds.

For deep networks, Reimers and Gurevych (2017) put the seed on the agenda of natural language processing, and Card et al. (2020) showed that many comparisons in the field are underpowered; in reinforcement learning, Henderson et al. (2018) (**??**) and Colas et al. (2018) raised the same questions, the latter with a power analysis for the number of seeds. Bouthillier et al. (2019) distinguished reproducing a result from reproducing a finding, and Gundersen et al. (2022) review the sources of irreproducibility in machine learning. Seeds change more than scores: they change the explanations of predictions (Madhyastha and Jain 2019) and generalization outside the training distribution (McCoy et al. 2020). That a best-of-k number depends on k is the starting point of Dodge et al. (2019) and of Chapter 4.

Translating the literature. Reimers and Gurevych (2017) report F1 as a fraction, so their $\sigma = 0.00241$ is our 0.24 points; their score distribution is our run distribution, and the "Evaluation 1" and "Evaluation 2" of their 2018 preprint are our one run per arm and best of k on development data. Dodge et al. (2020) call a run a trial, while in this book a trial is a hyperparameter configuration; their WI and DO seeds are our initialization and order seeds, their expected validation performance of the best of x trials is our $\mathbb{E}[M_x]$ on the validation set, and their standard deviation aggregated over WI seeds is the conditional spread $\sqrt{\sigma_b^2 + \sigma_e^2}$ of Theorem 1.8, averaged over seeds. What Picard (2021) calls the seed is our full set of seeds. In Bouthillier et al. (2021), data sampling is our split component, ξ_O collects our initialization, order and noise seeds, and numerical noise is the nondeterminism of Chapter 2.

References

Bouthillier, X., C. Laurent, and P. Vincent (2019). "Unreproducible Research is Reproducible". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 725–734.

Bouthillier, X. and G. Varoquaux (2020). *Survey of Machine-Learning Experimental Methods at NeurIPS2019 and ICLR2020*. Research Report hal-02447823. Inria Saclay Ile de France.

Bouthillier, X. et al. (2021). "Accounting for Variance in Machine Learning Benchmarks". In: *Proceedings of Machine Learning and Systems*. Vol. 3.

Card, D., P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky (2020). "With Little Power Comes Great Responsibility". In: *Proceedings of EMNLP 2020*, pp. 9263–9274.

Chiu, J. P. C. and E. Nichols (2016). "Named Entity Recognition with Bidirectional LSTM-CNNs". In: *Transactions of the Association for Computational Linguistics* 4, pp. 357–370.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2018). *How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments*. Preprint, arXiv:1806.08295.

Devlin, J., M.-W. Chang, K. Lee, and K. Toutanova (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of NAACL-HLT 2019*, pp. 4171–4186.

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2019). "Show Your Work: Improved Reporting of Experimental Results". In: *Proceedings of EMNLP-IJCNLP 2019*, pp. 2185–2194.

Dodge, J., G. Ilharco, R. Schwartz, A. Farhadi, H. Hajishirzi, and N. Smith (2020). *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. Preprint, arXiv:2002.06305.

Frankle, J., G. K. Dziugaite, D. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 3259–3269.

Gundersen, O. E., K. Coakley, C. Kirkpatrick, and Y. Gil (2022). *Sources of Irreproducibility in Machine Learning: A Review*. Preprint, arXiv:2204.07610.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger (2018). "Deep Reinforcement Learning That Matters". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1.

Lample, G., M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer (2016). "Neural Architectures for Named Entity Recognition". In: *Proceedings of NAACL-HLT 2016*, pp. 260–270.

Ma, X. and E. Hovy (2016). "End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF". In: *Proceedings of the 54th Annual Meeting of the ACL*, pp. 1064–1074.

Madhyastha, P. and R. Jain (2019). "On Model Stability as a Function of Random Seed". In: *Proceedings of CoNLL 2019*, pp. 929–939.

McCoy, R. T., J. Min, and T. Linzen (2020). "BERTs of a feather do not generalize together: Large variability in generalization across models with similar test set performance". In: *Proceedings of the Third BlackboxNLP Workshop*, pp. 217–227.

Mosbach, M., M. Andriushchenko, and D. Klakow (2021). "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations*.

Peters, M. E., M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer (2018). "Deep Contextualized Word Representations". In: *Proceedings of NAACL-HLT 2018*, pp. 2227–2237.

Picard, D. (2021). *torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision*. Preprint, arXiv:2109.08203.

Reimers, N. and I. Gurevych (2017). "Reporting Score Distributions Makes a Difference: Performance Study of LSTM-networks for Sequence Tagging". In: *Proceedings of EMNLP 2017*, pp. 338–348.

Reimers, N. and I. Gurevych (2018). *Why Comparing Single Performance Scores Does Not Allow to Draw Conclusions About Machine Learning Approaches*. Preprint, arXiv:1803.09578.

Searle, S. R., G. Casella, and C. E. McCulloch (1992). *Variance Components*. Wiley.

Student (1908). "The Probable Error of a Mean". In: *Biometrika* 6.1, pp. 1–25.

Tippett, L. H. C. (1925). "On the Extreme Individuals and the Range of Samples Taken from a Normal Population". In: *Biometrika* 17.3/4, pp. 364–387.

Tjong Kim Sang, E. F. and F. De Meulder (2003). "Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition". In: *Proceedings of CoNLL 2003*, pp. 142–147.

Zhang, T., F. Wu, A. Katiyar, K. Q. Weinberger, and Y. Artzi (2021). "Revisiting Few-sample BERT Fine-tuning". In: *International Conference on Learning Representations*.

2 Instability

Why a fixed seed does not fix a run

THE CLAIM *"Fixing the random seed makes a training run reproducible."*

Common practice in code and papers; framework documentation promises less

THE VERDICT Narrowed: a seed repeats a run only on an identical stack of code, kernels, and hardware, and where training is unstable, a one-bit change reproduces most of the seed-to-seed variability.

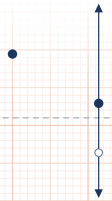
IN THIS CHAPTER. Setting every random seed is the standard recipe for a reproducible training run, and on one machine with deterministic kernels it does repeat a run bit for bit. Another GPU, library version, batch size, or order of summation can change the last bits of the arithmetic, and Summers and Dinneen (2021) found that moving one initial weight of a ResNet by the smallest representable amount produces as much run-to-run variability as all sources of randomness together. We prove how far a reordered floating-point sum can move, that training which contracts forgets such differences while training which expands amplifies them, and what a perturbed twin of a run measures about the run distribution. In the laboratory a fixed seed repeats a digits network exactly; a one-bit twin stays identical in function when the labels are clean and becomes largely a new draw when a fifth of them are noisy.

2.1 The claim: set every seed

Training code begins by seeding every random number generator it touches, and frameworks add switches for deterministic kernels: in PyTorch, one call makes operations with nondeterministic implementations use deterministic ones or raise an error, and a flag stops the convolution library from timing several algorithms and keeping the fastest (PyTorch Contributors 2026).[1] A paper then states that all seeds were fixed, and its reader takes the reported number to be a reproducible fact about the method.

The documentation promises less. PyTorch does not guarantee reproducible results across releases, commits, or platforms, nor between CPU and GPU with identical seeds, and warns that deterministic operations are often slower (PyTorch Contributors 2026). NVIDIA's matrix library cuBLAS repeats its results bit for bit only within one toolkit version, on GPUs of one architecture with the same number of multiprocessors (NVIDIA 2026a), and the convolution library cuDNN lists routines, among them algorithms for the backward pass of convolution and of max pooling, whose atomic

[1] The call is `torch.use_deterministic_algorithms(True)`, the flag `torch.backends.cudnn.benchmark = False`.



operations make the rounding differ from run to run (NVIDIA 2026b).[2]

The strongest version of the case was made carefully. Nagarajan et al. (2018), in a preprint, built a deterministic implementation of deep Q-learning, with a seeded generator for each source of randomness and PyTorch's deterministic GPU mode on, and released one source at a time.

> FINDING 2.1 (Nagarajan et al. 2018). Five runs of deep Q-learning on the game Breakout, 20 million steps each, with every seed fixed and deterministic GPU operations, produced identical learning curves. With one source of randomness released and five runs per source, the standard deviation of the final score relative to its mean was 12.4% for nondeterministic GPU operations, 6.7% for the sampling of minibatches, 16.9% for exploration, 37.7% for a stochastic environment, and 43.6% for the initial weights.

The authors separated *replicability*, repeating an experiment exactly, from *reproducibility*, obtaining the same results with minor differences, and called deterministic implementations necessary but not sufficient for the first: in their supplementary material, the same deterministic implementation produced different learning curves on two machines with different GPUs, operating systems, and library versions, while on each machine it repeated itself exactly.

The field received the shorter version: fix the seeds, and the result is reproducible. In a survey of 901 researchers and practitioners, Pham et al. (2020) found that 83.8% were unaware of, or unsure about, variability introduced by the deep learning libraries themselves, and only 10.4% expected two runs with the same seeds to differ in accuracy by 2 points or more; of 225 recent papers that trained deep networks, 19.5% (margin 3 points) reported several runs of one configuration.

Much of the practice is right. On a fixed stack of code, libraries, and hardware, exact repetition is possible and valuable: a changed number means changed code, which makes debugging possible, and a model or a published number can be regenerated for an audit. The labs of this book rely on it. Two questions remain: does the repetition survive a change of machine, library, or batch layout, and is the repeated number representative of the method? The first is about arithmetic, the second about the dynamics of training.

2.2 The experiment: six groups of five runs

The design of Nagarajan et al. (2018) is the cleanest experiment behind the practice: it builds a pipeline in which nothing varies and then lets one source of randomness vary at a time.

EXPERIMENTAL DESIGN · NAGARAJAN, WARNELL AND STONE 2018	
QUESTION	How much of the variability of a deep Q-learning agent does each source of randomness produce?

[2] An atomic addition lets many threads add into one memory location without losing updates. The additions happen in the order in which the threads arrive, which changes from run to run.

UNITS	training runs of deep Q-learning on Atari Breakout, 20 million steps each
TREATMENT	the one source of randomness left uncontrolled
LEVELS	none; GPU operations; sticky actions in the environment; exploration; initial weights; minibatch sampling
HELD FIXED	all other seeds; one machine (GeForce GTX 1080, PyTorch 0.3.0, CUDA 8.0, cuDNN 7.0); hyperparameters
RESPONSE	mean score over 100 greedy evaluation episodes from fixed start states, at the end of training and at the best checkpoint
REPLICATES	five runs per group
ANALYSIS	standard deviation and relative standard deviation in each group; tests of equal variance between groups at level 0.1

The design establishes that determinism can be reached on one machine and that releasing a single source spreads the outcome. It cannot show that the deterministic numbers recur on another machine, and the authors' own comparison of two machines shows that they do not. It ranks the sources only coarsely: for normal scores, the 95% interval for σ from a sample standard deviation S of five runs extends from $0.60S$ to $2.87S$ (Exercise 2.1), so the GPU and minibatch groups, at 12.4% and 6.7%, cannot be told apart, and the one difference in variance that the authors report at level 0.1 is between the GPU group and the initialization group.

Most important, the design cannot tell a source from the training that amplifies it. Each row of Finding 2.1 varies a source and, with it, the size and the time of its first effect: the initial weights differ at step zero by a full random draw, while the runs of the GPU group start from last-bit differences and, as the authors observed, share identical experiences for tens of thousands of steps. If training amplifies every perturbation, then each row measures the amplification as much as the source. The post-mortem separates the two.

2.3 The post-mortem: the stack and the dynamics

A fixed seed fixes the random numbers. It does not fix the arithmetic, which depends on the stack, and it does not fix the outcome, which depends on what training does with small differences.

THE STACK. Floating-point addition is not associative, so a change in the order in which a reduction adds its terms can change the last bits of the result (Section 2.4.1). On parallel hardware the order changes often: atomic additions complete in the order in which threads arrive, autotuning can choose a different algorithm in each run (Pham et al. 2020), libraries choose the blocking of a matrix product by the shapes of the matrices and hence by the batch size, and data-parallel training adds the gradients of several devices in an order set by their number. Pham et al. (2020) trained a WideResNet-28-10 on CIFAR-100 sixteen times on one GPU, with the same libraries and every algorithmic source of randomness fixed: the test accuracies ranged from 77.3% to 80.2%, and the accuracy on one class,

camel, from 38.1% to 90.5%.

Zhuang et al. (2022) separated the randomness that seeds control from that of the tooling: for ResNet-50 on ImageNet, two runs disagreed on 14.68% of the test predictions with tooling noise alone and on 14.89% with seed noise alone. Tensor processing units, deterministic by design, removed the tooling noise,[3] but a different order of the examples inside a batch still changed the trained models, even when the whole training set formed one batch, because that order is the order in which the gradient is accumulated. Deterministic GPU kernels took up to 7.46 times the GPU time of the default ones.[4] Even a kernel without atomic operations can return different bits for the same input in batches of different sizes. He (2025), in a blog post, traced the nondeterminism of language-model inference servers to this missing *batch invariance*: at temperature zero, 1000 completions of one prompt gave 80 distinct texts, and batch-invariant kernels made all 1000 identical.

[3] A TPU multiplies matrices with systolic arrays under a single-threaded, deterministic model of computation (Zhuang et al. 2022).

[4] On a P100 with 7×7 convolutions; on a V100 and a T4 the largest factors were 2.41 and 1.96.

THE DYNAMICS. Summers and Dinneen (2021) designed the experiment that separates a source from its amplification. They trained a ResNet-14 on CIFAR-10 for 500 epochs (cosine schedule from a learning rate of 0.4, batch size 512, momentum 0.9), 100 runs for each source of randomness with the others held fixed: the initialization, the shuffling of the data, the data augmentation, and the nondeterministic cuDNN kernels, alone and in combinations. Then they added a source of their own: in each run, one randomly chosen initial weight was moved to the next representable number, a change of the order of 10^{-10}.

> FINDING 2.2 (Summers and Dinneen 2021). ResNet-14 on CIFAR-10, 100 runs per source: every source of randomness, alone or combined with others, gave a standard deviation of test accuracy between 0.21 and 0.26 percentage points, and two runs disagreed on 10.5% to 10.7% of the test images. Moving one initial weight to the next representable number gave 0.21 points and 10.6%.

They called the phenomenon the *instability* of training, by analogy with numerical stability: viewed end to end, training is ill-conditioned, so that, as they argued, every source of randomness that changes the weights at all inherits at least the variability of a one-bit change. When they released a source only after N deterministic epochs, the variability of accuracy was nearly as large for $N = 50$ as for $N = 0$ and fell only gradually as the onset moved later.

The paper also narrows its own claim. In a linear model a one-bit change altered no test prediction, and in a network with one convolutional hidden layer and in a small network on MNIST it reproduced about half of the disagreement that all sources produced together (Table 2.1); the authors suggested that the degree of instability depends on the model, the data, and the degree of overfitting. Jordan (2024) replicated the effect by multiplying a single weight by 1.001: in CIFAR-10 trainings of 128 epochs this alone produced an average disagreement of 5.14%, against 5.19% between runs with entirely different seeds, and in short trainings 7.0–7.5% against about 8.5%.

data	model	all sources	one bit	ratio
CIFAR-10	linear	5.5	0.0	0.00
CIFAR-10	one convolutional hidden layer	12.7	6.4	0.50
MNIST	small convolutional network	0.56	0.30	0.54
CIFAR-10	one fully connected hidden layer	24.9	23.4	0.94
CIFAR-10	VGG-11	6.6	6.5	0.98
CIFAR-10	ResNet-14	10.7	10.6	0.99
ImageNet	ResNet-18	20.7	20.6	1.00
Penn Treebank	QRNN language model	17.4	17.7	1.02

Table 2.1. Disagreement between two runs, in percent of test images (of tokens for Penn Treebank), when every source of randomness varies and when only one initial weight moves by one unit in the last place; from Tables 2, 3, 4, 6, and 7 of Summers and Dinneen (2021), 100 runs per row except 20 for ImageNet. The ratio of the two columns is ours.

A related notion of stability concerns where training ends rather than what the network predicts. Frankle et al. (2020) trained two copies of a network from its state at step k with different data orders and asked whether the error rises along the straight line between the end points: only LeNet on MNIST was stable in this sense from initialization, ResNet-20 on CIFAR-10 after 3% of its training, and ResNet-50 on ImageNet after 20%. Fort et al. (2020) found that a chaotic early phase of two to three epochs determines the linearly connected basin in which training ends. Neither finding contradicts the onset experiment of Summers and Dinneen: two networks in one basin can still disagree on many test images. Stability in this sense returns in the lottery-ticket experiments of **??**.

VERDICT. Fixing every seed does repeat a run bit for bit, but only with the same code, library versions, kernels, and hardware, as the documentation of PyTorch, cuBLAS, and cuDNN says. Where it holds, the repetition is fragile. In the deep networks of Summers and Dinneen, moving one initial weight by one unit in the last place reproduced nearly all of the disagreement between independently seeded runs, while in a linear model it changed nothing and in two small models it reproduced about half. A fixed seed selects one draw from the run distribution and makes it repeatable; it does not make it representative.

NARROWED

2.4 The method: repeatable runs and reproducible results

Three different things go by the name of reproducibility, and a seed controls only the first.

Definition 2.3 (Three levels of reproducibility). Fix a training procedure (code, data, and hyperparameters) and an *environment* $\mathcal{E}$: the hardware, the library versions, the kernels selected, the numbers of devices and threads, and the layout of the batches. A run with seeds s in $\mathcal{E}$ produces an outcome $Y(s, \mathcal{E})$.

(i) The procedure is *repeatable* in $\mathcal{E}$ if two runs with the same seeds in $\mathcal{E}$ produce the same bits, for every s.

(ii) It is *distributionally reproducible* between $\mathcal{E}$ and $\mathcal{E}'$ if, with seeds drawn at random, $Y(s, \mathcal{E})$ and $Y(s, \mathcal{E}')$ have the same distribution.

(iii) A conclusion of an experiment, a decision computed from its runs such as "method 1 beats method 2", is *reproducible* if an independent replicate of the experiment, with fresh seeds and possibly in another environment, reaches the same decision with high probability.

Repeatability is a property of a pipeline on a machine, distributional reproducibility a property of the run distribution, and the reproducibility of a conclusion a property of the design: the size of the effect, the number of runs, and the test (Chapter 1). None implies the next: a repeatable pipeline can repeat an unrepresentative run, and a pipeline with nondeterministic kernels is not repeatable but can be distributionally reproducible. The three levels are close to what Goodman et al. (2016) call methods, results, and inferential reproducibility.[5]

[5] The words are used inconsistently across fields, sometimes with the meanings of *reproducible* and *replicable* swapped (Goodman et al. 2016). Nagarajan et al. (2018) use replicability for our repeatability.

Remark 2.4 (What a seed buys). A repeatable pipeline makes three things possible. Debugging: a changed number means a changed input, which bisection over versions of the code finds. Audit: a reported model can be regenerated. Pairing: two arms of a comparison run with the same seeds share their initialization and data order, and if their outcomes are positively correlated, the variance $\mathrm{Var}(Y_1) + \mathrm{Var}(Y_2) - 2\,\mathrm{Cov}(Y_1, Y_2)$ of their difference is smaller than with independent seeds (Chapter 1); the correlation lasts only as long as the two runs stay close, which in unstable training is not long (Section 2.5). A seed does not buy evidence: the number it repeats is one draw from the run distribution.

2.4.1 The arithmetic beneath a run

A binary floating-point format with *precision* p represents a nonzero number as $\pm m\,2^e$, with an integer significand $2^{p-1} \le m < 2^p$ and an integer exponent e in a fixed range. Between 1 and 2 the representable numbers are 2^{1-p} apart; this spacing is one *unit in the last place* (ulp) of 1, and half of it is the *unit roundoff* $u = 2^{-p}$, about 6.0×10^{-8} in binary32 (single precision), where $p = 24$, and 3.9×10^{-3} in the 16-bit format bfloat16. The IEEE 754 standard (IEEE 2019) requires that, in its default mode, a sum be rounded to a nearest representable number, which gives the model from which rounding-error analysis starts (Higham 2002, Chapter 2).

Assumption 2.5 (Standard model of floating-point addition). For floating-point numbers a and b whose sum does not overflow, the computed sum is $\mathrm{fl}(a+b) = (a+b)(1+\delta)$ for some δ with $|\delta| \le u$.

The exact sum of two floating-point numbers is in general not representable, the rounded sum of several depends on which are added first, and so floating-point addition is not associative.

EXAMPLE 2.6 (Three numbers). In binary32 let $x = (1, 2^{-24}, 2^{-24})$. The exact sum $1 + 2^{-24}$ lies halfway between the neighbors 1 and $1 + 2^{-23}$; the tie goes to the neighbor whose last bit is even, which is 1, and adding the second 2^{-24} gives 1 again. Added the other way, $2^{-24} + 2^{-24} = 2^{-23}$ exactly, and $1 + 2^{-23}$ is representable. The two orders give 1 and $1 + 2^{-23}$: they differ in the last bit, by $2^{-23} = 2u$.

Over a chain of operations the standard model compounds into products of factors $1 + \delta$. For integers $m \ge 0$ with $mu < 1$ define $\gamma_m = mu/(1-mu)$.

Lemma 2.7 (Products of rounding factors). *Let $m \ge 1$ with $mu < 1$. If $|\delta_k| \le u$ for $k = 1, \dots, m$, then $\prod_{k=1}^{m}(1+\delta_k) = 1 + \theta$ with $|\theta| \le \gamma_m$.*

PROOF. Each factor lies in $[1-u, 1+u]$, an interval of positive numbers because $u \le mu < 1$, so the product lies between $(1-u)^m$ and $(1+u)^m$. For the upper end, $1 + u \le e^u$, and $e^{mu} \le 1/(1-mu)$ because $e^{-x} \ge 1 - x$ for $x = mu \in [0, 1)$; hence $(1+u)^m - 1 \le 1/(1-mu) - 1 = \gamma_m$. For the lower end, Bernoulli's inequality, valid since $-u \ge -1$, gives $(1-u)^m \ge 1 - mu \ge 1 - \gamma_m$. ∎

A *summation tree* for $x_1, \dots, x_n$ is a binary tree whose leaves are the x_i and whose internal nodes add the values of their two children; evaluated in floating point, it computes a sum $\hat{s}_T$. The depth d_i of the leaf x_i is the number of internal nodes on its path to the root, and the depth of the tree is $h = \max_i d_i$. *Recursive summation* in the order $x_1, \dots, x_n$ computes $\hat{s}_1 = x_1$ and $\hat{s}_k = \mathrm{fl}(\hat{s}_{k-1} + x_k)$ for $k = 2, \dots, n$; its tree has depth $n - 1$. *Pairwise summation* adds neighbors in a balanced tree of depth $\lceil \log_2 n \rceil$. The reductions of parallel hardware use trees of this kind, whose shape depends on how the work is divided among threads.

Proposition 2.8 (Error of a floating-point sum). *Under Assumption 2.5, let $n \ge 2$, let $s = \sum_i x_i$, and let T be a summation tree of depth h with $hu < 1$.*

(i) The computed sum is $\hat{s}_T = \sum_{i=1}^{n} x_i(1+\theta_i)$ with $|\theta_i| \le \gamma_{d_i}$, so that $|\hat{s}_T - s| \le \gamma_h \sum_i |x_i|$.

(ii) Two summation trees of depths h and h' give sums that differ by at most $(\gamma_h + \gamma_{h'}) \sum_i |x_i|$. For recursive summation in two orders the bound is $2\gamma_{n-1} \sum_i |x_i| = 2(n-1)u \sum_i |x_i| + O(u^2)$, and for pairwise summation in two orders it is $2\gamma_{\lceil \log_2 n \rceil} \sum_i |x_i|$.

(iii) Recursive summation satisfies $|\hat{s}_n - s| \le \frac{u}{1-u} \sum_{k=2}^{n} |\hat{s}_k|$.

PROOF. The plan is an induction over the tree for (i), the triangle inequality for (ii), and a telescoping sum for (iii).

Part (i). For a node v let $\hat{v}$ be its computed value, so that $\hat{v} = x_i$ when v is the leaf x_i. For an internal node v let δ_v be the rounding error of its addition, and for a leaf x_i below v let $P(i, v)$ be the set of internal nodes on the path from x_i up to v, with v included and $P(i, x_i)$ empty. We show, by induction over the nodes with children before parents, that

$$\hat{v} = \sum_{x_i \text{ below } v} x_i \prod_{w \in P(i,v)} (1 + \delta_w). \tag{2.1}$$

For a leaf the product is empty and (2.1) reads $x_i = x_i$. For an internal node v with children a and b, the standard model gives $\hat{v} = (\hat{a} + \hat{b})(1 + \delta_v)$; inserting (2.1) for a and b, and using $P(i, v) = P(i, a) \cup \{v\}$ for the leaves below a and likewise for b, gives (2.1) for v. At the root the product for x_i has d_i factors, with $d_i \geq 1$ because $n \geq 2$ and $d_i u \leq hu < 1$; Lemma 2.7 turns it into $1 + \theta_i$ with $|\theta_i| \leq \gamma_{d_i} \leq \gamma_h$, since γ_m increases with m. Hence $|\hat{s}_T - s| = |\sum_i x_i \theta_i| \leq \gamma_h \sum_i |x_i|$.

Part (ii). The exact sum s is the same for both trees, so the triangle inequality and part (i) give the first bound. The recursive tree has depth $n - 1$, the balanced tree depth $\lceil \log_2 n \rceil$, and $\gamma_{n-1} = (n - 1)u + O(u^2)$.

Part (iii). For $k \geq 2$ the standard model gives $\hat{s}_{k-1} + x_k = \hat{s}_k/(1 + \delta_k)$, that is, $\hat{s}_k - \hat{s}_{k-1} - x_k = \hat{s}_k \, \delta_k/(1 + \delta_k)$. Summing over $k = 2, \ldots, n$ and using $\hat{s}_1 = x_1$, the left side telescopes to $\hat{s}_n - s$, and $|\delta_k/(1 + \delta_k)| \leq u/(1 - u)$. ■

In Example 2.6 the two orders differ by $2u$, while part (ii) allows $2\gamma_2(1 + 2^{-23}) \approx 4u$. For long sums the worst case is far from typical, and part (iii) says why: the error is controlled by the partial sums, and a sum of terms of mixed sign, such as the sum of per-example gradients over a minibatch, keeps them small; with rounding errors modeled as independent with mean zero, bounds with $\sqrt{n}\, u$ in place of nu hold with high probability (Higham and Mary 2019). Training computes such sums over the examples of a minibatch, the inner dimension of a matrix product, and the devices of data-parallel training, and every change of their order moves the result by an amount that Proposition 2.8 makes tiny. The question is what training does with it.

2.5 The method: how training amplifies a bit

Let θ_t be the state of a run after t steps, its parameters together with the optimizer's buffers, and write one step in exact arithmetic as $\theta \mapsto \Phi_t(\theta)$. The map Φ_t depends on the minibatch of step t, its learning rate, and the random numbers of dropout or augmentation; fixed seeds fix the sequence $\Phi_0, \Phi_1, \ldots$. A *twin* of a run uses the same seeds and differs by a perturbation: another initial state, as when one weight moves by an ulp, or other rounding errors, as when a minibatch is summed in another order. In floating point the run computes $\theta_{t+1} = \Phi_t(\theta_t) + \rho_t$ and its twin $\theta'_{t+1} = \Phi_t(\theta'_t) + \rho'_t$, with rounding errors ρ_t and ρ'_t; with $e_t = \theta'_t - \theta_t$ and $r_t = \rho'_t - \rho_t$,

$$e_{t+1} = \Phi_t(\theta_t + e_t) - \Phi_t(\theta_t) + r_t. \tag{2.2}$$

Lemma 2.9 (Propagation of a difference). *Suppose that* $\|\Phi_t(\theta'_t) - \Phi_t(\theta_t)\| \le L_t\|e_t\|$ *and* $\|r_t\| \le \varepsilon_t$ *for every* $t \ge 0$. *Then, for every* t,

$$\|e_t\| \le \Big(\prod_{k=0}^{t-1} L_k\Big)\|e_0\| + \sum_{j=0}^{t-1}\Big(\prod_{k=j+1}^{t-1} L_k\Big)\varepsilon_j.$$

In particular, if $L_t \le L < 1$ *and* $\varepsilon_t \le \varepsilon$ *for every* t, *then* $\|e_t\| \le L^t\|e_0\| + \varepsilon/(1-L)$.

Proof. By (2.2) and the hypotheses, $\|e_{t+1}\| \le L_t\|e_t\| + \varepsilon_t$. The bound holds for $t = 0$, where the product is empty and the sum has no terms, and inserting it for t into this inequality gives it for $t+1$. In the special case the sum is at most $\varepsilon \sum_{j\ge 0} L^j = \varepsilon/(1-L)$. ∎

If every step contracts, then a perturbation is forgotten and the rounding differences stay bounded. If steps expand, then the product of the factors grows exponentially. Gradient descent on a strongly convex loss is the first case.

Proposition 2.10 (Contraction forgets). *Let* $f : \mathbb{R}^d \to \mathbb{R}$ *be twice continuously differentiable with* $\mu I \preceq \nabla^2 f(\theta) \preceq MI$ *for every* θ, *where* $0 < \mu \le M$, *and let* $0 < \eta \le 2/(\mu + M)$. *Then the step* $\Phi(\theta) = \theta - \eta\nabla f(\theta)$ *satisfies* $\|\Phi(\theta) - \Phi(\theta')\| \le (1 - \eta\mu)\|\theta - \theta'\|$, *and two runs of gradient descent on* f *whose rounding errors differ by at most* ε *per step stay within* $(1-\eta\mu)^t\|e_0\| + \varepsilon/(\eta\mu)$ *of each other at every step* t.

Proof. The plan is to write the difference of two gradients with an averaged Hessian. For $\theta, \theta' \in \mathbb{R}^d$, the fundamental theorem of calculus applied to $s \mapsto \nabla f(\theta' + s(\theta - \theta'))$, whose derivative is continuous because f is twice continuously differentiable, gives $\nabla f(\theta) - \nabla f(\theta') = \bar{H}(\theta - \theta')$ with $\bar{H} = \int_0^1 \nabla^2 f(\theta' + s(\theta - \theta'))\,\mathrm{d}s$. As an average of such matrices, $\bar{H}$ is symmetric with $\mu I \preceq \bar{H} \preceq MI$. Hence $\Phi(\theta) - \Phi(\theta') = (I - \eta\bar{H})(\theta - \theta')$, and every eigenvalue of $I - \eta\bar{H}$ lies in $[1 - \eta M, 1 - \eta\mu]$. The condition $\eta \le 2/(\mu + M)$ gives $\eta M - 1 \le 1 - \eta\mu$, and, since $2/(\mu + M) \le 1/\mu$, also $1 - \eta\mu \ge 0$; so every eigenvalue has absolute value at most $1 - \eta\mu$, which bounds the norm of the symmetric matrix $I - \eta\bar{H}$. The second statement is Lemma 2.9 with $L_t = 1 - \eta\mu < 1$, for which $1 - L_t = \eta\mu$. ∎

Softmax regression with weight decay λ is λ-strongly convex on every minibatch, so stochastic gradient descent on it with a small enough step, and the same minibatches in both runs, forgets a one-ulp change; convexity is the reason Summers and Dinneen (2021) gave for the stability of their linear models. The bound $\varepsilon/(\eta\mu)$ can far exceed ε, but it does not grow with t.

Proposition 2.11 (Expansion amplifies). *Let* $f(\theta) = \frac{1}{2}\theta^{\top}H\theta$ *with* H *symmetric, let* v *be a unit eigenvector of* H *with eigenvalue* λ, *and let* $\eta > 0$. *Two runs of gradient descent on* f *in exact arithmetic, started at* θ_0 *and*

$\theta_0 + e_0$, differ at step t by $e_t = (I - \eta H)^t e_0$, and $\langle v, e_t\rangle = (1-\eta\lambda)^t\langle v, e_0\rangle$. If $|1-\eta\lambda| > 1$, which happens exactly when $\lambda < 0$ or $\eta\lambda > 2$, and $\langle v, e_0\rangle \neq 0$, then $\|e_t\| \geq |1-\eta\lambda|^t|\langle v, e_0\rangle|$; in particular, for every $\Delta > |\langle v, e_0\rangle|$, $\|e_t\| \geq \Delta$ from the step $\lceil \ln(\Delta/|\langle v, e_0\rangle|)/\ln|1-\eta\lambda|\rceil$ on.

PROOF. The step $\Phi(\theta) = (I - \eta H)\theta$ is linear, so $e_{t+1} = (I - \eta H)e_t$ and, by induction, $e_t = (I - \eta H)^t e_0$. Because H is symmetric and $Hv = \lambda v$, we have $\langle v, (I - \eta H)w\rangle = (1 - \eta\lambda)\langle v, w\rangle$ for every w, which gives the formula for $\langle v, e_t\rangle$, and the Cauchy–Schwarz inequality gives $\|e_t\| \geq |\langle v, e_t\rangle|$. The step count follows by taking logarithms in $|1 - \eta\lambda|^t|\langle v, e_0\rangle| \geq \Delta$, which is legitimate because $|1 - \eta\lambda| > 1$. ∎

A one-ulp change of a weight in binary32 has relative size about 2^{-24}, and growing to the size of the weights takes about $24 \ln 2/\ln|1-\eta\lambda| \approx 17/\ln|1-\eta\lambda|$ steps: about 175 when $|1 - \eta\lambda| = 1.1$, and 24 when it is 2. Both causes of expansion occur in deep learning: saddle points have directions of negative curvature, and Cohen et al. (2021) found that full-batch gradient descent on neural networks typically runs at the *edge of stability*, with the largest eigenvalue of the Hessian hovering just above $2/\eta$. A quadratic with an expanding direction diverges outright, however, and a run that stays bounded while separating from its twin needs nonlinearity. Gradient descent on a double well with a large step is exactly chaotic (Exercise 2.3), and Kong and Tao (2020) showed that deterministic gradient descent with a large learning rate on a multiscale objective can behave like a stochastic method. For a network, the linear picture becomes a heuristic.[6]

[6] The butterfly of chaos theory entered in 1972, in the title that the convener of a session chose for a talk by Lorenz, whose 1963 model of convection had shown the effect (Lorenz 1963).

HEURISTIC DERIVATION: THE DIVERGENCE TIME. While the twins are close, (2.2) is linear to first order: $e_{t+1} \approx J_t e_t + r_t$, where $J_t = D\Phi_t(\theta_t)$ is the Jacobian of the step along the run; for gradient descent $J_t = I - \eta\nabla^2\widehat{\mathcal{L}}_t(\theta_t)$, with $\widehat{\mathcal{L}}_t$ the loss on the minibatch of step t. Define the *cumulative exponent* $\Lambda(t) = \ln\big(\|J_{t-1}\cdots J_0 e_0\|/\|e_0\|\big)$. Then $\|e_t\| \approx \|e_0\|e^{\Lambda(t)}$, plus the rounding differences r_j carried forward by the later Jacobians, until the linearization fails at a distance Δ of the order of the distance between runs that share the initialization but not the subsequent noise. If $\Lambda(t) \approx \ell t$ with a positive rate ℓ, a finite-time Lyapunov exponent, then the twins separate exponentially, and the *divergence time* is

$$t^\star \approx \frac{\ln(\Delta/\|e_0\|)}{\ell}.$$

Three predictions follow: the logarithm of the distance grows linearly before it saturates near Δ; each factor of ten in the initial perturbation advances the divergence by $\ln(10)/\ell$ steps; and no perturbation acts as if it were smaller than the rounding differences that every step adds, so that below that floor a one-ulp change and a reordered sum are the same experiment. If the rate varies over training, then $t^\star$ solves $\Lambda(t^\star) = \ln(\Delta/\|e_0\|)$, and the twins separate where Λ rises steeply. If Λ stays bounded, as under Proposition 2.10, then they never separate. ◇

A network with ReLU activations is only piecewise smooth: when a perturbation pushes a pre-activation across zero, the Jacobian changes abruptly, and the difference between the twins can jump. Shamir et al. (2020), in a preprint, found that smooth activations improve the trade-off between accuracy and the reproducibility of predictions.

2.5.1 What a twin measures

The heuristic says when twins separate, not how different they end. The run distribution answers that question.

Proposition 2.12 (What a twin measures). *Let Y and Y' be the outcomes of a run and of its twin, with $\mathbb{E}Y^2 < \infty$, and let $\mathcal{G}$ be a σ-field, the information that the two runs share. Suppose that, conditionally on $\mathcal{G}$, the outcomes Y and Y' are independent and have the same distribution, and that $\operatorname{Var} Y > 0$. Let Y'' be the outcome of an independent run with the distribution of Y. Then*

$$\operatorname{Cov}(Y, Y') = \operatorname{Var}\big(\mathbb{E}[Y \mid \mathcal{G}]\big), \qquad \mathbb{E}(Y - Y')^2 = 2\,\mathbb{E}\operatorname{Var}(Y \mid \mathcal{G}),$$

so that $\rho = \operatorname{Corr}(Y, Y') = \operatorname{Var}(\mathbb{E}[Y \mid \mathcal{G}])/\operatorname{Var} Y$ lies in $[0, 1]$, and

$$\frac{\mathbb{E}(Y - Y')^2}{\mathbb{E}(Y - Y'')^2} = 1 - \rho.$$

Proof. The plan is to condition on $\mathcal{G}$. Write $m = \mathbb{E}[Y \mid \mathcal{G}]$. Because Y' has the conditional distribution of Y, $\mathbb{E}[Y' \mid \mathcal{G}] = m$ and $\mathbb{E}[Y'^2 \mid \mathcal{G}] = \mathbb{E}[Y^2 \mid \mathcal{G}]$; taking expectations, Y' has the mean and the variance of Y. Conditional independence gives $\mathbb{E}[YY' \mid \mathcal{G}] = m^2$, so $\operatorname{Cov}(Y, Y') = \mathbb{E}m^2 - (\mathbb{E}m)^2 = \operatorname{Var} m$. Since the means are equal,

$$\mathbb{E}(Y - Y')^2 = \operatorname{Var} Y + \operatorname{Var} Y' - 2\operatorname{Cov}(Y, Y') = 2(\operatorname{Var} Y - \operatorname{Var} m) = 2\,\mathbb{E}\operatorname{Var}(Y \mid \mathcal{G}),$$

the last step by the law of total variance. For the independent run, $\operatorname{Cov}(Y, Y'') = 0$, and the same computation gives $\mathbb{E}(Y - Y'')^2 = 2\operatorname{Var} Y$. Dividing, and using $\operatorname{Var} Y' = \operatorname{Var} Y$ in the definition of ρ, gives the ratio. Finally $0 \le \operatorname{Var} m \le \operatorname{Var} Y$ by the law of total variance, so $\rho \in [0, 1]$. ∎

For a twin, $\mathcal{G}$ holds the shared initialization, the data order, and the trajectory up to the divergence. If the twin separates at once and keeps nothing that matters, then $\rho = 0$ and the twin is a new draw from the run distribution. If it never separates, then $\rho = 1$ and the twin is a copy. In both cases Y' has the distribution of Y, so a two-sample test of twins against runs with fresh seeds cannot tell a copy from a new draw; the paired quantities can. Applied to the prediction on one test example, the same argument shows that twins disagree on it at most as often as independent runs (Exercise 2.4); the fraction of test examples on which two runs disagree is called *churn* (Milani Fard et al. 2016).

2.6 In the laboratory: twins of a digits network

A GPU is not needed to see the phenomena of this chapter: on a CPU we can create the perturbations that a GPU creates by accident, a moved bit and a reordered sum, on purpose and one at a time. The labs use the digits network of the book's kit, a perceptron with two hidden layers of 64 ReLU units and 8970 parameters.

LAB 2.1 · WHAT A FIXED SEED REPEATS

SCRIPT
02-instability-sums.py

SETUP. Binary32 arithmetic on a laptop CPU: the three numbers of Example 2.6; for each n from 10 to 10^5, twenty vectors of n standard normal numbers, each summed in 50 random orders, recursively and pairwise; the logits of a trained network for the 539 test images, computed in batches of 539, 64, and 1; and three runs with the same seeds, one of them on one thread instead of two. A few seconds.

PREDICT FIRST. How many of 50 random orders of 10^5 numbers give the same sum? Do the logits of an image depend on how many other images share its batch?

LOOK FOR. The number of distinct sums and their spread against Proposition 2.8; the images whose logits change in some bit; whether the hashes of the final weights agree.

The two orders of Example 2.6 gave 1 and $1 + 2^{-23}$. In Table 2.2, recursive summation of 10^5 numbers in 50 random orders gave 46 distinct results on average, spread over 1.6 units of $u \sum_i |x_i|$, where part (ii) of Proposition 2.8 allows 2×10^5; the mean error stayed near $0.27\, u \sum_i |x_i|$ for every n, and the largest ratio of an error to the bound of part (iii), over all 5000 recursive sums, was 0.57. The logits of a trained network did not change when the same batch was recomputed, but in batches of 64 those of 5.0% of the test images changed in some bit, and one image at a time those of every image changed, by at most 7.6×10^{-6}; no predicted class changed. The three runs with the same seeds gave weights with one hash: on this machine (PyTorch 2.13 with Apple's Accelerate library) the thread count did not change the order of the sums, which need not hold elsewhere.

LAB 2.2 · TWIN RUNS

SCRIPT
02-instability-twins.py

SETUP. The digits network trained by SGD with momentum 0.9, learning rate 0.1, and batch size 64 for 50 epochs (950 steps), in two regimes that differ only in the labels: clean, and with 20% of the training labels replaced by random labels. For each of ten seeds per regime: the run, a repeat, a one-ulp twin (one random initial weight moved to its neighbor by `torch.nextafter`), a reorder twin (the examples inside every minibatch permuted), an order twin (a new data-order seed), and an independent run, with the distance between parameter vectors every 5 steps. With noisy labels, also twins

	recursive			pairwise		
n	distinct	spread	bound	distinct	spread	bound
10	5.5	1.39	18	5.0	1.24	8
100	15.5	1.69	198	7.9	0.65	14
1000	23.6	1.58	1998	7.5	0.25	20
10 000	37.6	1.52	20 010	9.1	0.09	28
100 000	46.2	1.64	201 197	10.7	0.03	34

Table 2.2. Sums of n standard normal binary32 numbers in 50 random orders (Lab 2.1; 20 vectors per n, seed 20261017). Distinct: number of different results among the 50 orders, averaged over the vectors. Spread: largest minus smallest result, in units of $u \sum_i |x_i|$, averaged over the vectors. Bound: $2\gamma_h/u$, the worst case of Proposition 2.8(ii) in the same units, with $h = n-1$ for recursive and $h = \lceil \log_2 n \rceil$ for pairwise summation.

moved in a random direction by 10^{-7} to 10^{-2} of the initial weight norm. About 30 seconds.

PREDICT FIRST. With clean labels, will a one-ulp twin end with a different test error? With noisy labels? At which step will the distance between twins reach 1?

LOOK FOR. The floor at which every twin sits after a few dozen steps; whether and when the distance leaves it; where it saturates; how the step of divergence depends on the size of the initial change.

Figure 2.1 shows the result. A repeat with identical seeds stayed identical at all 950 steps in both regimes, and the script's own training loop, which adds the perturbations, reproduced the kit's runs bit for bit. Within 50 steps every twin, one-ulp or reorder, sat at a distance of about 10^{-6}, a few units of u relative to the norm of the weights: the rounding floor of the heuristic, at which a moved bit and a reordered sum are indistinguishable. In one pair of the ten the moved bit was rounded away within fifteen steps, and that twin stayed bitwise identical to its run in both regimes.[7]

With clean labels nothing more happened. The distance drifted to about 10^{-5}, at most 2.5×10^{-4}, while order twins ended about 3.3 apart; no one-ulp or reorder twin disagreed with its run on a single test image, while order twins disagreed on 1.15% and independent runs on 1.91% of them. The difference stayed near the rounding floor, as it would under Proposition 2.10, although the loss is not convex.

With noisy labels the twins stayed near the floor for several hundred steps and then separated within a few dozen: the logarithm of the distance grew at a median rate of 0.012 per step until the distance reached 10^{-4}, and of 0.093 per step from there to 1. The one-ulp twins crossed a distance of 1 at a median step of 535 (range 345–585) and the reorder twins at 512 (230–590), and both saturated near 11, while order twins ended about 25 apart. At the end, one-ulp twins disagreed with their runs on 20.9% of the test images and reorder twins on 23.4%, against 26.8% for order twins and

[7] A difference can vanish only by rounding: once the perturbed weight rounds to the same number as the original, every later computation is identical.

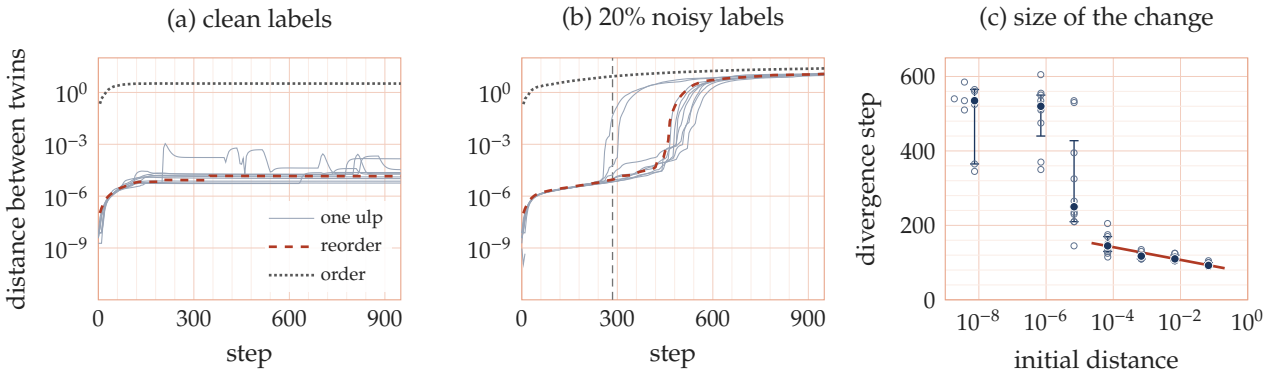


Figure 2.1. Twin runs of the digits network (8970 parameters; SGD with momentum 0.9, learning rate 0.1, batch size 64, 950 steps); seeds $i = 0, \ldots, 9$, run i with initialization and order seeds i. (a) Clean labels and (b) 20% random training labels: Euclidean distance between the parameters of a run and of its twin. Thin lines: one-ulp twins, one per seed; dashed: median over seeds of reorder twins; dotted: median of order twins. Gaps mark a distance of exactly zero; a repeat with identical seeds is at zero throughout and is not drawn. The vertical line in (b) is the median step at which the runs have fitted half of their 237 mislabeled training examples. (c) Noisy labels: first step at which the distance exceeds 1, against the initial distance, for one-ulp twins (left) and for twins moved in a random direction by $10^{-7}, \ldots, 10^{-2}$ of the initial weight norm. Circles: seeds; filled: medians with 95% percentile-bootstrap intervals; line: least-squares fit to the four largest sizes.

28.1% for independent runs. The burst came after a run had fitted half of its 237 mislabeled training examples (median step 285); at the divergence step a run had fitted a median 76% of them (range 59–81%). The instability lives in the phase in which the network memorizes wrong labels. In panel (c), initial changes of 10^{-5} to 10^{-2} of the weight norm diverged in the early phase, at median steps from 145 down to 92, a slope of 7.4 steps per unit of ln or an exponent of 0.135 per step, while changes of one ulp and 10^{-7} waited for the late burst, near steps 535 and 520, and 10^{-6} fell in between (median 250). The exponent is not constant, and the divergence time follows the cumulative exponent Λ rather than a single rate, as the last sentence of the heuristic allows.

LAB 2.3 · TWINS AGAINST NEW SEEDS

SETUP. The network, optimizer, and regimes of Lab 2.2. In each regime, 100 runs with seeds $i = 0, \ldots, 99$, their one-ulp twins, and 100 runs with new seeds $1000 + i$; with noisy labels also 100 early twins, moved by 1% of the initial weight norm, which diverge near step 90. Then 20 pairs for each of five learning rates from 0.0125 to 0.15. About 2.5 minutes.

PREDICT FIRST. Will a two-sample test tell twins from runs with new seeds? Will the correlation between a run and its twin?

LOOK FOR. The p-values of the permutation and Kolmogorov–Smirnov tests; the correlation between each run and its twin; the ratio of disagreements across learning rates.

SCRIPT
`02-instability-draws.py`

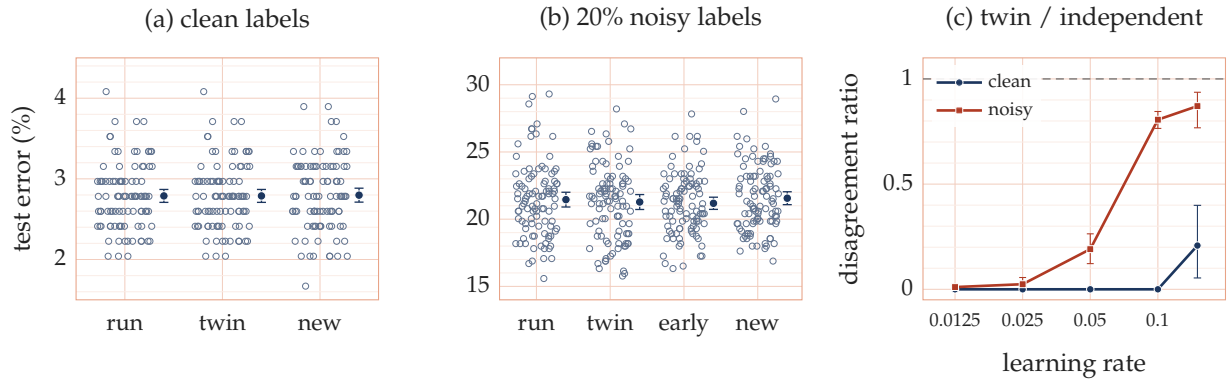


Figure 2.2. Test errors of the network of Figure 2.1. (a) Clean labels and (b) 20% random training labels: 100 runs (seeds $i = 0, \dots, 99$), their one-ulp twins, early twins moved by 1% of the initial weight norm in a random direction (noisy labels only), and 100 runs with new seeds $1000 + i$. Circles: runs, spread horizontally; filled, to the right of each group: means with 95% t-intervals. (c) Disagreement on the 539 test images between a run and its one-ulp twin, divided by the mean disagreement between a run and an independently seeded one, against the learning rate; 20 pairs per point, 95% percentile-bootstrap intervals over pairs. The dashed line is the ratio for two independent runs.

In both regimes the twins passed the tests of distributional reproducibility against new seeds (Figure 2.2).[8] With clean labels the mean test errors were $2.79 \pm 0.08\%$ for the runs, $2.79 \pm 0.08\%$ for their twins, and $2.80 \pm 0.09\%$ for new seeds (means with 95% t-intervals, 100 runs each), and twins against new seeds gave $p = 0.90$ in a two-sided permutation test of the means and $p = 0.82$ in a Kolmogorov–Smirnov test. With noisy labels the means were $21.45 \pm 0.55\%$, $21.28 \pm 0.56\%$, $21.18 \pm 0.45\%$ for the early twins, and $21.56 \pm 0.48\%$, and the tests gave $p = 0.44$ and $p = 0.47$ for the one-ulp twins. As Proposition 2.12 predicts, these tests cannot see what separates the regimes; the paired statistics can. With clean labels the correlation between the test errors of a run and its twin was 1.00, and no twin disagreed with its run on a single image: the twins were copies, and three of the 100 ended bitwise identical. With noisy labels the correlation was 0.25 (95% Fisher interval 0.06 to 0.43), against 0.12 (−0.07 to 0.31) between runs and new seeds, and the twins disagreed with their runs on 20.7% of the test images, against 27.4% for new seeds: a ratio of 0.75 (bootstrap interval 0.72 to 0.78). The early twins, which separate near step 90 instead of 535, came closer to new draws, with correlation 0.21 (0.01 to 0.39) and ratio 0.85 (0.83 to 0.87): the earlier twins separate, the less they share. The direct ratio of mean squared differences, 0.98 (0.70 to 1.37), is too noisy to use.

In panel (c) the disagreement ratio stayed at 0 with clean labels for every learning rate up to 0.1 and reached 0.21 (0.05 to 0.40) at 0.15. With noisy labels it rose from 0.01 at 0.0125, where the runs did not memorize the wrong labels in 50 epochs, through 0.19 at 0.05, to 0.81 at 0.1 and 0.87 (0.77 to 0.94) at 0.15. The miniature reproduces both halves of the verdict: a fixed seed repeats a run exactly, and a one-bit change is erased in the stable regime and becomes most of a new draw in the unstable one. It does not reach the ratios near 1 of Table 2.1, because its twins diverge late, after

[8] One test image is 0.19 percentage points of test error, which is why the runs in Figure 2.2 fall on a grid.

sharing half of their training. We chose the learning rate, the number of epochs, and the label noise after a short pilot scan, which the regime map reports in full; the network, data, and optimizer are the defaults of the kit.

2.7 Protocol

PROTOCOL 2.1 · REPEATABLE RUNS, REPRODUCIBLE RESULTS

- ☐ Give each source of randomness its own seed (initialization, data order, augmentation and dropout, data split) and record all of them with the result.
- ☐ For debugging and audit runs, turn on deterministic algorithms and turn off autotuning; record the code version, library versions, hardware, numbers of devices and threads, batch size, and determinism flags, and store a hash of the final weights.
- ☐ Check repeatability before relying on it: run one configuration twice and compare the hashes. If they differ, then find the nondeterministic operation before going on.
- ☐ Check stability: train a twin with one weight moved by one ulp, or with the examples of each minibatch reordered, and compare its predictions with the run's. If they differ, then treat every seed as a random draw and expect bitwise repetition to fail on other hardware.
- ☐ Draw seeds before training and never choose them by their results; report every run, with n runs per arm sized as in Chapter 1.
- ☐ When the hardware or a library changes, test distributional reproducibility: n runs with fresh seeds in each environment, a two-sample test of the outcomes, and the comparison of interest recomputed in both.
- ☐ State which level of reproducibility a reported number has: repeatable on a stated stack, or a summary of a run distribution with its interval.

The protocol does not make a result portable: no seeding makes two GPU architectures add in the same order, and only order-independent arithmetic, such as the reproducible summation of Demmel and Nguyen (2015), would. Determinism costs time, up to several times the GPU time (Zhuang et al. 2022), so it belongs in debugging and audit runs rather than in every run of a sweep. A twin test with a few pairs detects instability that matters but not a weak correlation between twins, which needs about a hundred pairs (Lab 2.3), and a two-sample test with few runs per environment has little power (Chapter 1). When a run is too expensive to repeat, as in large-scale pretraining, run the twin test on a small version of the model, report whether training was stable there, and treat differences between single runs smaller than the spread of seeds as unresolved.

Exercises

2.1 CONSOLIDATION. (Five runs and a standard deviation.) For normal outcomes, $(n-1)S^2/\sigma^2$ has the χ^2 distribution with $n-1$ degrees of freedom. (a) Derive the 95% interval for σ from S, and show that it runs from $0.60S$ to $2.87S$ for $n=5$ and from $0.76S$ to $1.46S$ for $n=20$. (b) Which rows of Finding 2.1 could be told apart with five runs per group?

2.2 CONSOLIDATION. (Momentum.) For heavy-ball momentum on $f(\theta)=\frac{1}{2}\lambda\theta^2$, the twin difference obeys $e_{t+1}=(1+\beta-\eta\lambda)e_t-\beta e_{t-1}$. (a) For $0\le\beta<1$, show that $e_t\to 0$ for every start exactly when $0<\eta\lambda<2(1+\beta)$, and that it grows exponentially when $\lambda<0$ or $\eta\lambda>2(1+\beta)$. (b) For $\eta=0.1$ and $\beta=0.9$, the settings of Lab 2.2, find the curvature above which a direction expands. *(Hint: a polynomial z^2+az+b with real coefficients has both roots inside the unit circle exactly when $|b|<1$ and $|a|<1+b$.)*

2.3 CONSOLIDATION. ★ (Chaotic gradient descent.) Gradient descent with step $\eta=2$ on the double well $f(x)=\frac{1}{4}(x^2-1)^2$ is the map $x\mapsto 3x-2x^3$. (a) Show that the minima $x=\pm1$ are unstable fixed points. (b) Substitute $x=\sqrt{2}\cos\varphi$ and show that the map becomes $\varphi\mapsto 3\varphi+\pi$ modulo 2π, and that $[-\sqrt{2},\sqrt{2}]$ is invariant. (c) Deduce that two orbits that start 2^{-24} apart in φ are of order one apart after about 15 steps, and check numerically that the average of $\ln|\Phi'(x_t)|$ along an orbit is $\ln 3\approx 1.099$. *(Hint:* $\cos 3\varphi=4\cos^3\varphi-3\cos\varphi$.*)*

2.4 CONSOLIDATION. (Twins disagree less.) Under the hypotheses of Proposition 2.12, let the outcome be the class that a run predicts for one test example, and let $p_k=\mathbb{P}(\text{the run predicts } k\mid\mathcal{G})$. Show that two twins disagree on the example with probability $1-\mathbb{E}\sum_k p_k^2$ and two independent runs with probability $1-\sum_k(\mathbb{E}p_k)^2$, that the first is at most the second, and that they are equal exactly when every p_k is almost surely constant.

2.5 LAB. (A smaller step.) Run `02-instability-twins.py` with the learning rate 0.025 instead of 0.1. Predict from Figure 2.2(c) whether the noisy-label twins will leave the rounding floor, then check, and relate the outcome to the fraction of mislabeled examples that the runs fit.

2.6 LAB. (Smooth activations.) Repeat the noisy-label part of Lab 2.3 with `activation="tanh"` in the configuration. Does the disagreement ratio change? Compare with the claim of Shamir et al. (2020) that smooth activations make predictions more reproducible, and say what your experiment can and cannot show about it.

2.7 LAB. (Where the exponent lives.) From the distances recorded in Lab 2.2, compute the growth rate of the logarithm of the distance over windows of 25 steps for the noisy-label one-ulp twins, and plot it against the step. Where is it largest, and what does that say about $\Lambda(t)$?

2.8 DESIGN. (Two GPU types.) A team wants to know whether moving a fine-tuning pipeline to another type of GPU changes the distribution of test accuracy. Seeds move the accuracy with a standard deviation of 0.4 points, and the budget is 40 runs. Give the runs per type, the test, the smallest shift detected with power 0.8 at level 0.05, and what must be held fixed and recorded.

2.9 DESIGN. (When does a run commit?) In the noisy regime of Lab 2.2, design an experiment with 200 runs that finds the step after which releasing a new data-order seed changes the test predictions less than half as much as releasing it at step zero, in the spirit of the onset experiment of Summers and Dinneen (2021). State the treatment and its levels, the response, the replicates, and the analysis.

2.10 CRITIQUE. A paper reports: "We fixed all random seeds (seed 42) for reproducibility. Our method improves test accuracy over the baseline by 0.4 points." Each method was trained once, on eight GPUs. Which level of Definition 2.3 does the statement establish? What would you ask the authors for, and what does Remark 2.4 say about whether the shared seed makes the comparison more precise?

2.11 CRITIQUE. (An invented report.) A team moves training to another type of GPU, sees the accuracy of a single run drop by 0.6 points, and concludes that the new GPU's arithmetic is less accurate. What else changed with the hardware, what should the drop be compared with, and which experiment would test the claim?

2.12 RESEARCH. (When does a run forget?) Proposition 2.10 covers strongly convex losses, the edge of stability of Cohen et al. (2021) places full-batch training where the sharpest direction neither contracts nor expands, and Figure 2.2(c) shows the instability switching on with the learning rate and the label noise. Open: predict the cumulative exponent $\Lambda(t)$ of a network from quantities measurable early in training, such as curvature relative to $2/\eta$, gradient noise, and the fraction of examples not yet fitted, and hence when a one-bit twin becomes a new draw.

Notes and References

Goldberg (1991) is the classic introduction to floating-point arithmetic, and Higham (2002) the reference for the rounding errors of summation, compensated summation included; Higham and Mary (2019) gave the probabilistic analysis behind the $\sqrt{n}\,u$ behavior of typical sums. Summation whose result does not depend on the order of the terms has been designed for parallel machines (Demmel and Nguyen 2015).

Sensitive dependence on initial conditions was found by Lorenz (1963) in a model of atmospheric convection, and Strogatz (2015) explains Lyapunov exponents and why the horizon of prediction grows only with the logarithm of the precision of the initial state. For training, Summers and Dinneen (2021)

gave the decisive experiment, Jordan (2024) replicated it while connecting run-to-run variance to the finite test set, and Kong and Tao (2020) and Cohen et al. (2021) studied gradient descent at learning rates where it does not converge in the classical sense.

On the reproducibility of machine learning research, Gundersen and Kjensmo (2018) surveyed 400 papers from two AI conferences and found that none documented every variable needed to reproduce it, Pineau et al. (2021) report on the code policy, the reproducibility challenge, and the checklist of NeurIPS 2019, and Bouthillier et al. (2019) showed that a result can be repeated exactly and still fail to hold under a slightly different experiment. Churn was named as a problem of shipping successive versions of a model (Milani Fard et al. 2016).

TRANSLATING THE LITERATURE. Summers and Dinneen (2021) call a source of randomness a *nondeterminism source*, our one-ulp twin a *random bit change*, and our churn *pairwise disagreement*; their *instability* is our amplification. Frankle et al. (2020) use *instability* for the rise of the error along the line between two runs spawned from one state, so that a run can be stable in their sense and unstable in ours. Zhuang et al. (2022) write ALGO for the randomness that seeds control and IMPL for the nondeterminism of the environment $\mathcal{E}$. Pham et al. (2020) call sources of randomness *NI-factors* and our repeats *fixed-seed identical training runs*; Nagarajan et al. (2018) call our repeatability *replicability*; and Jordan (2024) calls a twin made by multiplying one weight by 1.001 a *poke*.

References

Bouthillier, X., C. Laurent, and P. Vincent (2019). "Unreproducible Research is Reproducible". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 725–734.

Cohen, J. M., S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *International Conference on Learning Representations*.

Demmel, J. and H. D. Nguyen (2015). "Parallel Reproducible Summation". In: *IEEE Transactions on Computers* 64.7, pp. 2060–2070.

Fort, S., G. K. Dziugaite, M. Paul, S. Kharaghani, D. M. Roy, and S. Ganguli (2020). "Deep Learning versus Kernel Learning: An Empirical Study of Loss Landscape Geometry and the Time Evolution of the Neural Tangent Kernel". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 3259–3269.

Goldberg, D. (1991). "What Every Computer Scientist Should Know about Floating-Point Arithmetic". In: *ACM Computing Surveys* 23.1, pp. 5–48.

Goodman, S. N., D. Fanelli, and J. P. A. Ioannidis (2016). "What Does Research Reproducibility Mean?" In: *Science Translational Medicine* 8.341, 341ps12.

Gundersen, O. E. and S. Kjensmo (2018). "State of the Art: Reproducibility in Artificial Intelligence". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pp. 1644–1651.

He, H. (2025). *Defeating Nondeterminism in LLM Inference*. Connectionism (blog), Thinking Machines Lab. URL: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*. 2nd ed. Philadelphia: SIAM.

Higham, N. J. and T. Mary (2019). "A New Approach to Probabilistic Rounding Error Analysis". In: *SIAM Journal on Scientific Computing* 41.5, A2815–A2835.

IEEE (2019). *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019, revision of IEEE Std 754-2008. IEEE Computer Society.

Jordan, K. (2024). "On the Variance of Neural Network Training with respect to Test Sets and Distributions". In: *International Conference on Learning Representations*.

Kong, L. and M. Tao (2020). "Stochasticity of Deterministic Gradient Descent: Large Learning Rate for Multiscale Objective Function". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Lorenz, E. N. (1963). "Deterministic Nonperiodic Flow". In: *Journal of the Atmospheric Sciences* 20, pp. 130–141.

Milani Fard, M., Q. Cormier, K. Canini, and M. Gupta (2016). "Launch and Iterate: Reducing Prediction Churn". In: *Advances in Neural Information Processing Systems*. Vol. 29.

Nagarajan, P., G. Warnell, and P. Stone (2018). *Deterministic Implementations for Reproducibility in Deep Reinforcement Learning*. arXiv preprint 1809.05676.

NVIDIA (2026a). *cuBLAS Library Documentation: Results Reproducibility*. URL: https://docs.nvidia.com/cuda/cublas/ (visited on 10/17/2026).

NVIDIA (2026b). *cuDNN Developer Guide: Reproducibility (Determinism)*. URL: https://docs.nvidia.com/deeplearning/cudnn/latest/developer/misc.html (visited on 10/17/2026).

Pham, H. V., S. Qian, J. Wang, T. Lutellier, J. Rosenthal, L. Tan, Y. Yu, and N. Nagappan (2020). "Problems and Opportunities in Training Deep Learning Software Systems: An Analysis of Variance". In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 771–783.

Pineau, J., P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and H. Larochelle (2021). "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)". In: *Journal of Machine Learning Research* 22.164, pp. 1–20.

PyTorch Contributors (2026). *Reproducibility*. PyTorch 2.14 documentation, notes on randomness. URL: https://docs.pytorch.org/docs/2.14/notes/randomness.html (visited on 10/17/2026).

Shamir, G. I., D. Lin, and L. Coviello (2020). *Smooth Activations and Reproducibility in Deep Networks*. arXiv preprint 2010.09931.

Strogatz, S. H. (2015). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. 2nd ed. Westview Press.

Summers, C. and M. J. Dinneen (2021). "Nondeterminism and Instability in Neural Network Optimization". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 9913–9922.

Zhuang, D., X. Zhang, S. L. Song, and S. Hooker (2022). "Randomness in Neural Network Training: Characterizing the Impact of Tooling". In: *Proceedings of Machine Learning and Systems*. Vol. 4.

3 Many Tasks, Few Runs

Aggregates, intervals and multiplicity

THE CLAIM *"A higher median score over a few runs per game identifies the better algorithm."*

THE VERDICT Narrowed: rerun 100 times per game under one evaluation protocol, some claimed gains hold, while others are within noise or reverse.

The practice of the Atari 100k benchmark, 2019–2021, as audited by Agarwal et al. (2021)

IN THIS CHAPTER. Between 2019 and 2021, a succession of papers on the Atari 100k benchmark reported gains over earlier agents, mostly in the median human-normalized score over 3 to 20 runs per game, and the best were read as the new state of the art. Agarwal et al. (2021) trained five of these agents 100 times on each game: medians from a few runs scattered as widely as the gaps between agents, the sample median was biased, and some agents had been scored by a maximum. The method is to know what an aggregate over tasks estimates and how precisely: two levels of variation, the effective number of tasks behind the mean, the median and the interquartile mean, the stratified bootstrap and its shortfall with three runs, and the comparisons hidden in per-task tests, in maxima and in the choice of aggregate. In a twelve-task miniature built from digits, a baseline scored by its best checkpoint overtakes a better method, the aggregate decides between two others, and two hundred three-run replications show the median's interval missing a real gain in half of them.

3.1 The claim: a new state of the art every few months

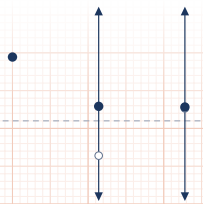
Reinforcement learning on Atari games is evaluated on a suite. The Arcade Learning Environment of Bellemare et al. (2013) made dozens of Atari 2600 games a common testbed, and Mnih et al. (2015) put the games on one scale: an agent's score on a game is rescaled so that a random policy scores 0 and a human player scores 1,

$$\text{human-normalized score} = \frac{\text{agent score} - \text{random score}}{\text{human score} - \text{random score}},$$

and the scores are summarized across games by their mean or their median. One number per agent made agents comparable at a glance, and the number became the result.

Kaiser et al. (2020) asked how well an agent can play after very little experience. Their model-based agent, SimPLe, learned from 100,000 interactions with each game, that is 400,000 frames with the usual frame skip of four, or about two hours of real-time play,[1] on 26 games, and they reported

[1] The exact budget in Kaiser et al. (2020) is $6400 \times 16 = 102{,}400$ interactions, 409,600 frames or 114 minutes at 60 frames per second.



the mean and standard deviation of five runs on each game, comparing with model-free agents game by game. The setting became the *Atari 100k* benchmark,[2] and its aggregate became the median human-normalized score over the 26 games.

A sequence of papers followed, each reporting gains over the agents it compared with. van Hasselt et al. (2019) changed a few hyperparameters of Rainbow (Hessel et al. 2018) (one update per environment step, 20-step returns, learning from replay after 1,600 transitions), and this *data-efficient Rainbow*, DER, beat SimPLe on 17 of the 26 games with five runs each.[3] In a preprint, Kielak (2020) trained Rainbow with eight updates per environment step, called it OTRainbow, for "overtrained", and reported a median of 0.204 against SimPLe's 0.102. CURL (Laskin et al. 2020) added a contrastive loss on image representations and reported a median of 0.175 over 20 runs, against 0.144 for SimPLe and 0.161 for DER: the "1.2x" gain on Atari in its abstract.[4] DrQ (Yarats et al. 2021) augmented the input images and reported a median of 0.268 over five runs, a new state of the art. SPR (Schwarzer et al. 2021) added a self-predictive loss and reported 0.415 over ten runs, "a 55% relative improvement" over the previous state of the art.

[2] The 26 games were those on which SimPLe or Rainbow obtained non-random results at 100k interactions (Kaiser et al. 2020, Section 6): the suite was selected with the help of the methods it was first used to compare.

[3] The SimPLe numbers in that comparison were those of SimPLe's first preprint; the conference version reports revised scores and concedes that tuned Rainbow is on par, better on 13 games and worse on 13 (Kaiser et al. 2020).

[4] The first two arXiv versions of CURL used 3 runs per game and reported a median of 0.24; the conference version moved to another code base and 20 runs.

FINDING 3.1 (Schwarzer et al. 2021, Table 1). On the 26 games of Atari 100k, the mean and median human-normalized scores of six agents were as follows; the runs per game are those of the original papers, as listed by Agarwal et al. (2021) and Kaiser et al. (2020). SPR led by both aggregates.

	SimPLe	DER	OTRainbow	CURL	DrQ	SPR
runs per game	5	5	5	20	5	10
mean	0.443	0.285	0.264	0.381	0.357	0.704
median	0.144	0.161	0.204	0.175	0.268	0.415

DrQ and then SPR were received as the new state of the art. The claim this chapter examines is the practice behind such readings: that the agent with the higher median over a few runs per game is the better agent. Prior results on the benchmark had been computed mostly from three or five runs per game, more rarely from ten or twenty (Agarwal et al. 2021).

The practice was reasonable. Five runs on each of 26 games is already 130 trainings per agent, and the median was chosen on purpose: on a few games an agent can reach several times the human score, and those games dominate a mean (by the mean, SimPLe is ahead of CURL and DrQ; by the median, behind both). The improvements were also, in part, real: DER and OTRainbow corrected a comparison against untuned baselines, the subject of Chapter 4, and SPR's lead was large. What no paper reported was how far its median would move when the runs were repeated.

3.2 The experiment: twenty-six games, a few runs, one number

The comparisons behind these claims share one design.

EXPERIMENTAL DESIGN · ATARI 100K COMPARISONS, 2019–2021

QUESTION	Is the new agent more sample-efficient than earlier agents on Atari?
UNITS	A run: one agent trained from scratch on one game for 100k interactions.
TREATMENT	The agent: the new method against three to six earlier methods.
TUNED	The new method's hyperparameters, sometimes per game; baselines as published.
HELD FIXED	The 26 games and the budget of 100k interactions.
NOT FIXED	The evaluation protocol of the baselines, their code bases, and the human reference scores used to normalize.
RESPONSE	A run's human-normalized score: its mean return over a set of evaluation episodes.
REPLICATES	3 to 20 runs per game, most often 3 or 5.
ANALYSIS	Mean over runs on each game; median and mean over games; point estimates set against numbers copied from earlier papers; win counts over games.

This design answers one question well: did the new agent's runs, on these games, average higher than the numbers printed in earlier papers? It cannot answer the question it was read as answering, because three of its features are invisible in its output.

First, the median of 26 game means is a random quantity, and one set of runs gives one draw of it. Its spread over reruns is not in the table, nor is the spread of the numbers it is compared with, although with five runs per game an interval could have been computed by resampling runs. The gaps between consecutive claims were small: 0.161 to 0.175 from DER to CURL, 0.204 to 0.268 from OTRainbow to DrQ.

Second, the comparison has no common control arm. Each paper ran its own agent and copied the baselines' numbers, so a difference in the table mixes the difference of agents with differences of evaluation protocol, code base and reference scores. Even one baseline's numbers moved between papers: SimPLe's median is 0.102 in Kielak (2020) and 0.144 in later tables, and DER's is 0.147 in early versions of CURL and 0.161 in the final one, through changes in the SimPLe results used and in the human reference scores.[5]

Third, a table over 26 games is a family of comparisons. A paper that bolds the best score on each game and counts the bold entries makes 26 comparisons at once, and a protocol that keeps the best of several evaluations makes several more per run, none of them adjusted for. None of the three is a flaw in the agents; each is a gap between what the design measures and what its single number was taken to mean.

[5] DrQ's own reported median moved between its arXiv versions, from 0.270 to 0.284 to 0.268 (Yarats et al. 2021).

3.3 The post-mortem: one hundred runs per game

Agarwal et al. (2021) measured what the design leaves out by running it a hundred times. For five agents, DER, OTRainbow, CURL, DrQ and SPR, and for DrQ(ϵ), DrQ with the standard exploration settings of the benchmark, they trained 100 independent runs on each of the 26 games and scored each run by its mean return over 100 evaluation episodes after training. DER and OTRainbow, which had no public code, were reimplemented in the Dopamine framework; CURL and SPR were run from their authors' code, and DrQ from code its authors provided. Subsampling these runs gives the distribution of every statistic that a paper with fewer runs could have reported.

> FINDING 3.2 (Agarwal et al. 2021, Section 3 and Appendix A). With 100 runs per game for each agent: (i) medians recomputed from as many runs as each paper had used (5, 10 or 20) vary widely, and the published medians lie far out in these distributions, some too high and some too low; (ii) the expected sample median changes with the number of runs, by an amount and in a direction that depend on the agent: for SPR the change from 5 to 100 runs, +0.03, is about 36% of its improvement over DrQ(ϵ), +0.08; (iii) 95% intervals for the median remain wide with 50 runs, and with no true difference at all two sets of 100 runs can differ by 20% in median, while the interquartile mean needs far fewer runs for the same precision; (iv) CURL's published scores were the maximum over 10 evaluations during training, and DrQ's code evaluated with a 5% chance of random actions instead of the standard 0.1%.

By the reported numbers OTRainbow was ahead of DER; with a hundred runs, DER "may in fact be better" than OTRainbow (Agarwal et al. 2021, Section 3), although part of this may come from implementation, since both were reimplemented. The protocol mattered more than the runs. Scored the way CURL was scored, by the maximum over evaluations during training, DER exceeds CURL's published scores by a wide margin, and scored at the end of training it is ahead of CURL: the gap between the two in the published table belongs to the protocol. SUNRISE (Lee et al. 2021), which reported improvements over DER, had taken the maximum over eight hyperparameter configurations separately for each game, with three runs each, and simulating that protocol on DER explained much of the gain (Agarwal et al. 2021, Appendix A.4). SPR's lead, by contrast, held: with a hundred runs it remained the strongest of the five.[6]

The audit found the same pattern on the 200-million-frame Atari benchmark, where the ranking of agents changed with the aggregate, on the DeepMind Control Suite, and on Procgen, where several claimed improvements had a probability of improvement of only 0.5 to 0.7. Its recommendations were interval estimates of aggregates from a stratified bootstrap, performance profiles, the interquartile mean in place of the median, the optimality gap and the probability of improvement, and publication of every run; the authors released a library, rliable, and all of their runs.

[6] Read from the axes of Figure 3 of Agarwal et al. (2021): the expected medians order the agents SPR, DrQ(ϵ), DER, OTRainbow, CURL at every number of runs from 3 to 100.

NARROWED

VERDICT. The claims that rested on a median of a few runs per game hold only where the gap between agents exceeds the uncertainty of that median and the agents were scored under one protocol. With a hundred runs per game, SPR stays ahead; DER, reported below OTRainbow and CURL, may be ahead of the first and is ahead of the second; and the advantage printed for CURL came from its evaluation protocol. The point estimates were not wrong as measurements; they could not carry the conclusions drawn from them.

The post-mortem did not show that these agents were poor, and its recommendations are not the only defensible ones. It argues for intervals rather than dichotomous tests, whereas earlier audits recommended tests: Henderson et al. (2018), whose seed experiment opens this book (**??**), proposed bootstrap intervals, significance tests and power analysis, and Colas et al. (2018, 2019) recommended Welch's t-test with corrections for multiple comparisons, warning that bootstrap tests have high false-positive rates with few runs. Jordan et al. (2020) built intervals with guaranteed coverage after finding that percentile bootstrap intervals failed 11% of the time with ten runs. These disagree about the form of an uncertainty statement, not about whether to make one; and the interquartile mean is not the true aggregate, only a choice of what to measure.

3.4 The method: what an aggregate estimates

An aggregate over a suite comes from a sample with two levels, tasks and runs within tasks; before computing an interval we need to know which level a claim is about and how the aggregate weighs the tasks.

TWO LEVELS OF VARIATION. A suite has J tasks, indexed by j, and each method is run n times on each task. The normalized score of run i on task j is Y_{ji}. Given the task, the runs are independent draws from the task's run distribution, with mean μ_j and variance σ_j^2, and the task mean $\bar{Y}_j = \frac{1}{n}\sum_i Y_{ji}$ estimates μ_j. A *suite-level* claim is about these J tasks, for instance about $\frac{1}{J}\sum_j \mu_j$: the agent is better on these 26 games. A *population-level* claim treats the tasks as a sample from a population of tasks: the agent is better on games like these. The two have different uncertainties.

Proposition 3.3 (Two levels of variation). *Let the pairs (μ_j, σ_j^2), $j = 1, \dots, J$, be independent and identically distributed, with $\omega^2 = \operatorname{Var}\mu_j$ and $\bar{\sigma}^2 = \mathbb{E}\sigma_j^2$ finite, and let the runs on task j be, given (μ_j, σ_j^2), independent with mean μ_j and variance σ_j^2. The suite mean $\bar{Y} = \frac{1}{J}\sum_j \bar{Y}_j$ satisfies*

$$\operatorname{Var}\left(\bar{Y} \,\middle|\, \mu_{1:J}, \sigma^2_{1:J}\right) = \frac{1}{J^2}\sum_{j=1}^{J}\frac{\sigma_j^2}{n}, \qquad \operatorname{Var}\bar{Y} = \frac{\omega^2}{J} + \frac{\bar{\sigma}^2}{nJ}. \tag{3.1}$$

PROOF. Given the tasks, the task means are independent with variances σ_j^2/n, and the variance of their average is the first formula. The law of

total variance adds the mean of this conditional variance, $\bar{\sigma}^2/(nJ)$, to the variance of the conditional mean $\frac{1}{J}\sum_j \mu_j$, which is ω^2/J because the μ_j are independent with variance ω^2. ■

More runs shrink only the second term; the first, ω^2/J, is fixed by the number of tasks. For two methods compared on the same tasks, apply the proposition to the per-task difference: ω^2 is then the variance of the effect across tasks, small when a method helps everywhere by the same amount and large when it helps on some tasks and hurts on others. The stratified bootstrap resamples runs within fixed tasks, so it estimates the conditional variance and supports suite-level claims. A test across tasks, such as the signed-rank test of Section 3.5, treats the tasks as the sampling units and supports population-level claims, but only to the extent that the tasks are a sample; a suite chosen by hand, like the 26 games, is not.

AGGREGATES. Five summaries of a method's scores are in common use (Agarwal et al. 2021). The *mean* is $\frac{1}{J}\sum_j \bar{Y}_j$, and the *median* is $\mathrm{med}(\bar{Y}_1, \dots, \bar{Y}_J)$. The *interquartile mean*, IQM, pools all nJ scores, discards the lowest and the highest quarter, and averages the middle half: it is the 25% trimmed mean of the pooled runs. The *optimality gap* at a target γ is $\mathrm{OG}_\gamma = \gamma - \frac{1}{nJ}\sum_{j,i} \min(Y_{ji}, \gamma)$, the average shortfall below the target, with $\gamma = 1$ for human level. The *probability of improvement* of method A over method B is the average over tasks of $\mathbb{P}(Y_j^{\mathrm{A}} > Y_j^{\mathrm{B}}) + \frac{1}{2}\mathbb{P}(Y_j^{\mathrm{A}} = Y_j^{\mathrm{B}})$ for independent runs of the two methods, estimated on each task by the Mann–Whitney statistic (Proposition A.16). Behind all of them stands the *score distribution* $\hat{F}(t) = \frac{1}{nJ}\sum_{j,i} \mathbf{1}\{Y_{ji} > t\}$, the fraction of runs above t, an unbiased estimate of $\frac{1}{J}\sum_j \mathbb{P}(Y_j > t)$; plotted against t it is a performance profile in the sense of Dolan and Moré (2002).

Each aggregate ignores something. The median sees the magnitude of the scores only on the one or two tasks in the middle: when it is positive, lowering the scores of every task below the middle, even to zero, leaves it unchanged. The mean is dominated by the tasks with the largest scores and the largest run-to-run variance; the interquartile mean ignores the extremes, which may include the tasks on which a method fails; the probability of improvement ignores the size of the improvement, and the optimality gap everything above its target. Each answers a different question, to be chosen before the scores are seen.

PRECISION: THE EFFECTIVE NUMBER OF TASKS. The median of J task means depends on all of them but moves with the run noise of only one or two: when the tasks are far apart, the task in the middle stays in the middle. The next theorem makes this exact for every aggregate that is a weighted sum of order statistics of the task means.

Theorem 3.4 (Effective number of tasks). *Let $X_j = \mu_j + e_j$ for $j = 1, \dots, J$, where $\mu_1 < \mu_2 < \cdots < \mu_J$ and the errors e_j are independent with mean 0 and variances s_j^2. Let $L(x) = \sum_{k=1}^{J} c_k x_{(k)}$, with weights $c_k \geq 0$ summing to 1, let $V = \sum_k c_k^2 s_k^2$, and let $\mathcal{A}$ be the event $X_1 < X_2 < \cdots < X_J$ that the*

errors do not reorder the tasks.

(i) *On $\mathcal{A}$, $L(X)-L(\mu)=\sum_k c_k e_k$; everywhere, $|L(X)-L(\mu)|\le \max_j |e_j|$.*

(ii) *The root mean squared error $R=\big(\mathbb{E}(L(X)-L(\mu))^2\big)^{1/2}$ satisfies $|R-\sqrt{V}|\le 2\big(\mathbb{E}\big[\max_j e_j^2\,\mathbf{1}_{\mathcal{A}^c}\big]\big)^{1/2}$.*

(iii) *If the errors are Gaussian with $s_j\le s$ and consecutive means differ by at least $g>0$, then*

$$|R-\sqrt{V}|\le 2s\left(3J(J-1)\,\Phi\big(-g/(s\sqrt{2})\big)\right)^{1/4}. \tag{3.2}$$

Proof. The plan is to compare $L(X)$ with the linear statistic $Z=\sum_k c_k e_k$, which it equals when the tasks keep their order, and to bound the difference on the event that they do not.

Part (i). On $\mathcal{A}$ the kth smallest of the X_j is X_k, so $L(X)=\sum_k c_k(\mu_k+e_k)=L(\mu)+Z$, where $L(\mu)=\sum_k c_k\mu_k$ because the μ_k are sorted. For the bound everywhere we use that order statistics are 1-Lipschitz in the maximum norm. Suppose $x_j\le y_j+\delta$ for every j. The k indices that carry the k smallest y_j carry k values x_j, each at most $y_{(k)}+\delta$, so $x_{(k)}\le y_{(k)}+\delta$; exchanging x and y gives $|x_{(k)}-y_{(k)}|\le\max_j|x_j-y_j|$. With $y=\mu$, and the weights nonnegative with sum 1,

$$|L(X)-L(\mu)|\le\sum_k c_k|X_{(k)}-\mu_k|\le\max_j|e_j|.$$

Part (ii). Let $D=L(X)-L(\mu)-Z$. By part (i), $D=0$ on $\mathcal{A}$. Since also $|Z|\le\sum_k c_k|e_k|\le\max_j|e_j|$, the same part gives $|D|\le 2\max_j|e_j|$ on $\mathcal{A}^c$. The errors are independent with mean 0, so $\mathbb{E}Z^2=V$. Minkowski's inequality for the norm $\big(\mathbb{E}(\cdot)^2\big)^{1/2}$ bounds $|R-\sqrt{V}|$ by $\big(\mathbb{E}D^2\big)^{1/2}$, which is at most $2\big(\mathbb{E}[\max_j e_j^2\,\mathbf{1}_{\mathcal{A}^c}]\big)^{1/2}$.

Part (iii). If $X_k<X_{k+1}$ for every k, then the X_j are in order, so $\mathcal{A}^c$ is contained in the union of the $J-1$ events $\{X_k\ge X_{k+1}\}$. Each says that $e_k-e_{k+1}\ge\mu_{k+1}-\mu_k\ge g$, and e_k-e_{k+1} is Gaussian with mean 0 and variance at most $2s^2$, so the union bound gives $\mathbb{P}(\mathcal{A}^c)\le(J-1)\Phi(-g/(s\sqrt{2}))$. Next we bound the expectation in part (ii) by the Cauchy–Schwarz inequality, then the maximum of the fourth powers by their sum, using $\mathbb{E}e_j^4=3s_j^4\le 3s^4$ for Gaussian errors:

$$\mathbb{E}\big[\max_j e_j^2\,\mathbf{1}_{\mathcal{A}^c}\big]\le\Big(\mathbb{E}\max_j e_j^4\Big)^{1/2}\mathbb{P}(\mathcal{A}^c)^{1/2}\le\big(3Js^4\big)^{1/2}\big((J-1)\Phi(-g/(s\sqrt{2}))\big)^{1/2}.$$

Inserting this bound in part (ii) gives (3.2). ∎

When the tasks are well separated, then, each such aggregate behaves like a weighted average of independent task errors; with equal run noise, $s_j=s$, its variance is that of an average of $J_{\mathrm{eff}}=1/\sum_k c_k^2$ independent tasks. The mean has $J_{\mathrm{eff}}=J$; the median has $J_{\mathrm{eff}}=1$ for an odd number of tasks and 2 for an even number; a 25% trimmed mean of the task means has

run distribution	median	interquartile mean	mean
normal	0.64	0.84	1
logistic	0.82	1.04	1
Laplace	2.00	1.63	1
Student's t, 3 degrees of freedom	1.62	1.97	1
normal, 5% contaminated with SD 10	3.45	4.48	1

Table 3.1. Asymptotic efficiency of the median and the interquartile mean relative to the mean, v_{mean}/v from Proposition 3.5, for one sample from five symmetric location families. Computed by numerical integration in `03-aggregates-reruns.py`.

$J_{\text{eff}} = J/2$ when J is a multiple of 4, and so does the interquartile mean of pooled runs when the scores of different tasks do not overlap. On Atari 100k the median carries the run noise of two games and the interquartile mean that of about thirteen. The bound (3.2) is loose: in a simulation with $J = 13$ Gaussian tasks (Lab 3.2), the median's error is already $0.995s$ when consecutive tasks are $g = 4s$ apart, and $0.891s$ at $g = 2s$.

The opposite extreme is a suite in which every task has the same run distribution. The pooled runs are then one sample from a location family, and the classical asymptotics apply.

Proposition 3.5 (One sample). *Let $Y_1, \dots, Y_N$ be independent with density $f(y-\theta)$, where f is symmetric about 0, has variance σ^2, and is continuous and positive at 0 and at its quartiles $\pm q$. As $N \to \infty$, the sample mean, the sample median and the 25% trimmed mean are asymptotically normal around θ, with variances v/N given by*

$$v_{\text{mean}} = \sigma^2, \qquad v_{\text{med}} = \frac{1}{4f(0)^2}, \qquad v_{\text{IQM}} = 4\Big(\int_{-q}^{q} y^2 f(y)\,\mathrm{d}y + \frac{q^2}{2}\Big).$$

Proof sketch. For the median, $\sqrt{N}(\text{med} - \theta) \le t$ holds exactly when at least half of the Y_i lie below $\theta + t/\sqrt{N}$; this count is binomial with success probability $\frac{1}{2} + f(0)t/\sqrt{N} + o(N^{-1/2})$, and the central limit theorem for the binomial gives the limit $\Phi(2f(0)t)$. For the trimmed mean, Stigler (1973) proves asymptotic normality with the variance of the influence function, here the winsorized observation $\max(-q, \min(Y-\theta, q))$ divided by $1 - 2\cdot\frac{1}{4}$; its second moment is the integral plus the two tails, each of mass $\frac{1}{4}$ at $\pm q$, which gives v_{IQM}. Quantiles and trimmed means are treated in general in van der Vaart (1998, Chapters 21–22). ∎

Table 3.1 gives the efficiencies. For normal runs the median wastes a third of the data and the interquartile mean a sixth; for heavy-tailed runs both beat the mean, and the interquartile mean is never far from the better of the other two. Between the two extremes, with normal run noise, the median's effective number of tasks lies between 1 and $2J/\pi \approx 0.64J$, and the interquartile mean's between about $J/2$ and $0.84J$. This is the case for the interquartile mean with few runs: it gives up little when the runs are well

behaved, much less than the median, and it does not let one task with wild runs dominate, as the mean does.

The sample median is also biased: it is an order statistic of noisy values, and its expectation moves away from the median of the true task means. Suppose two tasks are tied at μ, with $r-1$ tasks far below them and r far above, so that $J = 2r+1$ and the true median is μ. Then the sample median is the larger of the two tied estimates, with expectation $\mu + s\,e_2 = \mu + s/\sqrt{\pi}$, where e_2 is the expected maximum of two standard normals (Chapter 1); with the tied pair one place higher the bias is $-s/\sqrt{\pi}$. The bias shrinks with s as runs are added, and its sign depends on the tasks near the middle, which differ between methods: this is the bias that Agarwal et al. (2021) measured.

INTERVALS: THE STRATIFIED BOOTSTRAP. Suite-level uncertainty comes from the runs, so an interval should resample runs and keep the tasks. The *stratified bootstrap* draws n runs with replacement within each task, recomputes the aggregate, and repeats; the 2.5% and 97.5% quantiles of the recomputed values form the percentile interval (Definition A.2). For a difference between methods, both are resampled. With few runs the interval is too narrow, by a factor that is exact for the mean.

Proposition 3.6 (The bootstrap with n runs). *Given the scores, the stratified bootstrap variance of the mean aggregate equals $\frac{n-1}{n}$ times the unbiased estimate $\frac{1}{J^2}\sum_j S_j^2/n$ of its conditional variance (3.1), where S_j^2 is the sample variance of the n runs on task j.*

PROOF. Given the scores, the n resampled runs of task j are independent draws from the empirical distribution of its runs, which has mean $\bar{Y}_j$ and variance $\frac{1}{n}\sum_i (Y_{ji}-\bar{Y}_j)^2 = \frac{n-1}{n}S_j^2$. Their mean therefore has variance $\frac{n-1}{n}S_j^2/n$, the tasks are resampled independently, and the mean aggregate averages the J resampled task means. ■

With three runs the bootstrap standard error is $\sqrt{2/3} \approx 0.82$ times the usual one, and a percentile interval spans about $\pm 1.96 \times 0.82 = \pm 1.60$ standard errors, which covers 89% of the time for normal estimates, before counting the error of estimating each σ_j^2 from three runs; each task contributes at most ten distinct resampled means. Agarwal et al. (2021, Figure 6) found this undercoverage with three runs on Atari, and percentile intervals adequate from about ten runs per task. The median and the interquartile mean share the shortfall to first order. Widening the interval by $\sqrt{n/(n-1)}$ removes the part that Proposition 3.6 identifies, but not the rest.

3.5 The method: many comparisons

A suite multiplies comparisons: one per task, one per aggregate, one per baseline, and, hidden inside a protocol, one per evaluation or configuration over which a maximum is taken.

Proposition 3.7 (Many true nulls). *Suppose that m_0 of m tests concern true null hypotheses and that each of these rejects with probability α. Then the expected number of false rejections is $m_0\alpha$, and the probability of at least one is at most $m_0\alpha$; if the m_0 tests are independent, then it equals $1-(1-\alpha)^{m_0}$.*

PROOF. The number of false rejections is a sum of m_0 indicators with mean α each, so its expectation is $m_0\alpha$; the union bound gives the second statement. If the tests are independent, then the probability that none of them rejects is the product $(1-\alpha)^{m_0}$. ■

For a method no different from its baseline on each of 26 games, two-sided per-game tests at level 0.05 reject 1.3 times on average, and at least once with probability $1-0.95^{26}\approx 0.74$ when the games are independent. The *family-wise error rate*, the probability of at least one false rejection, is controlled by Bonferroni's correction, which tests each hypothesis at level α/m, and more powerfully by the step-down procedure of Holm (1979): order the p-values, compare the kth smallest with $\alpha/(m-k+1)$, and reject until the first comparison fails. Both are valid whatever the dependence between the tests (Proposition A.6 and Theorem A.7): with probability at least $1-\alpha$ every true null has p-value above α/m_0, and then Holm's procedure stops before the first true null. The *false discovery rate*, the expected proportion of false rejections among the rejections, is controlled at α by the step-up procedure of Benjamini and Hochberg (1995) for independent or positively dependent tests (Theorem A.8); under arbitrary dependence the level must be divided by $\sum_{k=1}^{m} 1/k$ (Benjamini and Yekutieli 2001). The choice follows the claim: "B is better on games X, Y and Z" is a list of discoveries, for which the false discovery rate fits; "B is never worse" is a claim about the whole family, which needs the family-wise rate.

TASKS AS THE SAMPLING UNITS. Demšar (2006) studied the same problem for classifiers compared over many data sets. Surveying papers at the International Conference on Machine Learning from 1999 to 2003, he found paired t-tests on each data set to be almost the only test in use, often summarized by counts of wins and losses. He recommended treating the data sets as the sampling units: for two methods, the Wilcoxon signed-rank test on the per-data-set differences (Wilcoxon 1945); for several, Friedman's test on the ranks of the methods within each data set (Friedman 1937), followed by Nemenyi's test for all pairs or Holm's procedure for comparisons with a control, drawn as a critical-difference diagram. These tests answer the population-level question of Proposition 3.3. A caution came later: the tests that follow Friedman's compare two methods through mean ranks that depend on the other methods in the comparison, so adding a third method can change whether the first two differ; pairwise signed-rank tests with a multiplicity correction avoid this (Benavoli et al. 2016).

HIDDEN MAXIMA. A protocol that reports the best of several evaluations makes a multiple comparison inside every run. The cost is easy to compute when the evaluations of one run are noisy measurements around a common value.

Proposition 3.8 (The best of k evaluations). *Let $X_i = \mu + \sigma\big(\sqrt{\rho}\,Z_0 + \sqrt{1-\rho}\,Z_i\big)$ for $i = 1, \dots, k$, with $Z_0, \dots, Z_k$ independent standard normal and $0 \le \rho \le 1$; the X_i have mean μ, standard deviation σ and pairwise correlation ρ. Then $\mathbb{E}\max_i X_i = \mu + \sigma\sqrt{1-\rho}\,e_k$.*

Proof. The shared term $\sigma\sqrt{\rho}\,Z_0$ is the same for every i, and $\sqrt{1-\rho} \ge 0$, so $\max_i X_i = \mu + \sigma\sqrt{\rho}\,Z_0 + \sigma\sqrt{1-\rho}\max_i Z_i$. Taking expectations, with $\mathbb{E}Z_0 = 0$ and $\mathbb{E}\max_i Z_i = e_k$, proves the claim. ■

The maximum rewards the part of the evaluation noise that is not shared between evaluations: the episode-to-episode noise of a finite evaluation, and the fluctuation of the policy between checkpoints. With ten evaluations, $e_{10} \approx 1.54$, so the best evaluation of a run that has stopped improving lies about $1.54\sqrt{1-\rho}$ standard deviations above its typical evaluation; a run that is still changing also contributes its best moment of training, a form of early stopping on the evaluation data. A maximum over eight hyperparameter configurations per game, the protocol of SUNRISE, is the best of $k = 8$ trials of Chapter 4, with $e_8 \approx 1.42$. In each case the published comparison set a maximum against an average.

The garden of forking paths. Gelman and Loken (2013, 2014) pointed out that a researcher who would have analyzed the data differently had they come out differently has made multiple comparisons, even when only one analysis was run and nothing was hidden. A suite of tasks has many such paths, and papers on Atari 100k took different ones: the aggregate, the normalization and its reference scores, the tasks, the number of runs and which runs to keep, the evaluation protocol, the baselines to rerun. Simmons et al. (2011) showed in psychology how a few such choices, each defensible, inflate the rate of false positives far above its nominal level, and the preprint of Dehghani et al. (2021) showed for machine-learning benchmarks how the choice of tasks alone changes which model ranks first. The remedy is the one experimental sciences use: fix the analysis before the data arrive, report every aggregate a reader might want, and publish the runs so that others can take the paths not taken.

3.6 In the laboratory: a suite of twelve tasks

The miniature of Atari 100k is a suite of twelve tasks built from the digits, with a fixed budget of 400 optimizer steps per task and normalized scores in which a simple reference plays the human. Method A is the volume's standard MLP. Method B adds label smoothing, which in a pilot of 16 runs per task was ahead of A by the mean, the median and the interquartile mean, so that every aggregate has the same correct answer; method C adds dropout and serves to show how aggregates disagree.

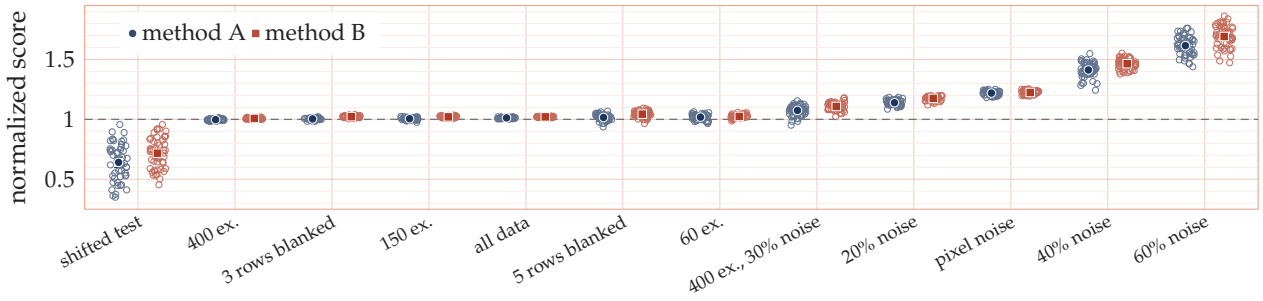


Figure 3.1. The suite of Lab 3.1: normalized scores of 50 runs of method A (ink, left of each task) and method B (vermilion, right) on each of the twelve tasks, ordered by A's mean (hollow marks, jittered horizontally), with the means and 95% t-intervals (filled marks; most intervals are shorter than the marks). Seeds 0–49 for A and 1000–1049 for B. The dashed line is the 1-nearest-neighbor reference.

LAB 3.1 · A SUITE OF TWELVE TASKS

SCRIPT
`03-aggregates-suite.py`

SETUP. Twelve tasks: the digits with all 1258 training examples or 400, 150 or 60 of them; with 20, 40 or 60% of the training labels replaced at random, or 30% on 400 examples; with Gaussian pixel noise; with the bottom 3 or 5 rows blanked; and with the test images shifted one pixel to the right. Each task gets 400 steps of SGD with momentum 0.9, learning rate 0.05 and batch 32. Method A is an MLP with two hidden layers of 64 units; B adds label smoothing 0.1; C adds dropout 0.2. A run's score is (accuracy − 0.1)/(reference − 0.1), the reference being the test accuracy of a 1-nearest-neighbor classifier on the same training set. Fifty runs per task and method (seeds $1000m + i$ for method m), 1800 runs in two to three minutes, each also evaluated every 40 steps for a protocol that keeps the best evaluation. Stratified bootstrap intervals (5000 resamples) from the first 3 runs of each task and from all 50.

PREDICT FIRST. B is ahead of A on every task. With three runs per task, which aggregate gives the narrowest interval for B − A? Scored by its best of ten evaluations, does A overtake B?

LOOK FOR. The spread of the runs on each task against the gap between A and B; how far each aggregate moves between 3 and 50 runs; the aggregate by which C beats A.

Figure 3.1 shows the suite. On every task B's mean is above A's, by between 0.005 (pixel noise) and 0.076 (the shifted test set and 60% label noise). The tasks differ in scale and noise as Atari games do. Without label noise, apart from the shifted test set, the scores lie between 1.00 and 1.22, with run standard deviations between 0.005 and 0.026; with label noise, where the reference copies the wrong labels, they reach 1.1 to 1.7; on the shifted test set A scores 0.641 ± 0.044 (mean and 95% t-interval over 50 runs), with a run standard deviation of 0.155. That one task contributes 62% of the run-noise variance of A's mean aggregate, and the two noisiest

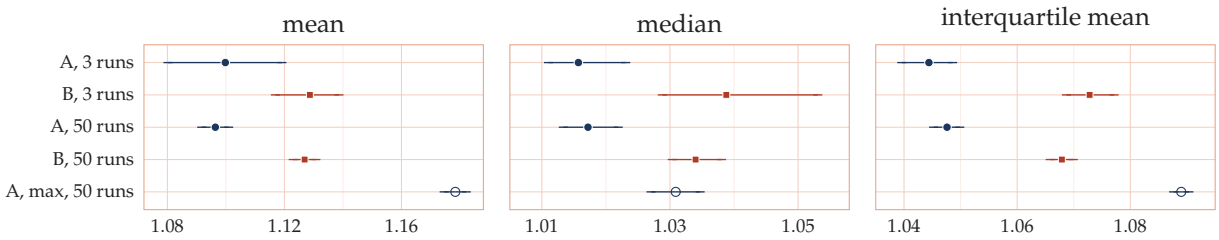


Figure 3.2. Aggregates over the twelve tasks of Lab 3.1, with 95% stratified bootstrap percentile intervals (5000 resamples), from the first 3 runs of each task and from all 50: method A (ink discs), method B (vermilion squares), and method A scored by its best of ten evaluations during training (hollow ink circle). Each panel has its own horizontal scale.

label-noise tasks another 27%.

Figure 3.2 gives the aggregates. With all 50 runs, B − A is +0.0305 by the mean, with 95% stratified bootstrap interval [0.0245, 0.0363], +0.0168 [0.0109, 0.0224] by the median and +0.0203 [0.0172, 0.0235] by the interquartile mean. From the first three runs of each task the same differences are +0.0288 [0.0064, 0.0515], +0.0231 [0.0090, 0.0354] and +0.0285 [0.0225, 0.0352]: every interval excludes zero, but the three-run interval of the interquartile mean is narrow and misses the fifty-run value, as Proposition 3.6 warns. The probability of improvement of B over A is 0.770 [0.744, 0.796] with 50 runs: on a typical task a run of B beats a run of A three times in four.

Scored by the best of its ten evaluations, A gains 0.082 by the mean, 0.014 by the median and 0.041 by the interquartile mean. The gains come from the label-noise tasks, +0.25 on 400 examples with 30% noise and +0.37 with 60%, where test accuracy peaks early and falls as the network fits the wrong labels. Against B scored at the end of training, this A "wins" by the mean, +0.0515 [0.0460, 0.0570], and by the interquartile mean, +0.0211 [0.0187, 0.0236], and ties by the median, −0.0032 [−0.0087, 0.0021]; scored the same way, B is ahead again by every aggregate. This is the comparison of CURL with DER in miniature: the protocol moved the result by more than the method did.

With dropout as the new method, the aggregates disagree even with 50 runs. C − A is +0.0384 [0.0320, 0.0445] by the mean and +0.0145 [0.0110, 0.0178] by the interquartile mean, but −0.0053 [−0.0106, −0.0001] by the median: dropout helps on the label-noise tasks, by up to 0.22, and slightly hurts on the three tasks in the middle of the suite, by 0.004 to 0.009, and C is ahead on 7 of the 12 tasks. Treating the tasks as the sampling units gives the population-level view of Proposition 3.3: the per-task differences B − A have a standard deviation of 0.026 across tasks, a task-level standard error of 0.0074 for their mean, and all twelve are positive (Wilcoxon signed-rank test, $p = 0.0005$), whereas for C − A the standard deviation is 0.071 and $p = 0.23$, so on tasks like these an advantage of C is not established. Friedman's test on the three methods gives $p = 0.009$, with average ranks 2.58, 1.33 and 2.08 for A, B and C, and only B and A differ by more than

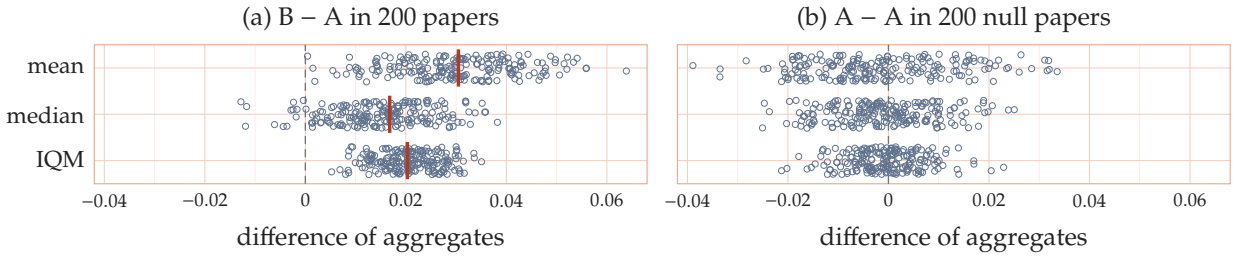


Figure 3.3. Replications of a three-run paper (Lab 3.2). (a) B − A by the mean, the median and the interquartile mean in 200 papers with 3 runs per task and method, drawn without replacement from the 50 runs of Lab 3.1 (seed 2026); hollow marks are papers, jittered vertically, and the vermilion bars are the differences computed from all 50 runs. (b) The same aggregates for 200 null papers that compare two disjoint sets of 3 runs of A.

Nemenyi's critical difference, 0.96 at level 0.05.

LAB 3.2 · TWO HUNDRED THREE-RUN PAPERS

SCRIPT
`03-aggregates-reruns.py`

SETUP. From the 50 runs of Lab 3.1, 200 papers, each with 3 runs per task of A and 3 of B drawn without replacement (seed 2026), and 200 null papers comparing two disjoint sets of 3 runs of A. For each paper: B − A by the mean, the median and the interquartile mean; 95% stratified bootstrap intervals (1000 resamples); Welch's test on each task, with Holm's and the Benjamini–Hochberg adjustments. The papers share runs of a finite bank, which slightly understates their spread. The script also computes Table 3.1 and the simulation after Theorem 3.4, in under a second once the bank exists.

PREDICT FIRST. In how many of the 200 papers does the median name A the winner? In how many null papers does at least one task differ at level 0.05 by Welch's test?

LOOK FOR. The spread of B − A across papers for each aggregate against its fifty-run value; the same spread in the null papers; the coverage of the three-run intervals; what Holm's procedure does to the per-task "wins".

Figure 3.3 shows the 200 papers. The standard deviation of B − A across papers is 0.0097 for the median, 0.0118 for the mean and 0.0060 for the interquartile mean, against fifty-run differences of 0.0168, 0.0305 and 0.0203. The median names A the winner in 12 papers (6.0%); the mean and the interquartile mean never do. The ratio of gap to spread decides it: 3.4 for the interquartile mean, 2.6 for the mean, 1.7 for the median. A paper's interval for B − A excludes zero in 95% of the papers for the interquartile mean, 78.5% for the mean, and 53% for the median: half of the papers that report the median with an honest interval cannot claim the improvement that is there.

The null papers show what three runs produce with nothing to find. The middle 95% of A − A runs from −0.019 to +0.019 for the median, as large as B's real gain by that aggregate, and from −0.015 to +0.014 for the

interquartile mean. Three-run intervals cover the fifty-run values in 82% to 89% of the papers for the mean, 77.5% to 82% for the median and 87% to 88% for the interquartile mean, against a nominal 95%, and the intervals for A − A exclude zero in 18%, 14% and 12% of the null papers. The average three-run median of A is 1.0208, above its fifty-run median 1.0172 by 0.0036: the bias of the sample median, here from the nearly tied tasks in the middle of the suite.

The spread of A's aggregates over the papers is 0.0093 for the mean, exactly the value of Proposition 3.3, 0.0068 for the median and 0.0048 for the interquartile mean, below the separated-task predictions of Theorem 3.4, 0.0098 and 0.0058, because the middle tasks of this suite lie closer together than their run noise. The mean is the noisiest of the three, although it uses every run, because one task with wild runs carries most of its variance.

Per-task tests show multiplicity directly. In the null papers, 33.5% have at least one task that differs at level 0.05 by Welch's test, against $1-0.95^{12} = 46\%$ predicted by Proposition 3.7 for exact independent tests (with three runs per arm Welch's test is conservative); after Holm's or the Benjamini–Hochberg procedure, 2.5% do. For B against A, 92.5% of the papers find at least one task significantly in favor of B by the unadjusted Welch test at level 0.05, and 18.5% after Holm's procedure: with three runs per task, honest per-task claims have little power. Among the 495 subsets of 8 of the 12 tasks, with all 50 runs, the verdict on C against A reverses for 42% of the subsets by the median, 18% by the interquartile mean and none by the mean; B, ahead on every task, is ahead on every subset.

The miniature reproduces the anatomy of the Atari findings: wide spreads of aggregates from three runs, a median less precise than the interquartile mean and biased, intervals that undercover, a maximum that outweighs a real method effect, and aggregates that disagree. It does not reproduce their size. The median picked the wrong winner in 6% of our papers because B's gain is 1.7 spreads of the three-run median, while on Atari 100k the median needed 50 to 100 runs before its intervals became small; Atari runs are noisier relative to the gaps between agents, and a noisier suite would come closer (Exercise 3.7).

3.7 Protocol

PROTOCOL 3.1 · REPORTING RESULTS OVER A SUITE OF TASKS

- ☐ Before running, write down the suite, the score normalization and its reference scores, the evaluation protocol, and one primary aggregate with its interval; say whether the claim is about these tasks or about tasks like them.
- ☐ Rerun every baseline under the same protocol, budget and code base as the new method; never set your runs against numbers copied from papers.
- ☐ Use at least ten runs per task when the budget allows; with fewer,

say so, and widen bootstrap intervals by $\sqrt{n/(n-1)}$.

- ☐ Report the primary aggregate of each method, and of the difference between methods, with a 95% stratified bootstrap interval; report the interquartile mean, the median and the mean together, with the performance profile.
- ☐ For a claim about tasks like these, add a test across tasks: the signed-rank test for two methods, Friedman's test with Holm's procedure against a control for several.
- ☐ Treat per-task claims as a family: Holm's procedure for claims of wins, the Benjamini–Hochberg procedure for lists of discoveries, or the probability of improvement with its interval instead.
- ☐ Never report a maximum over evaluations, configurations or seeds without the number it was taken over, and report the score at the end of training beside it.
- ☐ Publish the score of every run.

The protocol does not protect against a suite chosen, as the 26 games were, with the help of the methods it compares, nor against the slow overfitting of a community that reuses one suite for years; it does not make reimplemented baselines faithful (Chapter 4); and its intervals are conditional on the tasks. Its cost is runs, often more than a project can afford. Then spend the runs where the aggregate is sensitive, on the tasks near the middle for the median and the interquartile mean, rerun the strongest baseline rather than every one, report wider intervals rather than none, and say which claims the budget could not support.

Exercises

3.1 CONSOLIDATION. (Bonferroni and Šidák.) (a) Prove that testing each of m hypotheses at level α/m controls the family-wise error rate at α. (b) Show that for independent tests the level $1-(1-\alpha)^{1/m}$ gives a family-wise error rate of exactly α when every null is true (Šidák 1967), and that it exceeds α/m.

3.2 CONSOLIDATION. (Three procedures on one list.) Twelve per-task tests give the p-values 0.001, 0.004, 0.006, 0.012, 0.019, 0.030, 0.041, 0.060, 0.20, 0.35, 0.60 and 0.90. At $\alpha = 0.05$, find the hypotheses rejected by Bonferroni's correction, by Holm's procedure and by the Benjamini–Hochberg procedure, and say which claim each set of rejections supports.

3.3 CONSOLIDATION. (The median of tied tasks.) (a) Three tasks have means $\mu_1 = \mu_2 = 0$ and $\mu_3 = M$, and independent $\mathcal{N}(0, s^2)$ errors. Show that as $M \to \infty$ the sample median has expectation tending to $s/\sqrt{\pi}$ and variance tending to $s^2(1-1/\pi)$. (b) Use Proposition 3.5

to show that when all J tasks are tied, J times the variance of the median of the task means tends to $\pi s^2/2$. (c) For $J = 26$, with `scipy`'s convention of trimming $\lfloor J/4 \rfloor$ values at each end, find the effective number of tasks of the trimmed mean of well-separated task means.

3.4 CONSOLIDATION. (Interval coverage with few runs.) (a) Use Proposition 3.6 to show that, for normal task means with known variances, the percentile interval of the mean aggregate covers with probability $2\Phi\big(1.96\sqrt{(n-1)/n}\big) - 1$, and evaluate this for $n = 3, 5, 10$. (b) Why is the coverage in Lab 3.2 lower still?

3.5 LAB. (The aggregate decides.) In Lab 3.1, method C is ahead of A by the mean and the interquartile mean and behind by the median. Explain this from the per-task differences C – A that the script prints and the weights of each aggregate. Which aggregate would you have pre-registered for a claim about dropout?

3.6 LAB. (More runs per paper.) Rerun Lab 3.2 with 5 and with 10 runs per paper. Record the wrong-sign rates, how often the interval for B – A excludes zero, and the coverage, and compare the coverage with Exercise 3.4.

3.7 LAB. (A noisier suite.) Halve the budget of every task, or raise its label noise, rerun both labs, and measure how often the median names the wrong winner; relate the change to the ratio of gap to spread of each aggregate.

3.8 DESIGN. (Twenty-six games, three hundred runs.) A new agent is to be compared with two published baselines on the 26 games of Atari 100k, with compute for 300 runs in all. Decide the runs per game of each method, which baselines to rerun, the evaluation protocol, the primary aggregate and its interval, and the multiplicity adjustment, and say which claims the design can support.

3.9 DESIGN. (Four methods, twelve data sets.) Pre-register the analysis for comparing a new method with three baselines on 12 data sets with 5 runs each: the primary aggregate, the comparisons and their adjustment, and the test across data sets. State in advance what result would count as the new method being better.

3.10 CRITIQUE. (A bold column.) An invented paper bolds the best of five agents on each of 26 games, with baseline numbers taken from their papers, and claims to be "best on 17 of 26 games", a new state of the art in median score (0.312 against 0.298), and "significantly better on 9 games (Welch's t-test, $p < 0.05$, 3 seeds)". Audit the three claims.

3.11 CRITIQUE. (The best checkpoint.) An invented methods section reads: "We evaluate every 10k steps on 10 episodes and report, for each seed, the best evaluation; results are averaged over 5 seeds. Baseline results are those reported by the original authors." Use Proposition 3.8 to say

what the reported numbers estimate, and rewrite the paragraph so that the comparison is fair.

3.12 RESEARCH. (Aggregates robust to the choice of tasks.) Zhang and Hardt (2024) show for multi-task benchmarks a trade-off: the more the tasks disagree about the ranking of models, the more an aggregate ranking moves under irrelevant changes. Open: an aggregate whose interval accounts for both the runs and the selection of tasks, with a stability property under adding or removing tasks that it provably satisfies.

3.13 RESEARCH. (Three runs.) With three runs, each task's variance rests on two degrees of freedom. Open: intervals for the interquartile mean with nearly nominal coverage at $n = 3$, pooling variance information across tasks, for instance through a hierarchical model of the σ_j^2, with guarantees that survive skewed, heterogeneous run distributions like those of Figure 3.1.

Notes and References

Comparisons over many data sets were studied in machine learning before deep reinforcement learning made them urgent: Demšar (2006) recommended rank-based tests across data sets, and Benavoli et al. (2016) questioned the post hoc tests based on mean ranks. Performance profiles come from the benchmarking of optimization software (Dolan and Moré 2002), where they plot, for each solver, the fraction of problems solved within a factor of the best solver's time.

In deep reinforcement learning, Henderson et al. (2018) documented how seeds, hyperparameters, code bases and environments move reported results, and Colas et al. (2018, 2019) turned the lesson into tutorials on power analysis and the choice of test. Jordan et al. (2020) aggregated across environments through performance percentiles, with intervals of guaranteed coverage. Agarwal et al. (2021) proposed the stratified bootstrap, the interquartile mean and the score distribution as default tools and released them in the library rliable, whose interquartile mean is `scipy`'s 25% trimmed mean of all runs pooled over tasks and whose intervals are percentile intervals by default.

The statistics are older: Efron and Tibshirani (1993) is the standard introduction to the bootstrap, the asymptotics of trimmed means are due to Stigler (1973), and the statistic behind the probability of improvement to Mann and Whitney (1947). Šidák (1967) and Holm (1979) sharpened Bonferroni's correction, and Benjamini and Hochberg (1995) and Benjamini and Yekutieli (2001) replaced the family-wise error rate by the false discovery rate. The garden of forking paths is the phrase of Gelman and Loken (2013, 2014), and Dehghani et al. (2021) and Zhang and Hardt (2024) studied how the choice and number of tasks shape the rankings of machine-learning benchmarks.

TRANSLATING THE LITERATURE. Agarwal et al. (2021) write M tasks and N runs for our J and n, and $x_{m,n}$ for our Y_{ji}; their "score distribution" $\hat{F}_X(\tau)$ is our $\hat{F}(t)$, and their "true median" is our median of the μ_j. Demšar (2006) writes N data sets and k classifiers for our J tasks and number of methods, and his critical difference $q_\alpha\sqrt{k(k+1)/(6N)}$ is the one used in Lab 3.1. Holm (1979) writes n for our number of hypotheses m, and Benjamini and Hochberg (1995) write q^* for the level α at which we control the false discovery rate.

References

Agarwal, R., M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare (2021). "Deep Reinforcement Learning at the Edge of the Statistical Precipice". In: *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, pp. 29304–29320.

Bellemare, M. G., Y. Naddaf, J. Veness, and M. Bowling (2013). "The Arcade Learning Environment: An Evaluation Platform for General Agents". In: *Journal of Artificial Intelligence Research* 47, pp. 253–279.

Benavoli, A., G. Corani, and F. Mangili (2016). "Should We Really Use Post-Hoc Tests Based on Mean-Ranks?" In: *Journal of Machine Learning Research* 17.5, pp. 1–10.

Benjamini, Y. and Y. Hochberg (1995). "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing". In: *Journal of the Royal Statistical Society: Series B (Methodological)* 57.1, pp. 289–300.

Benjamini, Y. and D. Yekutieli (2001). "The Control of the False Discovery Rate in Multiple Testing under Dependency". In: *The Annals of Statistics* 29.4, pp. 1165–1188.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2018). *How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments*. arXiv:1806.08295.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2019). "A Hitchhiker's Guide to Statistical Comparisons of Reinforcement Learning Algorithms". In: *ICLR 2019 Workshop on Reproducibility in Machine Learning*. arXiv:1904.06979.

Dehghani, M., Y. Tay, A. A. Gritsenko, Z. Zhao, N. Houlsby, F. Diaz, D. Metzler, and O. Vinyals (2021). *The Benchmark Lottery*. arXiv:2107.07002.

Demšar, J. (2006). "Statistical Comparisons of Classifiers over Multiple Data Sets". In: *Journal of Machine Learning Research* 7, pp. 1–30.

Dolan, E. D. and J. J. Moré (2002). "Benchmarking Optimization Software with Performance Profiles". In: *Mathematical Programming* 91.2, pp. 201–213.

Efron, B. and R. J. Tibshirani (1993). *An Introduction to the Bootstrap*. Monographs on Statistics and Applied Probability 57. New York: Chapman & Hall.

Friedman, M. (1937). "The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance". In: *Journal of the American Statistical Association* 32.200, pp. 675–701.

Gelman, A. and E. Loken (2013). "The Garden of Forking Paths: Why Multiple Comparisons Can Be a Problem, Even When There Is No "Fishing Expedition" or "p-Hacking" and the Research Hypothesis Was Posited Ahead of Time". Unpublished manuscript, 14 November 2013.

Gelman, A. and E. Loken (2014). "The Statistical Crisis in Science". In: *American Scientist* 102.6, pp. 460–465.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger (2018). "Deep Reinforcement Learning That Matters". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1, pp. 3207–3214.

Hessel, M. et al. (2018). "Rainbow: Combining Improvements in Deep Reinforcement Learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1, pp. 3215–3222.

Holm, S. (1979). "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2, pp. 65–70.

Jordan, S. M., Y. Chandak, D. Cohen, M. Zhang, and P. S. Thomas (2020). "Evaluating the Performance of Reinforcement Learning Algorithms". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 4962–4973.

Kaiser, Ł. et al. (2020). "Model Based Reinforcement Learning for Atari". In: *International Conference on Learning Representations (ICLR)*.

Kielak, K. (2020). *Importance of Using Appropriate Baselines for Evaluation of Data-Efficiency in Deep Reinforcement Learning for Atari*. arXiv:2003.10181.

Laskin, M., A. Srinivas, and P. Abbeel (2020). "CURL: Contrastive Unsupervised Representations for Reinforcement Learning". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 5639–5650.

Lee, K., M. Laskin, A. Srinivas, and P. Abbeel (2021). "SUNRISE: A Simple Unified Framework for Ensemble Learning in Deep Reinforcement Learning". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 6131–6141.

Mann, H. B. and D. R. Whitney (1947). "On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other". In: *The Annals of Mathematical Statistics* 18.1, pp. 50–60.

Mnih, V. et al. (2015). "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540, pp. 529–533.

Schwarzer, M., A. Anand, R. Goel, R. D. Hjelm, A. Courville, and P. Bachman (2021). "Data-Efficient Reinforcement Learning with Self-Predictive Representations". In: *International Conference on Learning Representations (ICLR)*.

Šidák, Z. (1967). "Rectangular Confidence Regions for the Means of Multivariate Normal Distributions". In: *Journal of the American Statistical Association* 62.318, pp. 626–633.

Simmons, J. P., L. D. Nelson, and U. Simonsohn (2011). "False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant". In: *Psychological Science* 22.11, pp. 1359–1366.

Stigler, S. M. (1973). "The Asymptotic Distribution of the Trimmed Mean". In: *The Annals of Statistics* 1.3, pp. 472–477.

van der Vaart, A. W. (1998). *Asymptotic Statistics*. Cambridge Series in Statistical and Probabilistic Mathematics 3. Cambridge: Cambridge University Press.

van Hasselt, H., M. Hessel, and J. Aslanides (2019). "When to Use Parametric Models in Reinforcement Learning?" In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pp. 14322–14333.

Wilcoxon, F. (1945). "Individual Comparisons by Ranking Methods". In: *Biometrics Bulletin* 1.6, pp. 80–83.

Yarats, D., I. Kostrikov, and R. Fergus (2021). "Image Augmentation Is All You Need: Regularizing Deep Reinforcement Learning from Pixels". In: *International Conference on Learning Representations (ICLR)*.

Zhang, G. and M. Hardt (2024). "Inherent Trade-Offs between Diversity and Stability in Multi-Task Benchmarks". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research, pp. 58984–59002.

Part II

Fair Comparison

A comparison of two methods is a comparison of two procedures, each with its tuning, its pipeline and its budget. An experiment is fair when its arms differ only in what the claim is about.

4 Tuning Budgets

How an unequal search decides a comparison

THE CLAIM *"Adaptive gradient methods generalize worse than SGD."*

The received reading of Wilson et al. (2017)

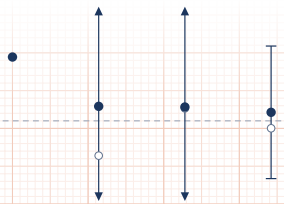
THE VERDICT Reversed: tuned over all their hyperparameters with equal budgets, adaptive methods match or beat SGD with momentum.

IN THIS CHAPTER. In 2017 Wilson and colleagues tuned the step size and its decay for five optimizers on four workloads and found that AdaGrad, RMSProp and Adam generalized worse than SGD with and without momentum; the field read this as a property of adaptive methods. Two years later Choi and colleagues reproduced the gap under the same protocol and removed it by also tuning ϵ and the momentum coefficient: with every hyperparameter searched, adaptive methods were never worse. The method of this chapter treats a training method as an update rule plus a search space plus a budget, compares methods by their budget curves, the expected result of a search of k trials, proves when such curves cross and when a larger search space contains a smaller one, and estimates the curves without bias. In the laboratory, on digits with distractor features, Adam loses to momentum on nine splits of ten under the 2017 protocol and ties it under the 2019 protocol with the same sixteen trials.

4.1 The claim: the marginal value of adaptive methods

By 2017 Adam had become a default choice in much of deep learning. Kingma and Ba (2015) had proposed it as a method that needs little tuning: it keeps exponential moving averages of the gradient and of its square, divides the first by the square root of the second, coordinate by coordinate, and so gives every parameter its own step size. They recommended defaults, a step size $\alpha = 10^{-3}$, decay rates $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and a constant $\epsilon = 10^{-8}$ added to the denominator to keep it away from zero, and frameworks adopted such values as their defaults, with varying choices of ϵ. AdaGrad and RMSProp, the older *adaptive gradient methods*, work the same way with other averages.

Ashia Wilson, Rebecca Roelofs, Mitchell Stern, Nathan Srebro and Benjamin Recht asked what the adaptivity buys (Wilson et al. 2017). Their paper has two parts. The first is a constructed problem: least-squares classification in which the first feature equals the label, two features always equal one, and



every training example has features of its own that no other example shares. Gradient descent and SGD converge to the minimum-norm interpolating solution, which, under a mild condition on the class counts, classifies new points correctly; AdaGrad, RMSProp and Adam, under a condition on the signs of the gradients, converge to a solution proportional to the sign vector of $X^\top y$, which weights the label feature and the two constant features equally and labels every new point positive. When positive labels have probability p slightly above one half, its test error $1 - p$ is close to one half. The example is a theorem, and it stands: adaptive methods can find different solutions, and worse ones.

The second part is an experiment on four workloads: a VGG network on CIFAR-10, a character-level language model (a two-layer LSTM) on Tolstoy's *War and Peace*, and two LSTM parsers on the Penn Treebank, one discriminative and one generative. Five optimizers were compared: SGD, the *heavy ball* method, that is SGD with momentum 0.9, and the three adaptive methods. For each optimizer the authors tuned the initial step size and the schedule by which it decays, and left the other hyperparameters at their defaults.

> FINDING 4.1 (Wilson et al. 2017). With the initial step size and its decay tuned for each optimizer and all other hyperparameters at their defaults, SGD gave the best result on each of the four workloads, ahead of every adaptive method, although the adaptive methods sometimes fit the training set better. On CIFAR-10 SGD had a test error of 7.65% and the best adaptive method, RMSProp, 9.60% (means over five runs; standard deviations 0.14 and 0.19). On the discriminative parser the margin was small, a development F1 of 91.24 for SGD against 91.11 for Adam.

The authors drew two conclusions. Adaptive methods, contrary to their reputation, had needed as much tuning as SGD; and the paper's abstract suggests that practitioners should "reconsider the use of adaptive methods to train neural networks". The paper appeared at NeurIPS 2017, and its observation quickly became a premise. Keskar and Socher (2017) proposed starting with Adam and switching to SGD, citing Wilson et al. for the observation that adaptive methods generalize worse; Loshchilov and Hutter (2019) motivated decoupled weight decay (AdamW) by the same gap; AdaBound (Luo et al. 2019) bounded Adam's step sizes so that it would turn into SGD late in training; and a NeurIPS 2020 paper set out to explain theoretically why SGD generalizes better than Adam (Zhou et al. 2020). In this received reading the protocol had dropped out: adaptive methods generalize worse.

What the original work got right is substantial. The constructed example is correct. The measurements were careful for their time, with five runs per selected setting, means and standard deviations, and learning curves, and they reproduce: as we will see, the gap appears again when the protocol is repeated. The observation that adaptive methods need tuning was right too. And the protocol was a common one. Practitioners tuned the learning rate and its schedule; ϵ, there to keep a denominator away from zero, was seldom tuned, and β_1, β_2 were left at the values that every framework

shipped.[1] The question this chapter asks is what such a protocol can and cannot decide.

[1] Wilson et al. list momentum, ϵ and the initialization of the accumulators among the hyperparameters they could have tuned, and say that a full grid over them was infeasible.

4.2 The experiment: five optimizers, one knob

EXPERIMENTAL DESIGN · WILSON ET AL. 2017

QUESTION	Do adaptive methods find solutions that generalize worse than those of SGD?
UNITS	Training runs on four workloads: VGG on CIFAR-10, an LSTM language model on *War and Peace*, two LSTM parsers on the Penn Treebank.
TREATMENT	The optimizer: SGD, heavy ball (momentum 0.9), AdaGrad, RMSProp, Adam.
TUNED	The initial step size, on a logarithmic grid of five values extended when the best lay at an edge; the decay scheme.
HELD FIXED	Momentum, β_1, β_2, ϵ, accumulator initialization (framework defaults); model, batch size, epochs.
SELECTION	Best development score within the epoch budget; on CIFAR-10, lowest training loss.
RESPONSE	Test error, test loss, development F1, development perplexity.
REPLICATES	Five runs of each selected setting from random initializations.
ANALYSIS	Means and standard deviations over the five runs; learning curves.

Read as a measurement, the record is sound. Every optimizer received the same kind of search, about five step sizes for each decay scheme, the selected settings were rerun from fresh initializations, and the spread over five runs was reported. Under this protocol, on these workloads, the adaptive methods lost.

Read as evidence for the claim, the record has a gap in the row HELD FIXED. The hyperparameters left at their defaults are not neutral background shared by the arms: they are the ones that distinguish the arms. Momentum is the parameter that separates heavy ball from SGD; ϵ decides how adaptive Adam is, and β_1, β_2 set the memory of its two averages. With $\epsilon = 10^{-8}$, Adam divides by $\sqrt{\hat{v}} + \epsilon \approx \sqrt{\hat{v}}$, and every coordinate moves at about the same speed whatever the size of its gradient, the mechanism of the constructed example. With ϵ large compared with $\sqrt{\hat{v}}$, the denominator is nearly constant and Adam moves like momentum SGD with a rescaled learning rate. The experiment compared heavy ball at its tuned learning rate with Adam pinned to one end of its own range. The design could show that Adam with its default ϵ and β's generalizes worse than momentum on these workloads; it could not show that the adaptive update rule does.

A second gap is in what "the same search" meant. Each optimizer got about five trials per decay scheme, but a trial count measures effort, not coverage: five learning rates cover the hyperparameter that matters most for heavy ball, but only a one-dimensional slice of Adam's four-dimensional space. Equal numbers of trials over unequal spaces are an unequal search.[2]

[2] Bergstra and Bengio (2012) made a related point against grids: when only a few hyperparameters matter, a grid spends most of its trials varying the others.

The post-mortem takes up both.

4.3 The post-mortem: the knob that was not turned

Dami Choi, Christopher Shallue, Zachary Nado, Jaehoon Lee, Chris Maddison and George Dahl studied why published optimizer comparisons disagreed (Choi et al. 2019).[3] Their central tool is a definition. One optimizer *includes* another if, for every setting of the second's hyperparameters, some sequence of settings of the first produces trajectories that converge to the second's on the same training history. SGD is momentum with momentum zero, momentum is included in RMSProp and in Adam, and Nesterov's method is included in NAdam. For Adam the construction is the one sketched above: take ϵ large, scale the learning rate with it, and set β_1 to the momentum coefficient. Proposition 4.6 proves the version we need.

Inclusion makes a prediction. If a more general optimizer is tuned over all of its hyperparameters, with a search space that reaches the region where it imitates its special case, then with enough trials it should do at least as well as the special case. A comparison in which Adam loses to momentum is therefore a statement about the search, not about Adam.

[3] The paper has remained an arXiv preprint (first version October 2019, third version June 2020); we cite it as such.

EXPERIMENTAL DESIGN · CHOI ET AL. 2019

UNITS	Seven workloads, from a small CNN on Fashion-MNIST to ResNet-50 on ImageNet and a Transformer on LM1B, with Wilson et al.'s VGG on CIFAR-10 among them.
TREATMENT	SGD, momentum, Nesterov, RMSProp, Adam, NAdam.
TUNED	Every optimizer hyperparameter, ϵ, $1-\beta_1$ and $1-\beta_2$ included, on log scales; a linear learning-rate decay; weight decay or label smoothing on some workloads.
BUDGET	Quasi-random search, 10 to 100 feasible trials per optimizer depending on the workload.
SELECTION	Lowest validation error; the test error of that trial.
ANALYSIS	Bootstrap over trials for the result of tuning; five reruns in the replication of Wilson et al.

The numbers in this section are those reported by the papers. Choi et al. first repeated Wilson et al.'s VGG experiment with its original protocol, a grid over the learning rate only, and reproduced the gap: RMSProp and Adam had worse test error. Tuning momentum and ϵ as well made the four optimizers nearly identical; tuning only the learning-rate schedule also brought them within error bars of each other. Across all seven workloads, with every hyperparameter tuned, the adaptive methods never underperformed momentum or SGD within error bars, and on some they were better. The best values of ϵ spanned ten orders of magnitude across four workloads; the best ResNet-50 result, a validation accuracy of 77.1%, came from NAdam with $\epsilon = 9475$, a value so large that NAdam behaves almost like Nesterov momentum, its special case.[4] They concluded that the search space may be the single most important factor behind conflicting optimizer rankings, and that studies that fix $\epsilon = 10^{-8}$ owe their readers a

[4] Reported by Choi et al.; we could not check it independently. They compare it with a published momentum result of 76.5%, a number from another paper rather than a controlled arm.

justification.

Choi et al. were also candid about their own search. They chose each final search space after an initial round of search over a broader one, checked that the optimum lay away from the boundaries, published both spaces, and noted that the effort that went into choosing them cannot easily be quantified. A search space is itself the result of tuning, and the trial count does not show it.

BENCHMARKS THAT CHARGE FOR TUNING. If tuning must be counted, the result of a comparison depends on how much of it is allowed. Sivaprasad et al. (2020) made the budget an axis: for each optimizer they drew hyperparameters from a prior and reported the expected performance of the best of k random-search trials, as a function of k, on image, text and synthetic problems, most of them from the DeepOBS benchmark suite. Adam with only its learning rate tuned was the most reliable choice at every budget, and especially at small ones; tuning more of Adam's hyperparameters paid off only at larger budgets.[5] Schmidt et al. (2021) ran more than fifty thousand runs: fifteen optimizers, eight problems, four schedules, and budgets of one run at the default setting and of 25, 50 and 75 random-search trials. No optimizer dominated, Adam remained a strong contender, rankings moved with the budget and the search space, and trying several optimizers at their defaults did about as well as tuning one.

[5] Their priors were calibrated by fitting distributions to the good configurations of a broad random search on the same tasks, a further instance of a search space shaped by earlier tuning.

THE SAME PATTERN ELSEWHERE. Layer-wise adaptive optimizers were proposed for very large batches. Nado et al. (2021) (a preprint) tuned Nesterov momentum and Adam as carefully as the published LARS and LAMB pipelines and matched them: on ResNet-50 at batch size 32,768, Nesterov momentum reached a median validation accuracy of 75.97% over 50 runs, the same as LARS, and in BERT pretraining at the same batch size Adam reached a SQuAD F1 of 91.58 against LAMB's 91.48. In language-model pretraining, Wen et al. (2026) tuned AdamW and a set of recent optimizers by coordinate-wise sweeps over all their hyperparameters, at model sizes from 0.1 to 1.2 billion parameters: speedups over AdamW that other papers had claimed, 1.4 to 2×, shrank to at most about 1.4×, and to about 1.1× at 1.2 billion parameters. They also found that rankings read at intermediate checkpoints can flip by the end of training, because the learning-rate decay changes the curves late (**??** meets the same effect in scaling laws). The AlgoPerf benchmark (Dahl et al. 2023) turned these lessons into rules: an algorithm is scored by its time to reach a target on fixed hardware, and the tuning protocol is part of the submission, either a search space from which a fixed number of trials is drawn or no tuning at all. In its first competition (Kasimbeg et al. 2025), a Distributed Shampoo submission was about 28% faster than the NadamW baseline, by the geometric mean over eight workloads, under five trials per study.[6]

[6] The benchmark paper specified twenty trials per study; the competition used five, repeated in five studies, and scored the median.

THE MECHANISM. Three features of the protocol combined, and the method of this chapter treats each. First, the knob that was held fixed was the one that decides how adaptive Adam is, so "Adam at its defaults" was compared with "momentum tuned". Second, equal numbers of trials

over spaces of different dimension and shape are unequal searches, and which method wins depends on the budget, in a way that can be computed (Proposition 4.3). Third, the result was reported as a property of the update rule, without the budget and the search space that produced it.

The post-mortem literature is not unanimous in its details. Nesterov momentum was a strong alternative on some problems of Schmidt et al. (2021), and SGD variants did best on two problems of Sivaprasad et al. (2020) at large budgets; theoretical work on why the two families find different solutions continues (Zhou et al. 2020); and Wilson et al.'s example remains a correct statement about what an untuned adaptive method can converge to. The verdict concerns the general claim.

VERDICT. Tuned over all of their hyperparameters with equal care, adaptive methods were never worse than momentum or SGD on the workloads of Choi et al., including the one on which the gap was first measured, and a benchmark that charges for tuning found Adam the cheaper method to tune. The gap was produced by the protocol: ϵ and the moment parameters were held at defaults, which fixes Adam in the corner of its search space where it differs most from momentum. Read as a statement about update rules, the received claim does not survive; where tuned methods still differ, the difference more often favors the adaptive ones.



4.4 The method: a method is a protocol

The first lesson is a change of unit. A comparison of training methods compares procedures, and the procedure includes its tuning.

Definition 4.2 (Trial, protocol, training method). A *trial* is one hyperparameter configuration h from a *search space* $\mathcal{H}$, trained with its own seeds and scored. A *tuning protocol* specifies $\mathcal{H}$; how the configurations of the trials are chosen, for instance independently from a sampling distribution ν on $\mathcal{H}$ (*random search*); a *budget*, a number k of trials or the compute they may use; a selection rule, usually the best validation score; and a reporting rule, usually the test score of the selected configuration retrained with fresh seeds. A *training method* is an update rule together with a tuning protocol.

Under random search the trials of a method are independent and identically distributed: trial j yields a selection score V_j, such as a negative validation error, and a reported score Y_j, such as a test accuracy. The result of a search of budget k is $M_k = Y_J$, where J is the trial among the first k with the largest V (ties broken at random). The *budget curve* of the method is $k \mapsto \mathbb{E}M_k$. When the selection score and the reported score coincide, M_k is the best of k draws, the order statistic of Chapter 1; for normal scores with mean μ and standard deviation σ, $\mathbb{E}M_k = \mu + \sigma e_k$ (Proposition A.9). We take scores to be maximized; for an error, apply the results to its negative.[7]

[7] The tuning playbook of Godbole et al. (2023) calls the hyperparameters that must be retuned for each arm of a comparison its nuisance hyperparameters.

The budget curve is the object a comparison should report. Its value at $k = 1$ is the average quality of the search space; its limit is the best the space contains; and its shape says how fast a search climbs from one to the other. Two methods can trade places along the way.

Proposition 4.3 (Budget curves). *Let the trial scores of two arms have distribution functions F_1 and F_2 with finite means, and let $M_k^{(a)}$ be the best of k independent trials of arm a. Then:*

(i) for all budgets $k_1, k_2 \geq 1$,

$$\mathbb{E}M_{k_1}^{(1)} - \mathbb{E}M_{k_2}^{(2)} = \int_{-\infty}^{\infty} \bigl(F_2(x)^{k_2} - F_1(x)^{k_1}\bigr)\,\mathrm{d}x; \tag{4.1}$$

(ii) for each arm, $k \mapsto \mathbb{E}M_k^{(a)}$ is nondecreasing and converges to the upper end $s_a = \sup\{x : F_a(x) < 1\}$ of the distribution, which may be $+\infty$;

(iii) if $s_1 > s_2$, then $\mathbb{E}M_k^{(1)} > \mathbb{E}M_k^{(2)}$ for every sufficiently large k. If in addition $\mathbb{E}M_1^{(1)} < \mathbb{E}M_1^{(2)}$, then the budget curves cross.

Proof. The plan is to write each expected maximum as an integral of its distribution function and then to let k grow.

Part (i). For a random variable Z with finite mean and distribution function G, Fubini's theorem applied to its positive and negative parts gives $\mathbb{E}Z = \int_0^\infty (1 - G(x))\,\mathrm{d}x - \int_{-\infty}^0 G(x)\,\mathrm{d}x$. The best of k independent draws from F is at most x exactly when every draw is, so its distribution function is F^k, and its mean is finite because $|M_k|$ is at most the sum of the k absolute values. Write the formula for $M_{k_1}^{(1)}$ and for $M_{k_2}^{(2)}$ and subtract: all four integrals are finite, and on both half-lines the integrands combine to $F_2^{k_2} - F_1^{k_1}$.

Part (ii). Realize the trials of an arm as one infinite independent sequence $Y_1, Y_2, \ldots$; then $M_k = \max_{j \leq k} Y_j$ is nondecreasing in k, and so is its mean. Let $M_\infty = \sup_j Y_j$. Every Y_j is at most s_a almost surely, so $M_\infty \leq s_a$; and for $x < s_a$, $\mathbb{P}(M_\infty \leq x) \leq \mathbb{P}(M_k \leq x) = F_a(x)^k$ for every k, which tends to 0 because $F_a(x) < 1$. Hence $M_\infty = s_a$ almost surely. The variables $M_k - Y_1$ are nonnegative and increase to $M_\infty - Y_1$, so by monotone convergence $\mathbb{E}M_k - \mathbb{E}Y_1$ increases to $s_a - \mathbb{E}Y_1$, where $\mathbb{E}Y_1$ is finite.

Part (iii). Since $s_2 < s_1$, s_2 is finite, and $M_k^{(2)} \leq s_2$ almost surely gives $\mathbb{E}M_k^{(2)} \leq s_2$ for every k. By part (ii) there is a k_0 with $\mathbb{E}M_{k_0}^{(1)} > s_2$, and by monotonicity $\mathbb{E}M_k^{(1)} > s_2 \geq \mathbb{E}M_k^{(2)}$ for every $k \geq k_0$. If arm 2 is ahead at $k = 1$, then the difference changes sign between $k = 1$ and k_0. ∎

The integrand in (4.1) explains which part of a method the budget sees. For large k, $F(x)^k$ is close to 0 unless $F(x)$ is within about $1/k$ of 1, so at budget k a method is judged by roughly its best $1/k$ fraction of configurations: its average at $k = 1$, its upper tail at large k.

Example 4.4 (Two curves that cross). Let every configuration of arm 2 reach an accuracy of $b = 0.93$, a robust method with a small search space. In arm

1, a fraction $p = 0.1$ of configurations reach $a = 0.95$ and the rest diverge to $c = 0.50$, a broad space with a better corner. The best of k trials of arm 1 is a unless all k trials diverge, so $\mathbb{E}M_k^{(1)} = a - (a - c)(1 - p)^k = 0.95 - 0.45 \cdot 0.9^k$, which is 0.545 at $k = 1$, 0.793 at $k = 10$ and first exceeds 0.93 at $k = 30$. An experiment with 29 trials per arm declares arm 2 the better method; one with 30 declares arm 1. Neither is wrong about its budget, and both are wrong if they omit it.

Corollary 4.5 (Unequal budgets). *Let both arms have the same distribution function F with finite mean, and let arm 1 be tuned with k_1 trials and arm 2 with $k_2 < k_1$. Then*

$$\mathbb{E}M_{k_1}^{(1)} - \mathbb{E}M_{k_2}^{(2)} = \int_{-\infty}^{\infty} F(x)^{k_2}\bigl(1 - F(x)^{k_1 - k_2}\bigr)\,\mathrm{d}x \;\geq\; 0,$$

with equality only if the scores are almost surely constant. For normal scores with standard deviation σ the difference is $\sigma(e_{k_1} - e_{k_2})$.

Proof. Take $F_1 = F_2 = F$ in (4.1) and factor F^{k_2} out of the integrand, which is nonnegative because $0 \leq F \leq 1$. The integral vanishes only if $F(x) \in \{0, 1\}$ for almost every x, which for a right-continuous distribution function means a point mass. For normal scores, $\mathbb{E}M_k = \mu + \sigma e_k$ for each arm. ∎

The corollary is the arithmetic of a common reporting practice. A new method tuned with $k_1 = 50$ trials and a baseline tuned with $k_2 = 5$ differ by $\sigma(e_{50} - e_5) = 1.09\sigma$ when they are the same method, and a baseline given a single trial loses by $\sigma e_{50} = 2.25\sigma$, where σ is the spread of trial scores over the search space. A difference of this size says nothing about the methods.

Inclusion

Wilson et al.'s protocol held Adam's ϵ at 10^{-8}. The next result shows what that excluded: at large ϵ, with a matched learning rate, Adam is momentum SGD. We state it for the update rules as PyTorch implements them, with the momentum coefficient written β_1, as in Adam, and operations on vectors taken coordinate by coordinate.

Proposition 4.6 (Adam contains momentum). *Fix a number of steps $T \geq 1$, a starting point $\theta_0 \in \mathbb{R}^d$, coefficients $\beta_1, \beta_2 \in [0, 1)$ and a learning rate $\eta > 0$. Let $g_1, \ldots, g_T \colon \mathbb{R}^d \to \mathbb{R}^d$ be the minibatch gradients of the T steps, an L2 term $\lambda\theta$ included if there is one. Momentum SGD sets $b_0 = 0$ and, for $t = 1, \ldots, T$, $b_t = \beta_1 b_{t-1} + g_t(\theta_{t-1})$ and $\theta_t = \theta_{t-1} - \eta b_t$. For $\epsilon > 0$, Adam with coefficients $\beta_1, \beta_2, \epsilon$ starts at $\theta_0^\epsilon = \theta_0$ with $m_0 = v_0 = 0$ and sets*

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\theta_t^\epsilon = \theta_{t-1}^\epsilon - \alpha_t \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t)} + \epsilon},$$

with $g_t = g_t(\theta^\epsilon_{t-1})$ and step sizes

$$\alpha_t = \frac{\epsilon\,\eta\,(1-\beta_1^t)}{1-\beta_1}. \tag{4.2}$$

If each g_t is continuous at the momentum iterate θ_{t-1}, then $\theta^\epsilon_t \to \theta_t$ as $\epsilon \to \infty$, for $t = 1, \dots, T$.

PROOF. The plan is an induction on t with a stronger hypothesis: as $\epsilon \to \infty$, $\theta^\epsilon_t \to \theta_t$, $m_t \to (1-\beta_1)b_t$, and v_t converges to a finite limit $\bar v_t$. At $t = 0$ all three hold, with $\bar v_0 = 0$. Suppose they hold at $t-1$. By the continuity hypothesis, $g_t(\theta^\epsilon_{t-1}) \to g_t(\theta_{t-1})$, so

$$m_t \to \beta_1(1-\beta_1)b_{t-1} + (1-\beta_1)g_t(\theta_{t-1}) = (1-\beta_1)b_t,$$

and $v_t \to \bar v_t = \beta_2 \bar v_{t-1} + (1-\beta_2)g_t(\theta_{t-1})^2$. With the step size (4.2) the bias correction $1-\beta_1^t$ cancels, and Adam's step becomes

$$\frac{\eta}{1-\beta_1}\, m_t \cdot \frac{1}{1 + \sqrt{v_t/(1-\beta_2^t)}\,/\epsilon}.$$

The first factor tends to ηb_t. In the second, $\sqrt{v_t/(1-\beta_2^t)}$ converges to the finite limit $\sqrt{\bar v_t/(1-\beta_2^t)}$, so its ratio to ϵ tends to 0 and the factor tends to 1. Hence $\theta^\epsilon_t \to \theta_{t-1} - \eta b_t = \theta_t$. ∎

For a ReLU network the continuity hypothesis can fail only if some unit sits exactly at its kink, for some example, at a momentum iterate, which one does not expect under a continuous random initialization. Two remarks qualify the result. With a constant step size $\alpha = \epsilon\eta/(1-\beta_1)$ instead of (4.2), the same argument shows that Adam tends to momentum SGD with the learning rate $\eta/(1-\beta_1^t)$ at step t, which starts ten times too large (for $\beta_1 = 0.9$) and settles within a few dozen steps. Adam's bias correction acts as a schedule. And the inclusion is a limit: a search space whose ϵ stops at 10^{-8} never reaches it, and one that reaches it may still spend most of its trials elsewhere. Inclusion says what a search over Adam's space can find; Proposition 4.3 says how many trials finding it costs.

Figure 4.1 shows the limit in a real network. With the matched step sizes, the distance between the final weights falls from 0.20 at $\epsilon = 10^{-2}$ to 3.9×10^{-3} at $\epsilon = 10^2$ and 6.7×10^{-11} at $\epsilon = 10^8$, by exactly a factor of 100 over the last factor of 100 in ϵ, the rate of Exercise 4.4; from $\epsilon = 10^2$ on, Adam's test error equals momentum's. With a constant step size the distance stops at 0.12, and the test error settles at 16.12%: a different run of about the same quality.

THE UNIT OF BUDGET. Counting trials is fair only when trials cost the same. If a trial of method 1 costs c_1 and one of method 2 costs c_2, a comparison at total compute C sets $k_a = C/c_a$ and compares $\mathbb{E}M^{(1)}_{C/c_1}$ with $\mathbb{E}M^{(2)}_{C/c_2}$, which by (4.1) can favor the cheaper method even when the costlier one is better

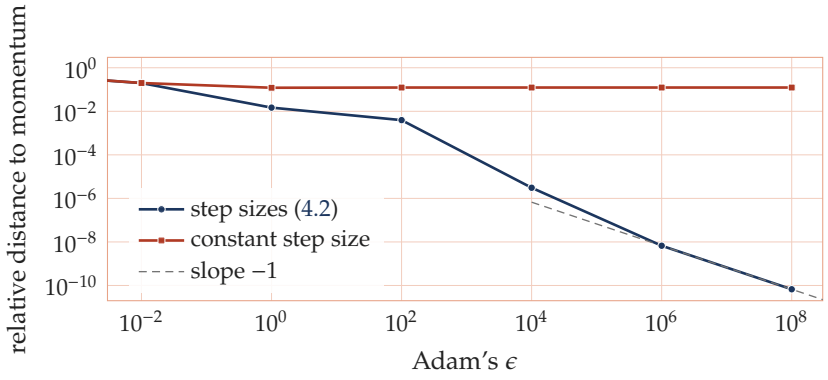


Figure 4.1. A numerical check of Proposition 4.6 (`04-budgets-inclusion.py`). Momentum SGD ($\eta = 0.03$, $\beta_1 = 0.9$) and Adam ($\beta_2 = 0.999$) are trained from the same initial weights on the same minibatches for 900 steps, on split 0 of the task of Lab 4.1, in double precision; the vertical axis is $\|\theta_T^\epsilon - \theta_T\|/\|\theta_T\|$ for a single run of each, so there are no intervals. With the step sizes (4.2) the distance falls to 7×10^{-11}, and from $\epsilon = 10^2$ on the test error equals momentum's 16.46%; with a constant step size the bias correction leaves a distance of 0.12.

trial for trial. The time to reach a target on fixed hardware, the measure of AlgoPerf (Dahl et al. 2023), folds the cost per step, the number of steps and the tuning protocol into one number; its external-tuning rules fix the number of trials and let the submitter choose the search space, and its self-tuning rules allow no external search and give a longer time limit. Which unit is right depends on the claim: "better per trial", "better per GPU-hour" and "better for a practitioner with an afternoon" are three different claims, and **??** shows how the choice of what to hold fixed decides another famous comparison. The design of the search space belongs in the budget too, although it is rarely counted.

4.5 The method: estimating a budget curve

A budget curve is an expectation over searches, and running many searches is expensive. One pool of trials can stand in for all of them. Suppose we have run K independent trials of a method and want $\mathbb{E}M_k$ for every $k \le K$. Order the pool by its selection scores, $V_{(1)} < \cdots < V_{(K)}$, and let $Y_{[i]}$ be the reported score of the trial with the ith smallest selection score, its *concomitant*.

Theorem 4.7 (An unbiased budget curve). *Let $(V_1, Y_1), \ldots, (V_K, Y_K)$ be independent and identically distributed, with $\mathbb{E}|Y_1| < \infty$ and the V_j almost surely distinct. For $1 \le k \le K$, the statistic*

$$\hat{M}_k = \sum_{i=k}^{K} \frac{\binom{i-1}{k-1}}{\binom{K}{k}} Y_{[i]} \tag{4.3}$$

is the average, over all $\binom{K}{k}$ subsets S of k trials of the pool, of the reported

score of the trial in S with the largest selection score, and $\mathbb{E}\hat{M}_k = \mathbb{E}M_k$.

PROOF. The plan is to count, for each trial, the subsets that select it, and then to use the fact that every subset is a search of budget k.

The average over subsets. A subset S selects its member of largest rank in the ordering by V. The trial of rank i is selected by exactly the subsets that contain it and $k-1$ trials of smaller rank, and there are $\binom{i-1}{k-1}$ of those, none when $i < k$. Summing the selected scores over all subsets therefore gives $\sum_i \binom{i-1}{k-1} Y_{[i]}$, and dividing by the number of subsets gives (4.3).

Unbiasedness. Fix a subset S. Its k pairs are independent with the common law of (V_1, Y_1), and the trial it selects is the same function of them as J is of the first k trials of a search; so its selected score has the law of M_k, and in particular a finite mean, because its absolute value is at most $\sum_{j\in S}|Y_j|$. Each of the $\binom{K}{k}$ terms of the average therefore has expectation $\mathbb{E}M_k$, and so has the average. ∎

Counting the subsets of $\{1, \dots, K\}$ of size k by their largest element shows that the weights sum to one, $\sum_{i=k}^{K} \binom{i-1}{k-1} = \binom{K}{k}$. Ties in the selection score are common, since a validation error on 300 images takes only 301 values; a secondary score such as the validation loss, or an independent random number, breaks them and keeps the hypothesis. The statistic is a *U-statistic* in the sense of Hoeffding (1948), with the selected score of k trials as its kernel; for fixed k, and a kernel with finite variance, it is asymptotically normal as K grows.

Dodge et al. (2019) proposed reporting the *expected validation performance*, the expected best validation score as a function of the budget, and estimated it with the expected maximum of k draws from the empirical distribution of the pool, that is, of k draws *with* replacement. In the notation of (4.3), with $Y = V$, that estimator is

$$\tilde{M}_k = \sum_{i=1}^{K} \left(\left(\frac{i}{K}\right)^k - \left(\frac{i-1}{K}\right)^k \right) Y_{(i)}.$$

Tang et al. (2020) showed that it is biased, always toward a worse best.

Proposition 4.8 (Drawing with replacement is pessimistic). *For real numbers $y_{(1)} \le \cdots \le y_{(K)}$ and $1 \le k \le K$, let U_k and W_k be the right-hand sides of (4.3) and of the display above, with $y_{(i)}$ in place of $Y_{[i]}$ and $Y_{(i)}$. Then $W_k \le U_k$, with strict inequality if $k \ge 2$ and $y_{(1)} < y_{(K)}$. Consequently, for independent scores with a finite mean, $\mathbb{E}\tilde{M}_k \le \mathbb{E}M_k$.*

PROOF. The plan is to write both statistics as $y_{(K)}$ minus a weighted sum of the gaps between consecutive order statistics, and to compare the weights. Both are averages $\sum_i w_i y_{(i)}$ with weights $w_i = G(i) - G(i-1)$ for a function G that increases from $G(0) = 0$ to $G(K) = 1$; summation by parts turns such an average into

$$\sum_{i=1}^{K} w_i y_{(i)} = y_{(K)} - \sum_{i=1}^{K-1} G(i)\big(y_{(i+1)} - y_{(i)}\big). \tag{4.4}$$

For W_k, $G_W(i) = (i/K)^k$, the probability that k draws with replacement all fall among the i smallest values. For U_k, $G_U(i) = \binom{i}{k}/\binom{K}{k}$, the probability that a random subset of size k does, by the identity for the weights applied with i in place of K. By (4.4), $U_k - W_k = \sum_{i=1}^{K-1} \big(G_W(i) - G_U(i)\big)\big(y_{(i+1)} - y_{(i)}\big)$, and every gap is nonnegative, so it suffices to show $G_U(i) \le G_W(i)$. If $i < k$, then $G_U(i) = 0$. If $i \ge k$, then $G_U(i) = \prod_{r=0}^{k-1} \frac{i-r}{K-r}$, a product of k factors in $[0, 1]$, and each is at most i/K because $(i - r)K \le i(K - r)$ amounts to $r(K - i) \ge 0$. For strictness, let $k \ge 2$ and $1 \le i \le K - 1$. For $i < k$ we have $G_U(i) = 0 < G_W(i)$; for $i \ge k$ the factor with $r = 1$ is strictly smaller than i/K, because $K - i > 0$. Every coefficient is then positive, and some gap is positive when $y_{(1)} < y_{(K)}$. For the last claim, take expectations and use Theorem 4.7 with $Y = V$. ■

The bias is not the whole story. Dodge et al. (2021) compared the estimators by variance and mean squared error as well: in their simulations the unbiased estimator had the largest variance, and the two biased estimators led to the fewest wrong conclusions about which of two models was better. The trade-off is real, and the bias is small when k is a small fraction of K; in the main pools of Lab 4.2, up to $k = K/4$, the two estimates of the best validation error differ by less than 0.1 points. What matters more is to report K, the estimator, and how far the curve is pushed.

How far it can be pushed has a simple answer. The probability that the trial selected from a random k-subset is among the m best of the pool is $1 - \binom{K-m}{k}/\binom{K}{k}$, and the same factor-by-factor comparison as in the proof above bounds $\binom{K-m}{k}/\binom{K}{k}$ by $(1 - m/K)^k$. For $m \ge K/k$ this gives at least $1 - (1 - 1/k)^k \ge 1 - e^{-1} \approx 0.63$. At budget k, then, $\hat{M}_k$ rests mostly on the best K/k trials of the pool: at $k = K/4$, on about four. Intervals from the bootstrap, which resamples the pool (Theorem A.3), cannot create trials better than the best one observed, and Tang et al. (2020) found that bootstrap intervals for the expected maximum covered the truth only about 70% of the time at a nominal 95% when $k = 20$ and $K = 50$. We quote comparisons at $k \le K/10$ and draw curves no further than $K/4$.

THE SELECTION SCORE IS OPTIMISTIC. The best validation score of k trials overstates the selected trial's performance on fresh data, increasingly with k: selection on a noisy score picks trials that were lucky on that score, the winner's curse of Chapter 1, and Chapter 5 computes the effect exactly for a Gaussian proxy. With a small validation set, more trials can even make the selected trial worse on the test set,[8] because the search begins to fit the noise of the validation set (Cawley and Talbot 2010). The budget curve to report is therefore the one for the test score of the selected trial, as in (4.3) with $V \ne Y$, and the configuration that a paper finally reports should be retrained with fresh seeds, since its own run was selected for its luck.

[8] With 300 validation images and an error near 15%, the binomial standard error of a validation error is about two points (Chapter 1).

4.6 In the laboratory: two protocols on one task

On the plain digits, momentum and Adam generalize equally well: in the control of Lab 4.2 below, their budget curves agree within a quarter of a point at every budget from six trials on, in both kinds of search space. To reproduce the original gap in miniature we add the ingredient of Wilson et al.'s constructed example, input coordinates that carry no information about the label but let the network fit its small training set. Each image gets 256 distractor features, independent standard normal numbers drawn once per image, appended to its 64 pixels.

LAB 4.1 · TWO PROTOCOLS, ONE COMPARISON

SCRIPT
`04-budgets-protocols.py`

SETUP. Digits with 256 distractor features: 300 training images, 300 validation images, and the remaining 1197 as test set. A two-layer MLP of width 64, 100 epochs, batch size 32, constant learning rate. *Protocol 2017*: for each optimizer, a logarithmic grid of 16 learning rates (momentum 10^{-3} to 1; Adam $10^{-4.5}$ to $10^{-1.5}$), momentum $\beta_1 = 0.9$, Adam's $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$, no weight decay. *Protocol 2019*: for each optimizer, 16 random-search trials, log-uniform in every coordinate, over the learning rate, $1 - \beta_1$ and an L2 weight decay λ for momentum, and over the learning rate, $\epsilon \in [10^{-8}, 10^{2}]$, $1 - \beta_1$, $1 - \beta_2$ and λ for Adam, whose learning rate is drawn relative to $\max(\epsilon, 10^{-2})$. In both, the trial with the lowest validation error is retrained with five fresh seeds and their mean test error is the protocol's result. Everything is replicated on 10 data splits, each with new distractors, search draws and seeds: 840 runs, about three minutes.

PREDICT FIRST. Under the 2017 protocol, which optimizer has the lower test error, and by how much? With the same sixteen trials searched over all hyperparameters, does the gap survive, shrink or reverse? Which optimizer gains more from the larger search?

LOOK FOR. The differences split by split, not only the means; the values of ϵ and λ that the 2019 search selects; retrained runs of a selected configuration that fail.

Figure 4.2 shows the result. Under the 2017 protocol Adam had the higher test error on nine of the ten splits (two-sided sign test $p = 0.021$), by a median of 3.2 points. The mean difference, 1.69 ± 4.10 points (mean and 95% t-interval over splits, Theorem A.1), is diluted by one failure: on split 6 the grid selected momentum's learning rate 0.1, at the edge of stability, on the strength of one good run, and of its five retrained runs one ended at 87% test error and two above 25%.[9] Under the 2019 protocol, with the same sixteen trials per optimizer, the difference was -0.44 ± 0.59 points, with Adam lower on five splits of ten: the gap vanished. Both optimizers improved, Adam by 6.04 ± 0.49 points, to 16.11 ± 0.57%, and momentum to 16.55 ± 0.70%.

The selected hyperparameters show how. The 2019 searches mostly chose a sizable weight decay (median $\lambda = 0.027$ for Adam and 0.017 for

[9] At the edge of stability the seed decides whether a run survives; Chapter 2 studies how small differences grow during training.

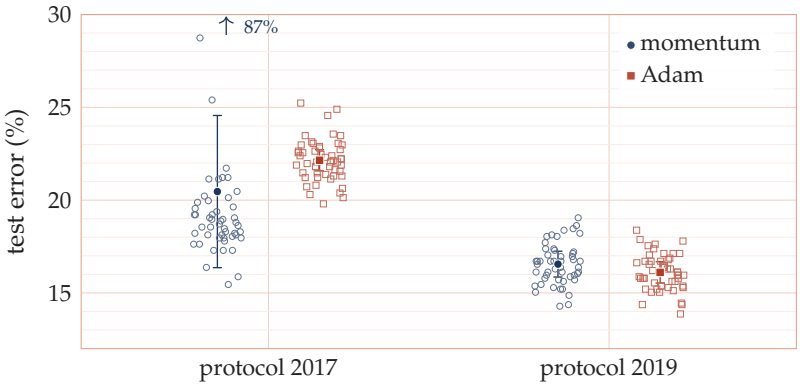


Figure 4.2. Lab 4.1: test error of the configuration selected by each protocol, for momentum (circles) and Adam (squares). Hollow marks: the five retrained runs of the selected configuration on each of 10 data splits (50 per arm and protocol, jittered horizontally); one run, at 87%, lies above the scale. Filled marks: the mean over the 10 splits of the five-run means, with a 95% t-interval over splits. Digits with 256 distractor features; 16 trials per optimizer and protocol.

momentum), a regularizer that the 2017 protocol held at zero for both arms, and half of Adam's selections had $\epsilon \geq 10^{-3}$, three of them above 1, where Adam moves almost like momentum. Which of the two closes the gap is the subject of Exercise 4.5. One more effect is visible in the numbers: the selected trial's own run was better on the test set than its five reruns, by 0.05 to 1.5 points depending on the arm, the optimism of selecting on a single run.

LAB 4.2 · BUDGET CURVES

SCRIPT
`04-budgets-curves.py`

SETUP. Split 0 of the task of Lab 4.1. Four pools of $K = 200$ random-search trials, one run each: momentum and Adam, each over its 2017 space (the learning rate only, log-uniform over the range of the grid) and its 2019 space (ranges in the scripts). For each pool, the expected test error of the trial that a search of k trials selects, for $k = 1, \ldots, 50$, by (4.3), with 95% percentile intervals from 1000 bootstrap resamples of the pool. A control repeats the four pools with $K = 100$ on the plain digits. 1200 runs, about four minutes.

PREDICT FIRST. At $k = 1$, which of the four arms is best? Do the curves of the two 2017 spaces cross, and where? Does more search always lower the test error of the selected trial?

LOOK FOR. Crossings; the budget beyond which the curves are flat; the distance between the expected best validation error and the expected test error of the selected trial; the control.

Figure 4.3 shows the four curves. In the 2017 spaces they cross. A single trial favors Adam, 26.1% (interval [25.1, 27.1]) against 43.2% ([38.8, 48.0]), because a third of momentum's learning-rate range diverges while Adam's

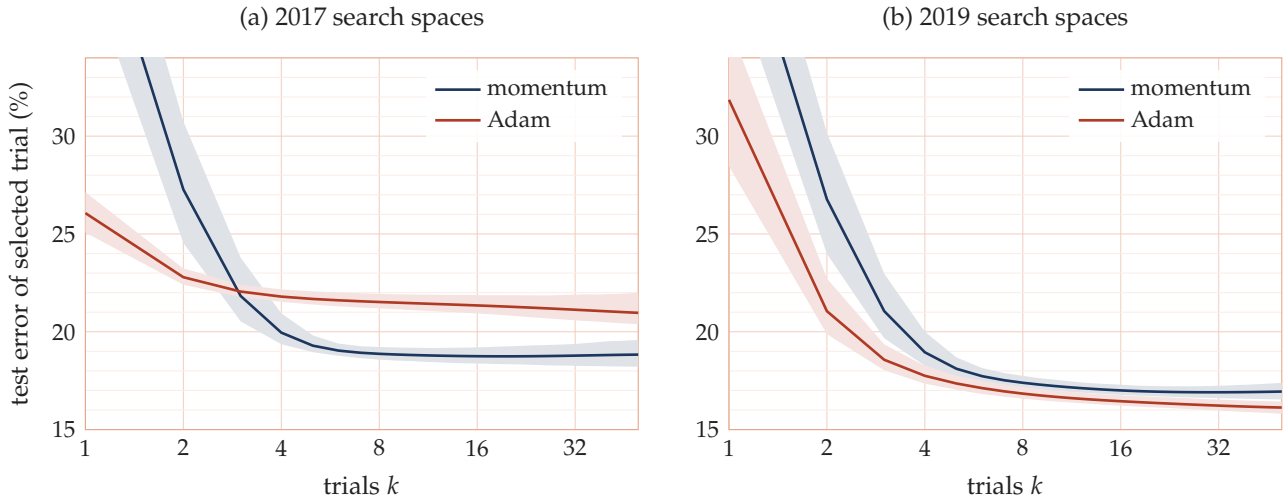


Figure 4.3. Lab 4.2: budget curves, the expected test error of the trial selected by a random search of k trials, estimated by (4.3) from pools of $K = 200$ trials per arm (one run each; split 0 of the task of Lab 4.1). Bands: 95% percentile intervals from 1000 bootstrap resamples of each pool. (a) Learning rate only, other hyperparameters at their defaults. (b) All hyperparameters. At $k = 1$ the momentum curves start at 43%, above the scale, because a third of their learning rates diverge.

range is forgiving; from three trials on momentum is ahead, and at $k = 16$ Adam's curve is higher by 2.6 points (95% bootstrap interval [2.0, 3.2]). This is Proposition 4.3 at work, with momentum the arm whose space has the higher ceiling and the lower floor. In the 2019 spaces Adam is ahead at every budget, by 0.55 points at $k = 16$ (interval [0.17, 0.90]): on this split, the comparison reversed rather than vanished. The same proposition appears within Adam: its 2019 space contains far more diverging trials than its 2017 space (33 of 200 against 6) and also better ones, so the two Adam curves cross between $k = 1$ and $k = 2$, after which the larger space is ahead by 4.9 points at $k = 16$.

Beyond about sixteen trials every curve is nearly flat. The expected best validation error keeps falling, from 15.5% at $k = 16$ to 14.8% at $k = 50$ for momentum's 2017 space, while the test error of the selected trial stays at about 18.8% (point estimates from the same pool): past this point the search mostly fits the noise of 300 validation images. The with-replacement estimate of the best validation error was larger than the unbiased one in all eight pools at every budget from two on, as Proposition 4.8 requires, but by less than 0.1 points in the main pools and 0.2 in the control pools. On the plain digits the control shows no gap: at $k = 10$, Adam minus momentum is -0.02 points (interval $[-0.23, 0.19]$) in the 2017 spaces and 0.01 points ($[-0.30, 0.30]$) in the 2019 spaces.

The miniature reproduces the shape of the story, with one honest qualification. The gap under the 2017 protocol needed the distractor features; without them the two optimizers tie under both protocols. What the lab shows in general is the mechanism: a comparison decided by a protocol that searched one optimizer's distinguishing hyperparameters and not the

other's, and a gap that a search of the same size over all of them removes.

4.7 Protocol

PROTOCOL 4.1 · COMPARING TWO TRAINING METHODS

- ☐ Write down each arm as an update rule plus a search space, a sampling distribution, a budget, a selection rule and a reporting rule, before running anything.
- ☐ Give the arms equal budgets in the unit of the claim: trials, steps, FLOPs or wall-clock time.
- ☐ Search every hyperparameter that distinguishes the arms, ϵ, the momentum coefficients and weight decay among them, over ranges wide enough that no optimum lies at an edge; report the ranges and how they were chosen.
- ☐ Select on validation data; report the test score of the selected configuration retrained with fresh seeds.
- ☐ Repeat the whole tuning protocol with new search draws, seeds and, where possible, data splits; report the paired differences with an interval.
- ☐ Report budget curves with the unbiased estimator (4.3) and bootstrap intervals for k up to about $K/4$, and say whether and where they cross.
- ☐ Treat hyperparameters taken from a paper as tuning done on other data: count them in the budget, or retune.
- ☐ State the conclusion with its budget: "A beats B for searches of k trials or more over these spaces".

The protocol makes a comparison fair to the update rules within the stated budget and spaces, and no further. It does not protect against search spaces designed with hindsight, refined on the very workload by an earlier search, as Choi et al. acknowledge for their own, nor against unequal engineering effort in the implementations. Its estimator assumes independent trials: a Bayesian optimizer or a successive-halving scheme chooses trials adaptively, and its budget curve must be estimated by repeating the whole search (Exercise 4.12). At the scale of large models even one search per arm can be unaffordable; then tune the hyperparameters that distinguish the arms on a smaller proxy and transfer them with stated rules, or fall back to the weakest honest claim, "at the settings we could afford", and report the budget that each arm actually received.

Exercises

4.1 CONSOLIDATION. (The shape of a budget curve.) Let F have a finite mean. (a) Use (4.1) to show that $\mathbb{E}M_{k+1} - \mathbb{E}M_k = \int F(x)^k(1 - F(x))\,\mathrm{d}x$. (b) Deduce that the increments are nonnegative and nonincreasing, so that $k \mapsto \mathbb{E}M_k$ is concave on the integers. (c) Compute the budget curve of the uniform distribution on $[a, b]$, and the number of trials needed to come within 1% of $b - a$ of the top.

4.2 CONSOLIDATION. (Crossing points.) In Example 4.4, show that arm 1 is ahead exactly when $k > \ln\big((a - c)/(a - b)\big)/\ln\big(1/(1 - p)\big)$, and find the crossing budget when the good fraction p is 0.01. What should a paper that compares the two arms with 100 trials each conclude, and how should it say so?

4.3 CONSOLIDATION. (The estimator by hand.) For $K = 4$ trials whose selection scores are 0.3, 0.9, 0.5 and 0.7 and whose reported scores are, in the same order, 0.40, 0.80, 0.70 and 0.60, compute $\hat{M}_2$ from (4.3) and check it against the average over the six pairs. Then compute $\tilde{M}_2$ for the validation scores and verify Proposition 4.8.

4.4 CONSOLIDATION. ★ (Rates and relatives of inclusion.) (a) In Proposition 4.6, assume each g_t is L-Lipschitz. Show that $\|\theta_T^\epsilon - \theta_T\| \le C/\epsilon$ for all large ϵ, with C depending on T, η, β_1, β_2, L and the gradients along the momentum path. (b) Show that with $\beta_1 = 0$ no schedule is needed: Adam with the constant step size $\epsilon\eta$ tends to SGD with learning rate η. (c) RMSProp with momentum, as implemented in PyTorch, divides the gradient by $\sqrt{v_t} + \epsilon$ before the momentum buffer: show that it contains momentum SGD with no schedule at all. *(Hint: in (a), carry a bound of order $1/\epsilon$ through the induction, using that the factor $1/(1 + x)$ differs from 1 by at most x.)*

4.5 LAB. (Which knob closes the gap?) Modify `04-budgets-protocols.py` so that the 2019 protocol searches Adam's learning rate and weight decay with $\epsilon = 10^{-8}$ fixed, and, separately, its learning rate and ϵ with no weight decay. Which part of the larger search closes the gap of Lab 4.1? Is the answer the same for $1 - \beta_1$?

4.6 LAB. (A naive search space.) Sample Adam's learning rate and ϵ independently, log-uniform on $[10^{-5}, 1]$ and $[10^{-8}, 10^2]$, instead of drawing the learning rate relative to $\max(\epsilon, 10^{-2})$. Rerun `04-budgets-curves.py` for this arm. How many trials does Adam now need to match momentum's 2019 curve? Relate your answer to the fraction of the space where Adam trains at all.

4.7 LAB. (Coverage.) Using the 200 trials of one pool of Lab 4.2 as a population, draw 300 subpools of $K = 40$ trials, compute the bootstrap interval for $\mathbb{E}M_k$ in each, and estimate its coverage of the population value for $k = 2, 4, 10, 20$. Compare with the rule of thumb of Section 4.5.

4.8 DESIGN. (Equal compute.) Method A's trials cost twice as much as method B's, and you have 64 GPU-hours, enough for 32 trials of A. Design a comparison that answers "which method should a practitioner with this budget use?" and one that answers "which update rule is better per trial?" Say what each reports, how many trials each arm gets, and how the conclusions can differ, using (4.1).

4.9 DESIGN. (A new optimizer for pretraining.) You may run 96 training runs of a 100-million-parameter language model to compare a new optimizer with AdamW. Decide which hyperparameters to search for each arm, how to split the runs between search and replication, how to choose the search spaces without spending the budget on them, and what to plot. Which finding of Wen et al. (2026) constrains your design?

4.10 CRITIQUE. (Recommended settings.) A paper compares its method, tuned by 100 trials of Bayesian optimization, with three baselines run at "the hyperparameters recommended by their authors". The method wins by 0.8 points; the spread of trial scores over its search space has standard deviation about 1 point. Using Corollary 4.5, say what this comparison can and cannot show, and what the authors should have run.

4.11 CRITIQUE. (A table of best results.) A benchmark table lists, for each of six optimizers, the best test accuracy over "extensive tuning", without the number of trials or the search spaces, and bolds the largest. List what is missing, rank the omissions by how much they could change the ranking, and propose a replacement figure.

4.12 RESEARCH. (Adaptive search.) Bayesian optimization and successive halving choose trials adaptively, so the trials of a search are not independent and Theorem 4.7 does not apply. Open: an estimator of the budget curve of an adaptive search method from fewer runs than repeating whole searches, with a valid interval; or a proof that none can exist without assumptions on the response surface.

Notes and References

The case for random search over grid search was made by Bergstra and Bengio (2012), who showed that it matches or beats grid search at a fraction of the cost, because on a given task few hyperparameters matter, and proposed it as the baseline for adaptive search methods (Chapter 5 makes it the placebo arm of a search). The expected best result as a function of the budget was proposed as a reporting standard by Dodge et al. (2019), with a checklist for reporting hyperparameter searches; Tang et al. (2020) proved that their estimator is biased and derived the unbiased one, and Dodge et al. (2021) weighed bias against variance. Theorem 4.7 with concomitants, which turns the curve of the best validation score into the curve of the selected trial's test score, is the form we found most useful in practice.

Cawley and Talbot (2010) analyzed over-fitting in model selection, the reason that curve can rise.

The pattern of this chapter, a gap that closes when the baseline is tuned as hard as the proposal, recurs across the field. Melis et al. (2018) found that standard LSTMs, properly regularized and tuned by large-scale black-box search, outperformed more recent recurrent architectures in language modeling; Lucic et al. (2018) found that most GAN variants reached similar scores given enough hyperparameter search and random restarts. Chapter 5 adds the placebo arm, **??** the question of which budget axis is held fixed, and **??** the effect of the learning-rate schedule on curves read before their end.

For practice, the tuning playbook of Godbole et al. (2023) classifies hyperparameters as scientific, nuisance and fixed; when optimizers are compared, the optimizer is scientific and its own hyperparameters are nuisance hyperparameters to be tuned separately for each arm. It suggests tuning only Adam's learning rate with fewer than about ten trials, adding β_1 at 10 to 25, ϵ beyond 25, and β_2 with substantially more.

TRANSLATING THE LITERATURE. Dodge et al. (2019) write n for our budget k and V_n^* for our M_k with $Y = V$; their expected validation performance $\mathbb{E}[V_n^* \mid n]$ is our $\mathbb{E}M_k$, and their estimator is our $\tilde{M}_k$. Tang et al. (2020) write B for our pool size K and $\hat{U}_n^B$ and $\hat{V}_n^B$ for our $\hat{M}_k$ and $\tilde{M}_k$. Choi et al. (2019) write γ for the momentum coefficient, our β_1, and v for momentum's buffer, our b; their Adam adds ϵ outside the square root and applies the bias corrections to the step, as TensorFlow does, so their ϵ is our ϵ times $\sqrt{1-\beta_2^t}$, the same after a few thousand steps when $\beta_2 = 0.999$. Wilson et al. (2017) write HB for heavy ball, our momentum SGD with $\beta_1 = 0.9$, and use "dev" for validation. Sivaprasad et al. (2020) call our 2017 space for Adam "Adam-LR".

References

Bergstra, J. and Y. Bengio (2012). "Random Search for Hyper-Parameter Optimization". In: *Journal of Machine Learning Research* 13, pp. 281–305.

Cawley, G. C. and N. L. C. Talbot (2010). "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation". In: *Journal of Machine Learning Research* 11, pp. 2079–2107.

Choi, D., C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl (2019). *On Empirical Comparisons of Optimizers for Deep Learning*. Preprint, arXiv:1910.05446 (version 3, 2020).

Dahl, G. E. et al. (2023). *Benchmarking Neural Network Training Algorithms*. Preprint, arXiv:2306.07179 (version 2, 2025).

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2019). "Show Your Work: Improved Reporting of Experimental Results". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, pp. 2185–2194.

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2021). "Expected Validation Performance and Estimation of a Random Variable's Maximum". In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. Association for Computational Linguistics, pp. 4066–4073.

Godbole, V., G. E. Dahl, J. Gilmer, C. J. Shallue, and Z. Nado (2023). *Deep Learning Tuning Playbook*. https://github.com/google-research/tuning_playbook. Version 1.0.

Hoeffding, W. (1948). "A Class of Statistics with Asymptotically Normal Distribution". In: *The Annals of Mathematical Statistics* 19.3, pp. 293–325.

Kasimbeg, P. et al. (2025). "Accelerating Neural Network Training: An Analysis of the AlgoPerf Competition". In: *International Conference on Learning Representations*.

Keskar, N. S. and R. Socher (2017). *Improving Generalization Performance by Switching from Adam to SGD*. Preprint, arXiv:1712.07628.

Kingma, D. P. and J. Ba (2015). "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations*.

Loshchilov, I. and F. Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations*.

Lucic, M., K. Kurach, M. Michalski, S. Gelly, and O. Bousquet (2018). "Are GANs Created Equal? A Large-Scale Study". In: *Advances in Neural Information Processing Systems 31*, pp. 700–709.

Luo, L., Y. Xiong, Y. Liu, and X. Sun (2019). "Adaptive Gradient Methods with Dynamic Bound of Learning Rate". In: *International Conference on Learning Representations*.

Melis, G., C. Dyer, and P. Blunsom (2018). "On the State of the Art of Evaluation in Neural Language Models". In: *International Conference on Learning Representations*.

Nado, Z., J. M. Gilmer, C. J. Shallue, R. Anil, and G. E. Dahl (2021). *A Large Batch Optimizer Reality Check: Traditional, Generic Optimizers Suffice Across Batch Sizes*. Preprint, arXiv:2102.06356.

Schmidt, R. M., F. Schneider, and P. Hennig (2021). "Descending through a Crowded Valley: Benchmarking Deep Learning Optimizers". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 9367–9376.

Sivaprasad, P. T., F. Mai, T. Vogels, M. Jaggi, and F. Fleuret (2020). "Optimizer Benchmarking Needs to Account for Hyperparameter Tuning". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 9036–9045.

Tang, R., J. Lee, J. Xin, X. Liu, Y. Yu, and J. Lin (2020). "Showing Your Work Doesn't Always Work". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 2766–2772.

Wen, K., D. Hall, T. Ma, and P. Liang (2026). "Fantastic Pretraining Optimizers and Where to Find Them". In: *International Conference on Learning Representations*. arXiv:2509.02046.

Wilson, A. C., R. Roelofs, M. Stern, N. Srebro, and B. Recht (2017). "The Marginal Value of Adaptive Gradient Methods in Machine Learning". In: *Advances in Neural Information Processing Systems 30*, pp. 4148–4158.

Zhou, P., J. Feng, C. Ma, C. Xiong, S. Hoi, and W. E (2020). "Towards Theoretically Understanding Why SGD Generalizes Better Than Adam in Deep Learning". In: *Advances in Neural Information Processing Systems 33*.

5 Controls

Random search as the placebo arm

THE CLAIM *"Neural architecture search finds better architectures than random search."*

As reported for ENAS (Pham et al. 2018) and DARTS (Liu, Simonyan and Yang 2019)

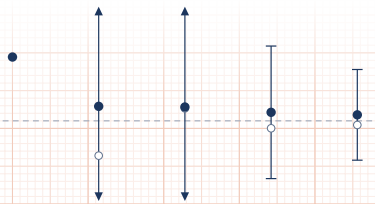
THE VERDICT Vanished: trained with the same pipeline, architectures sampled at random from the same space do about as well.

IN THIS CHAPTER. In 2018 and 2019 weight sharing made neural architecture search cheap, and ENAS and DARTS reported architectures much better than those of random search. When later studies gave the random arm the same space, the same training pipeline and enough draws, the searched architectures were ahead by a few tenths of a point on CIFAR-10 and not at all on Penn Treebank; the original gaps came from tiny random arms, from proxies that barely ranked architectures, and from pipelines that differed between arms. The method is the control arm of the older sciences: a placebo that samples the space, a sham that spends the same budget without the search strategy, and positive and negative controls. We prove that a search guided by a Gaussian proxy gains over the placebo the proxy's correlation with the truth times the gain of a perfect search, and reports a gain inflated by the inverse square of that correlation. On a benchmark of 192 digits networks, random search with a short-training proxy does at least as well as evolution, and a baseline copied from a weaker pipeline makes the search look more than twice as useful as it is.

5.1 The claim: better than random, at a thousandth of the cost

The first architecture searches were expensive. Zoph and Le (2017) trained a recurrent controller with reinforcement learning to write down network descriptions, with the validation accuracy of each trained network as the reward, and found a CIFAR-10 network with 3.65% test error and a recurrent cell that beat the LSTM on Penn Treebank. Searching for a repeatable cell made the result transferable to ImageNet (Zoph et al. 2018), and evolution in the same cell space produced AmoebaNet (Real et al. 2019), at search costs that Liu et al. (2019) list as 2000 and 3150 GPU-days.

WEIGHT SHARING. Pham et al. (2018) cut the cost by three orders of magnitude with *weight sharing*. In ENAS every candidate architecture is a subgraph of one large graph and inherits its weights, so no candidate is trained from scratch: a controller samples subgraphs and is trained by



policy gradient to maximize a reward measured on validation data, while the shared weights are trained on the sampled subgraphs. On one GTX 1080Ti the search took less than sixteen hours, and ENAS found a recurrent cell with test perplexity 56.3 on Penn Treebank and a convolutional cell with 2.89% test error on CIFAR-10 when trained with cutout, 3.54% without.[1] Liu et al. (2019) made the search differentiable. In DARTS each edge of the cell computes a softmax-weighted mixture of the candidate operations; the mixture weights are learned by gradient descent on the validation loss, the network weights on the training loss, and the strongest operations are kept at the end. Second-order DARTS reached 2.76 ± 0.09% test error on CIFAR-10 (mean and standard deviation of ten trainings of the selected cell) after four GPU-days of search, and 55.7 test perplexity on Penn Treebank after one.

[1] The arXiv version of the paper reports a perplexity of 55.8; the proceedings of ICML 2018 report 56.3.

THE RANDOM ARMS. Both papers asked whether the search mattered, which many papers of the period did not (Lindauer and Hutter 2020). ENAS sampled one architecture uniformly from each of its search spaces and trained it like the searched ones; DARTS ran a random search, drawing 24 cells (eight recurrent cells on Penn Treebank) and keeping the one with the best validation error after a short training.

> FINDING 5.1 (Pham et al. 2018; Liu et al. 2019). In the ENAS spaces, one uniformly sampled recurrent cell reached test perplexity 81.2 on Penn Treebank, against 56.3 for the ENAS cell, and one pair of sampled convolutional cells reached 6.77% test error on CIFAR-10, against 3.54% for the ENAS cells, both trained without cutout. In the DARTS space, the best of 24 random cells, kept for its validation error after 100 epochs, reached 3.29 ± 0.15% test error, against 2.76 ± 0.09% for second-order DARTS (means and standard deviations of ten trainings); the best of eight random recurrent cells, kept after 300 epochs, reached perplexity 59.4, against 55.7.

The DARTS paper drew a measured conclusion: random search was competitive on both tasks, which "reflects the importance of the search space design", and DARTS, at comparable or lower cost, improved on it substantially. The field received the stronger half. Weight-sharing searches, many of them variants of DARTS, multiplied, and papers compared themselves in tables of test errors and GPU-days taken from one another. The received claim was that a cheap search finds better architectures than random search does.

WHAT WAS RIGHT. The cost reduction was real, and it brought architecture search within reach of a university laboratory. Both papers ran a random control and published their code, and DARTS reported the variation of its final trainings. The claim was also reasonable. Searches had produced state-of-the-art classifiers for CIFAR-10 and ImageNet (Real et al. 2019; Zoph et al. 2018), and in a controlled comparison in the NASNet space, with the same code for every arm and five repeats, evolution and reinforcement learning both compared favorably against random search (Real et al. 2019).[2] What went wrong was not that search is useless, but that the control arms

[2] Each arm of that comparison trained 20,000 models for 25 epochs on 450 GPUs, about a week per experiment; the random arm was a random search with the same budget, a sham in the language of Section 5.4.

of the cheap searches were too weak to show what the search added.

5.2 The experiment: one search against one random draw

The comparison behind the received claim is clearest in the DARTS paper's CIFAR-10 table.

EXPERIMENTAL DESIGN · LIU, SIMONYAN AND YANG 2019, CIFAR-10

QUESTION	Does gradient-based search find better cells than random search in the same space?
TREATMENT	the search method: second-order DARTS, first-order DARTS, or random search
SPACE	the DARTS cells (convolutions, poolings and skip connections on a fixed cell graph)
SELECTION	DARTS run four times, random search drawing 24 cells; in each arm the cell with the best validation error after 100 epochs of training from scratch is kept
HELD FIXED	the final pipeline: 20 cells, 36 channels, 600 epochs, cutout, path dropout, auxiliary towers
RESPONSE	test error of the kept cell
REPLICATES	ten trainings of the kept cell; each arm's whole procedure (4 GPU-days of search) run once
ANALYSIS	mean and standard deviation of the ten trainings, beside numbers from other papers

The design has a strength that tables of the period often lacked: the random arm is concurrent, with the same space and the same pipeline, so the pipeline cannot explain the gap between 3.29% and 2.76%. Its weakness is the unit of replication. The claim is about two procedures, each of which returns a random architecture; the experiment ran each procedure once and trained the returned architecture ten times. The interval ±0.09 describes the trainings of one cell, not the variation of the search, and whether 3.29% was a typical outcome of a 24-draw random search or an unlucky one, the design cannot say. Nor does it say how good the average architecture of the space is, so the reader cannot split DARTS's 2.76% into what the space and pipeline give every architecture and what the search adds. The other rows of the table came from other papers, some re-trained under DARTS's setup; comparisons with them are historical. The ENAS controls had one architecture per space.

A reader in 2019 could conclude that DARTS had beaten one draw of random search under one pipeline, but not how large the effect of the search was relative to its own variability, nor how close a randomly sampled cell, trained the same way, would come.

5.3 The post-mortem: a fair draw for the random arm

Three studies, published in 2019 and 2020, gave the random arm what it had lacked: more draws, repeated searches, and the same pipeline.

Random search, done properly. Li and Talwalkar (2019) treated architecture search as hyperparameter optimization and ran two strong random baselines in the DARTS spaces. Random search with early stopping (the bandit method ASHA, from one to 300 epochs) evaluated more than 700 cells on CIFAR-10 in nine GPU-days and reached 3.03 ± 0.13% test error (mean and standard deviation of ten trainings), against 3.29% for the DARTS random baseline; on Penn Treebank it reached perplexity 56.4, against 59.4. Random search with weight sharing, which ranks thousands of random cells with weights trained on uniformly sampled subgraphs, reached 55.5 on Penn Treebank, better than DARTS's 55.7, and 2.85 ± 0.08% on CIFAR-10, close to DARTS's 2.76 ± 0.09% and to their own rerun of it, 2.78 ± 0.12%. They found none of twelve recent search papers exactly reproducible from what was released, and their own method, rerun with new seeds, gave 56.5 instead of 55.5.

The search phase against a placebo. Yu et al. (2020) compared each search with one architecture sampled uniformly from the same space and trained like the searched one, and repeated every arm ten times with different seeds.[3] On Penn Treebank the random cells reached validation perplexity 60.13 ± 0.65 (mean and standard deviation over the seeds), against 59.88 ± 1.92 for ENAS, 60.61 ± 2.54 for DARTS and 61.99 ± 1.95 for NAO; Welch's t-tests gave $p = 0.73$, 0.62 and 0.02, the last because NAO was worse. On CIFAR-10 the random cells reached 96.44 ± 0.19% accuracy and the three searches between 96.62% and 96.86%, less than half a point better. They traced the weakness to the proxy: in a reduced recurrent space of 32 cells, each trained ten times to obtain its true rank, the Kendall rank correlation between the ranking given by shared weights and the true one averaged −0.004 over ten runs, and in NAS-Bench-101 spaces of three to seven nodes it fell from 0.44 to 0.20; samplers trained on true accuracies instead beat random sampling with probability 0.90 (ENAS) and 0.92 (NAO). Their reading was that the spaces were constrained so well that a random architecture already did well.

[3] The paper first circulated in 2019 with Christian Sciuto as first author and is often cited as Sciuto et al.; the published version lists Kaicheng Yu and Christian Sciuto as equal first authors.

Eight methods, five datasets, one placebo. Yang et al. (2020) ran eight search methods on five image datasets, eight searches and eight random architectures per method and dataset, all trained with each method's own pipeline. Their score, the relative improvement over the placebo in percent of its accuracy, ranged on CIFAR-10 from −0.48 to 0.74 (DARTS 0.32, ENAS 0.01), and on the other datasets several methods fell below their placebo. The training protocol mattered more: from the plainest pipeline to one with every trick, the same random cells gained more than three points, while the best improvement of a search over random sampling on CIFAR-10 was 0.69 points. Over 214 random cells of the DARTS space, trained with the DARTS pipeline, accuracy was 97.03 ± 0.23% (mean and standard deviation), from 96.18% to 97.56%; a deliberately weaker set of operations moved this

distribution by only 0.18 points, and two trainings of the same 32 cells with different seeds ranked them with Kendall $\tau = 0.48$.

Worse than the placebo, and better. In the tabular benchmark NAS-Bench-201, where all 15,625 cells have been trained, DARTS with its original hyperparameters converged to a cell made only of skip connections, with 54.30% test accuracy on CIFAR-10, while random search reached 93.70 ± 0.36% (mean and standard deviation of 500 runs) and the best cell 94.37% (Dong and Yang 2020); Zela et al. (2020) found such degenerate solutions of DARTS in a wide range of spaces. The same literature marks the limits of the verdict: in NAS-Bench-101, where the proxy is full training, regularized evolution and two Bayesian optimizers overtook random search after about 25 evaluated architectures and reached its final performance about five times faster (Ying et al. 2019). What vanished is the margin of the cheap, weight-sharing searches over a fair random arm in the spaces where they were introduced.

The mechanism. Four things made a small effect look large. The random arms were underpowered: one architecture per space for ENAS, one random search for DARTS, with selection after a short training that Li and Talwalkar (2019) found unstable. The spaces were narrow by design, so the spread between random architectures was about the size of the noise between seeds. The proxies, shared weights, ranked architectures little better than chance, and a search guided by a proxy that does not rank cannot beat the placebo (Section 5.5). And comparisons with published numbers mixed pipelines whose differences were worth more than the search. The same pattern appears wherever new methods are compared with old numbers: given the same tuning and pipeline, the standard LSTM matched or beat newer recurrent cells, the NAS cell among them (Melis et al. 2018), the GAN variants tested did not consistently beat the original (Lucic et al. 2018), and claimed gains over decade-old metric-learning methods shrank to marginal (Musgrave et al. 2020). The missing control was the old method given the new method's budget and pipeline (Chapter 4).

verdict. The large margin of the cheap, weight-sharing searches over random architectures did not survive a fair random arm. With the same space, pipeline and enough draws, random architectures matched the searched cells on Penn Treebank and came within a few tenths of a point on CIFAR-10, a remainder of the order of the seed-to-seed variation that ten searches detected for some methods and not for others. The reported gaps came from random arms of one to twenty-four architectures, from proxies that barely ranked, and, in tables of published numbers, from different pipelines. With a proxy that ranks well and a space with real spread, search does beat random search, by a small margin.

VANISHED

5.4 The method: placebo, sham and the spread of a space

An experiment learns about a treatment only when its arms differ in nothing else, and the older sciences name the arms that make this true. A *negative control* is a condition in which the effect cannot occur, and the assay must show none; epidemiologists borrowed the practice from the laboratory to detect bias (Lipsitch et al. 2010). A *positive control* is a condition in which the effect is known to occur, and the assay must show it; the guideline on control groups in clinical trials calls the ability of a trial to tell an effective treatment from an ineffective one its *assay sensitivity* (International Conference on Harmonisation 2000). A *placebo* is an inert treatment that looks like the real one, and controls for every influence on the course of the disease other than the drug, regression to the mean and expectations included. A *sham* procedure, in surgery and device trials, is the whole procedure without its active step. The guideline treats the external or *historical control*, a group treated earlier or elsewhere, separately, because it can rarely guarantee that the groups are comparable.[4]

A search method has the same anatomy. It consists of a search space $\mathcal{A}$ with a sampling distribution π (uniform, for a finite space); a *proxy*, a cheap and noisy measurement of an architecture's quality such as the validation error after a short training or under shared weights; a budget of k proxy evaluations; a rule that selects the winner; and a final training pipeline that produces the reported number. The active ingredient is the search strategy; everything else is the vehicle.

Definition 5.2 (Controls for a search method). For a search method with space $\mathcal{A}$, distribution π, proxy, budget k, selection rule and final pipeline:

(i) the *placebo arm* draws one architecture from π and trains it with the final pipeline (random sampling);

(ii) the *sham arm* draws k architectures independently from π, evaluates them with the same proxy, selects with the same rule and trains the winner with the final pipeline (random search with the same budget);

(iii) a *positive control* is a run in which a gain is known to exist, such as a perfect proxy (an *oracle*) or a planted architecture of known quality, and the experiment must detect it;

(iv) a *negative control* is a run in which no gain can exist, such as a proxy whose values are permuted among the architectures, and the experiment must report none;

(v) a *historical control* is a number produced earlier by another pipeline, usually in another paper.

The placebo asks whether searching helps at all, and measures the space; the sham asks whether this search strategy helps more than drawing at

[4] Sacks et al. (1982) found six therapies tested both ways: 44 of 56 trials with historical controls (79%) found the new therapy better, but only 10 of 50 randomized trials (20%). The treated patients did about equally well in both; the historical control groups did worse.

random with the same proxy and budget.[5] A method that beats its placebo but not its sham owes its gain to the proxy and the budget, which random search exploits equally well. Both arms are concurrent: they share the space, the proxy and the pipeline with the search. A historical control shares none of them. **??** uses the same device for ablations, with a shuffled pruning mask as the sham.

[5] Lindauer and Hutter (2020) make the same distinction, between *random sampling*, which returns one random architecture at no search cost, and *random search*, which runs as long as the method it is compared with, and recommend reporting both.

WHAT A HISTORICAL COMPARISON MEASURES. Write $\mu_p(a)$ for the true error of architecture a trained by pipeline p, the mean of its run distribution over seeds, and $\bar{\mu}_p$ for the mean of $\mu_p(A)$ over $A \sim \pi$, the expected error of the placebo. A paper that reports its searched architecture A under a new pipeline against a baseline b measured under an old pipeline reports $\mu_{\text{old}}(b) - \mu_{\text{new}}(A)$. Adding and subtracting $\mu_{\text{new}}(b)$ and $\bar{\mu}_{\text{new}}$ splits it into three parts:

$$\mu_{\text{old}}(b) - \mu_{\text{new}}(A) = \underbrace{\mu_{\text{old}}(b) - \mu_{\text{new}}(b)}_{\text{pipeline}} + \underbrace{\mu_{\text{new}}(b) - \bar{\mu}_{\text{new}}}_{\text{baseline against space}} + \underbrace{\bar{\mu}_{\text{new}} - \mu_{\text{new}}(A)}_{\text{search}}. \tag{5.1}$$

Only the last term belongs to the search, and only a concurrent placebo arm under the new pipeline measures it. The middle term is negative when the baseline is better than the average architecture of the space, as a well-chosen textbook architecture usually is; the first is the co-treatment of **??**.

THE SPREAD OF A SPACE. Let Y be the test error of one run and $\mu(a) = \mathbb{E}[Y \mid a]$ the *true error* of architecture a under the fixed pipeline. For $A \sim \pi$, the mean $\bar{\mu}$ of $\mu(A)$ is the expected error of the placebo, its standard deviation $\sigma_{\mathcal{A}}$ is the *spread* of the space, and $H = \bar{\mu} - \mu^\star$, with $\mu^\star$ the smallest true error in the space, is its *headroom*. A search that returns the architecture $A_{\mathcal{S}}$ has *placebo-adjusted gain* $G = \bar{\mu} - \mathbb{E}\mu(A_{\mathcal{S}})$.

REMARK 5.3 (No search beats the headroom). Every search satisfies $G \le H$, because $\mu(A_{\mathcal{S}}) \ge \mu^\star$ for every outcome of the search. In a space whose headroom is a few tenths of a point, no strategy can gain more than that over the placebo, however expensive.

The spread is measured with noise. If each of m sampled architectures is trained n times and $\bar{Y}_a$ is its mean, then $\operatorname{Var} \bar{Y}_a = \sigma^2_{\mathcal{A}} + \sigma^2/n$, where σ^2 is the run-to-run variance within an architecture, so the standard deviation of the observed means overstates the spread; the one-way random-effects estimator subtracts the within-architecture mean square divided by n (Theorem A.12; the crossed version is in Chapter 1). The best observed mean is biased downward like every best-of-k number (Chapter 1), so the headroom is estimated honestly only by retraining the winner with fresh seeds.

How many random draws does a perfect search need? The answer does not depend on the size of the space.

Proposition 5.4 (Random search reaches a quantile). *Let $A_1, \dots, A_k$ be independent draws from π, and let $Q \subseteq \mathcal{A}$ be a set with $\pi(Q) = q \in (0, 1]$,*

for instance the best q-fraction of the space. Then

$$\mathbb{P}\big(A_i \in \mathcal{Q} \text{ for some } i \le k\big) = 1 - (1-q)^k \ge 1 - e^{-kq},$$

so $k \ge \ln(1/\delta)/q$ draws reach $\mathcal{Q}$ with probability at least $1-\delta$.

Proof. The events $\{A_i \notin \mathcal{Q}\}$ are independent, each with probability $1-q$, so all k occur with probability $(1-q)^k$, and the complement is the stated event. The inequality $1 - q \le e^{-q}$, which says that the convex function e^{-q} lies above its tangent at $q = 0$, gives $(1-q)^k \le e^{-kq}$, and $k \ge \ln(1/\delta)/q$ makes $e^{-kq} \le \delta$. The degenerate case $q = 1$ is covered as well: every draw lies in $\mathcal{Q}$, and the probability is $1 - 0^k = 1$. ∎

With $q = \delta = 0.05$, sixty draws reach the best 5% of a space with probability $1-0.95^{60} = 0.954$, whether the space has a hundred architectures or 10^{18}.[6] But a random search returns a draw from $\mathcal{Q}$ only when its proxy recognizes it, which is the subject of the next section.

[6] Bergstra and Bengio (2012) use the same formula, $1-(1-v/V)^T$ for a target of relative volume v/V and T trials, to compare random search with grids.

5.5 The method: a proxy caps what a search can find

A proxy evaluation of an architecture returns a random number P, such as the validation error of one short training with a fresh seed. For $A \sim \pi$ and one proxy evaluation of it, $T = \mu(A)$ and P form a random pair (T, P). Random search with budget k draws k independent copies $(T_1, P_1), \dots, (T_k, P_k)$ and returns the architecture with the smallest proxy error. What that does to $\mathbb{E}T$ runs through the *regression function* m, defined by $m(P) = \mathbb{E}[T \mid P]$: the expected true error of an architecture given what the proxy says.

Lemma 5.5 (Selection acts through the regression). *Let $(T_1, P_1), \dots, (T_k, P_k)$ be independent copies of a pair (T, P) with $\mathbb{E}|T| < \infty$, and let $m(P) = \mathbb{E}[T \mid P]$. Let the index $J \in \{1, \dots, k\}$ be a function of $P_1, \dots, P_k$ and of a random variable U independent of all the pairs. Then $\mathbb{E}T_J = \mathbb{E}m(P_J)$.*

Proof. The plan is to condition on everything the selection sees. Let $\mathcal{G}$ be the σ-field generated by $P_1, \dots, P_k$ and U; by hypothesis the indicator $\mathbf{1}\{J = i\}$ is $\mathcal{G}$-measurable. The pair (T_i, P_i) is independent of the other proxies and of U, so conditioning on them adds nothing to conditioning on P_i, and $\mathbb{E}[T_i \mid \mathcal{G}] = m(P_i)$ almost surely. In the display below, the first equality is the tower property with the $\mathcal{G}$-measurable indicator taken out of the conditional expectation, legitimate because $|T_i \mathbf{1}\{J = i\}| \le |T_i|$ is integrable, and the second is the identity $\mathbb{E}[T_i \mid \mathcal{G}] = m(P_i)$:

$$\mathbb{E}\big[T_i \mathbf{1}\{J = i\}\big] = \mathbb{E}\big[\mathbf{1}\{J = i\}\, \mathbb{E}[T_i \mid \mathcal{G}]\big] = \mathbb{E}\big[\mathbf{1}\{J = i\}\, m(P_i)\big].$$

Summing over i gives $\mathbb{E}T_J = \sum_i \mathbb{E}[T_i \mathbf{1}\{J = i\}] = \mathbb{E}m(P_J)$. ∎

The lemma is the theory of the negative control: if the proxy carries no information, so that m is constant, then $\mathbb{E}T_J = \bar{\mu}$ for every rule that looks only at proxy values, however clever, and the search is the placebo. With a

Gaussian proxy the gain can be computed. Recall from Chapter 1 that e_k is the expected maximum of k independent standard normal variables.

Theorem 5.6 (Selection on a Gaussian proxy). *Let (T, P) be bivariate normal with means μ_T, μ_P, standard deviations $\sigma_T, \sigma_P > 0$ and correlation ρ, let $(T_1, P_1), \dots, (T_k, P_k)$ be independent copies, and let J be the index of the smallest P_i. Then*

$$\mathbb{E}T_J = \mu_T - \rho\,\sigma_T\,e_k \qquad \text{and} \qquad \mathbb{E}P_J = \mu_P - \sigma_P\,e_k. \tag{5.2}$$

In the additive model $P = T + \xi$, with $\xi \sim \mathcal{N}(0, s^2)$ independent of T, the correlation is $\rho = \sigma_T/\sqrt{\sigma_T^2 + s^2}$.

Proof. The plan is to compute the regression function, apply Lemma 5.5, and evaluate the expected minimum. The P_i have a continuous distribution, so ties have probability zero and J is almost surely a function of the proxies; the lemma applies with no extra randomness U.

The regression. Let $\beta = \rho\sigma_T/\sigma_P$ and $R = T - \mu_T - \beta(P - \mu_P)$. Then $\operatorname{Cov}(R, P) = \rho\sigma_T\sigma_P - \beta\sigma_P^2 = 0$. The pair (R, P) is a linear image of (T, P) and so is jointly normal, and uncorrelated jointly normal variables are independent; hence $\mathbb{E}[R \mid P] = \mathbb{E}R = 0$, that is, $m(P) = \mu_T + \beta(P - \mu_P)$. By Lemma 5.5, $\mathbb{E}T_J = \mu_T + \beta\,(\mathbb{E}P_J - \mu_P)$.

The minimum. The variables $Z_i = (P_i - \mu_P)/\sigma_P$ are independent standard normals and $(P_J - \mu_P)/\sigma_P = \min_i Z_i = -\max_i(-Z_i)$, where the $-Z_i$ are again independent standard normals; so $\mathbb{E}\min_i Z_i = -e_k$ and $\mathbb{E}P_J = \mu_P - \sigma_P e_k$. Substituting, $\mathbb{E}T_J = \mu_T - \beta\sigma_P e_k = \mu_T - \rho\sigma_T e_k$.

The additive model. Independence of T and ξ gives $\operatorname{Cov}(T, P) = \sigma_T^2$ and $\sigma_P^2 = \sigma_T^2 + s^2$, so $\rho = \sigma_T^2/(\sigma_T\sigma_P) = \sigma_T/\sqrt{\sigma_T^2 + s^2}$. ∎

With $T = \mu(A)$, so that $\mu_T = \bar{\mu}$ and $\sigma_T = \sigma_{\mathcal{A}}$, random search with a Gaussian proxy has placebo-adjusted gain $G = \rho\,\sigma_{\mathcal{A}}\,e_k$. There are three ways to be no better than the placebo: a narrow space ($\sigma_{\mathcal{A}}$ small), a small budget (e_k grows only like $\sqrt{2\ln k}$), or a proxy that does not rank (ρ near zero).

Corollary 5.7 (Efficiency and the winner's curse). *Under the hypotheses of Theorem 5.6:*

(i) the placebo-adjusted gain of the proxy-guided search is the fraction ρ of the gain $\sigma_T e_k$ of the same search with a perfect proxy;

(ii) in the additive model, the gain that the search reports on its own proxy, $\mu_P - \mathbb{E}P_J$, is the true gain divided by ρ^2.

Proof. A perfect proxy selects the smallest T_i, and the computation of the minimum in the proof of Theorem 5.6, applied to the T_i, gives $\mathbb{E}\min_i T_i = \mu_T - \sigma_T e_k$; dividing $\rho\sigma_T e_k$ by $\sigma_T e_k$ proves (i). In the additive model $\mathbb{E}\xi = 0$ gives $\mu_P = \mu_T$, and $\sigma_P = \sigma_T/\rho$, so the reported gain is $\sigma_P e_k = \sigma_T e_k/\rho$ while the true gain is $\rho\sigma_T e_k$; their ratio is $1/\rho^2$. ∎

Part (ii) is the *winner's curse* of a search: the winner's proxy score is the best of k noisy numbers, and its noise was selected along with its quality. With $\rho = 0.5$ a search reports four times the gain it achieved. Animal breeders know Theorem 5.6 in the additive case as the breeder's equation $R = h^2 S$: the response to selection is the heritability times the selection differential.[7]

Search papers report the quality of a proxy as a rank correlation, Kendall's τ (Kendall 1938): the probability that two architectures are ordered the same way by the proxy and by the truth, minus the probability that they are ordered oppositely (Proposition A.15). For Gaussian pairs it determines ρ.

Lemma 5.8 (Kendall's τ of a Gaussian pair). *Let (X, Y) be bivariate normal with positive variances and correlation ρ, and let $(X_1, Y_1), (X_2, Y_2)$ be independent copies. Then*

$$\tau = \mathbb{P}\big((X_1 - X_2)(Y_1 - Y_2) > 0\big) - \mathbb{P}\big((X_1 - X_2)(Y_1 - Y_2) < 0\big) = \frac{2}{\pi} \arcsin \rho.$$

Proof. The plan is to reduce to a pair of standard normals and read the answer off an angle. The differences $U = (X_1 - X_2)/\sqrt{2 \operatorname{Var} X}$ and $V = (Y_1 - Y_2)/\sqrt{2 \operatorname{Var} Y}$ are jointly normal with unit variances and, because the copies are independent, correlation ρ; they have the signs of $X_1 - X_2$ and $Y_1 - Y_2$. If $\rho = \pm 1$, then $V = \pm U$ almost surely and $\tau = \pm 1 = \frac{2}{\pi} \arcsin(\pm 1)$. Let $|\rho| < 1$ and $\varphi = \arcsin \rho \in (-\pi/2, \pi/2)$, so that $\sqrt{1 - \rho^2} = \cos \varphi$. The pair (U, V) has the law of $(Z_1, Z_1 \sin \varphi + Z_2 \cos \varphi)$ with independent standard normals Z_1, Z_2; write $(Z_1, Z_2) = \Lambda(\cos \Theta, \sin \Theta)$ in polar coordinates, with Θ uniform on $[0, 2\pi)$ and $\Lambda > 0$. Then $U = \Lambda \cos \Theta$ and $V = \Lambda \sin(\Theta + \varphi)$. Both are positive when $\Theta \in (-\pi/2, \pi/2)$ and $\Theta + \varphi \in (0, \pi)$ modulo 2π, that is, on the arc $(-\varphi, \pi/2)$ of length $\pi/2 + \varphi$, and both are negative on the opposite arc, of the same length. Hence $\mathbb{P}(UV > 0) = \frac{1}{2} + \varphi/\pi$, the event $UV = 0$ has probability zero, and $\tau = 2\mathbb{P}(UV > 0) - 1 = 2\varphi/\pi$. ∎

Inverting, $\rho = \sin(\pi\tau/2)$: $\tau = 0.2$ corresponds to $\rho = 0.31$, and $\tau = 0.5$ to $\rho = 0.71$.

Example 5.9 (Reading published correlations). The weight-sharing ranking of Yu et al. (2020) in their 32-cell recurrent space had $\tau = -0.004$, so $\rho \approx 0$, and Theorem 5.6 predicts no gain over the placebo, which is what they found. In the seven-node NAS-Bench-101 space $\tau = 0.195$ gives $\rho = 0.30$: the search captures 30% of what a perfect search with the same draws would. Two single trainings of the same 32 DARTS cells had $\tau = 0.48$ (Yang et al. 2020), a correlation $\rho_2 = \sin(0.24\pi) = 0.68$. If each training is the true quality plus independent noise, then $\rho_2 = \sigma_{\mathcal{A}}^2/(\sigma_{\mathcal{A}}^2 + \sigma^2)$, and their standard deviation of 0.23 points across cells splits into a spread $\sigma_{\mathcal{A}} \approx 0.19$ and a seed noise $\sigma \approx 0.13$ points. A perfect search of 24 random cells, the size of the DARTS random arm, would gain $\sigma_{\mathcal{A}} e_{24} \approx 0.19 \times 1.95 = 0.37$ points over the average cell; a proxy with $\tau = 0.2$, about 0.11, less than the noise of one training. These are orders of magnitude, resting on a Gaussian model and another group's implementation.

[7] The equation goes back to Jay Lush's *Animal Breeding Plans* of 1937; Falconer and Mackay (1996) derive it. Here $S = \sigma_P e_k$, $h^2 = \rho^2$, and $R = \rho \sigma_T e_k$.

The theorem assumes that the regression of the truth on the proxy is a straight line all the way into the tail where the winners come from. Real proxies have a second, fixed error, which bends it.

A BIASED PROXY REACHES A PLATEAU. Let a proxy evaluation of a be $\mu(a) + b(a) + \varepsilon$, with a bias $b(a)$ that belongs to the architecture (a short training favors fast learners) and noise ε that is fresh at each evaluation. In a finite space, as the budget grows the draws cover the space and the winner tends to the minimizer of $\mu + b$, perturbed by the selected noise; its expected true error tends to a limit set by how well $\mu + b$ ranks the architectures, not by k, short of the headroom. A strategy that concentrates its evaluations on the proxy's favorites, as evolution does, reaches this plateau sooner, and when the bias is large a better optimizer of the proxy can end with a worse architecture than random search. ◇

REMARK 5.10 (How many searches). The unit of a search experiment is the search. With n searches per arm and r trainings of each returned architecture, an arm's mean has variance $(\sigma_S^2 + \sigma^2/r)/n$, where σ_S is the standard deviation of the true error of what the search returns ($\sigma_S = \sigma_{\mathcal{A}}$ for the placebo). The sample-size formula of Proposition A.14 gives $n \approx 2(z_{0.975} + z_{0.8})^2(\sigma_S^2 + \sigma^2/r)/G^2$ per arm for power 0.8 at level 0.05. With $\sigma_S = 0.2$, $\sigma = 0.13$ and $r = 1$ (Example 5.9), a gain of $G = 0.3$ points needs about ten searches per arm, and a gain of 0.2 points about 22.

5.6 In the laboratory: a tabular benchmark of 192 networks

Architecture search on the digits data is cheap enough to do exhaustively: we train every architecture of a small space several times, store the results in a *tabular benchmark*, and run thousands of searches as table lookups, as NAS-Bench-101 does for CIFAR-10 (Ying et al. 2019). The data are labkit's split 0 of the scikit-learn digits; its 1258 training images are divided once into 958 for training and 300 for validation, and the 539 test images, each worth 0.19 points of test error, are used only to define the truth.

LAB 5.1 · THE SPREAD OF A SEARCH SPACE

SETUP. Every MLP with depth 1 to 4, width 16, 32, 64 or 128, activation ReLU, tanh or GELU, dropout 0 or 0.25, and batch normalization on or off: 192 architectures. One fixed pipeline: Adam with learning rate 10^{-3}, batch 64, 30 epochs at a constant learning rate, on the 958 training images. Each architecture is trained six times (seeds 2000–2005) and scored on the test images; the mean of the six is its true error. 1152 runs, three to four minutes.

PREDICT FIRST. How far is the average architecture from the best? Is the spread between architectures larger or smaller than the spread between two seeds of one architecture? Answer for the full space and for the 96 architectures of width 64 or 128.

SCRIPT
05-controls-space.py

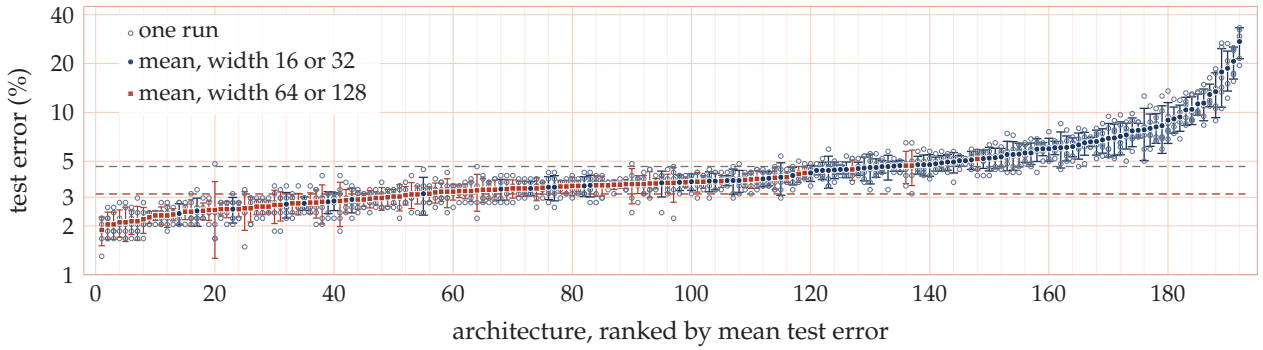


Figure 5.1. The spread of the search space of Lab 5.1: 192 MLP architectures for the digits data, ranked by mean test error over six runs (seeds 2000–2005, Adam, 30 epochs). Hollow marks: the six runs of each architecture. Filled marks: means with 95% t-intervals, vermilion squares for the 96 curated architectures (width 64 or 128) and ink circles for the rest. Dashed lines: the expected error of a randomly sampled architecture, 4.64% in the full space (grey) and 3.14% in the curated space (vermilion). Logarithmic vertical axis.

LOOK FOR. The shape of the ranked curve, the overlap of the intervals near the top, and where the wide architectures sit.

WHAT THE SPACE LOOKS LIKE. Figure 5.1 shows two spaces in one. The full space has a long tail: its mean true error is 4.64%, the standard deviation of its means 3.18 points, and its worst architecture, four layers of 16 ReLU units with dropout, errs on 27.33% of the test images. Width decides most of it (mean error 7.99% at width 16, 2.91% at width 128), then batch normalization and dropout. The best architecture, two layers of 128 GELU units with batch normalization, reaches 1.89 ± 0.38% (mean and 95% t-interval over six runs), so the headroom of the full space is 2.76 points. The 96 wide architectures form the kind of space a designer writes down first: its mean is 3.14%, its headroom 1.26 points, and the random-effects estimate of its spread, $\sigma_{\mathcal{A}} = 0.62$ points, is about 1.3 times the seed noise within an architecture, $\sigma = 0.47$ points; in the DARTS space the ratio was about $0.19/0.13 \approx 1.5$ (Example 5.9). Two single runs rank the curated architectures with Kendall $\tau = 0.52$ on average (0.39 to 0.66 over the fifteen pairs of seeds), against 0.48 in the DARTS space, and eleven of the 95 other curated architectures cannot be told apart from the best by a Welch test at level 0.05.

LAB 5.2 · A SEARCH, ITS SHAM AND ITS CONTROLS

SCRIPT
`05-controls-search.py`

SETUP. Four more runs of each architecture (seeds 1000–1003) record the validation error after every epoch: the proxy of fidelity f is the error after f epochs, and the main proxy is $f = 2$. An evaluation draws one of the four runs at random, as NAS-Bench-101 does. Five arms, each repeated 2000 times with $k = 1, 2, 4, \ldots, 64$ evaluations: the placebo, random search (the sham), regularized evolution (population

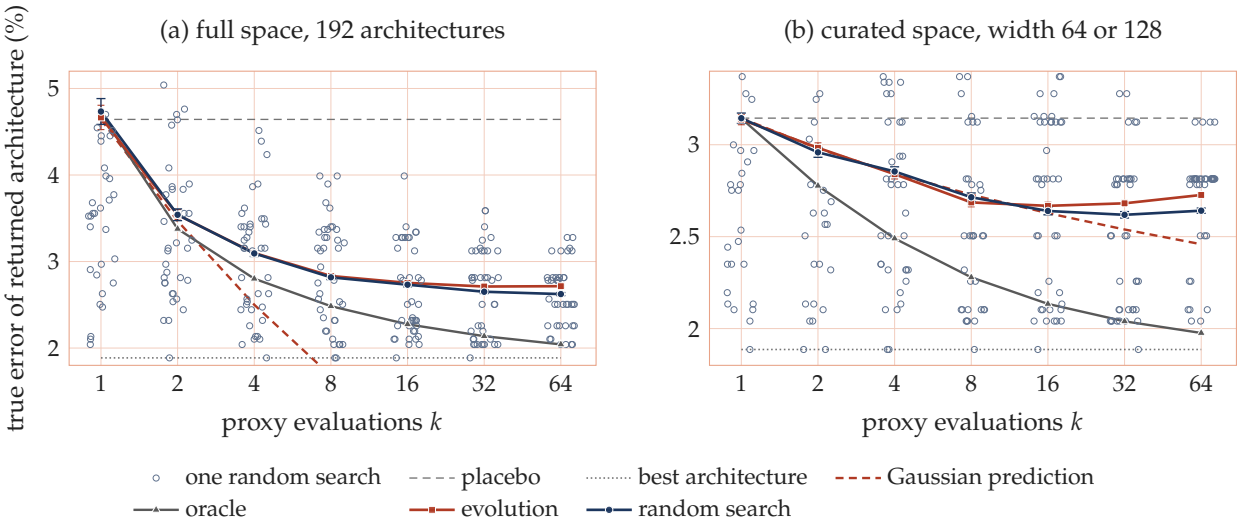


Figure 5.2. Searches on the benchmark of Lab 5.2, guided by the two-epoch proxy (four runs per architecture, seeds 1000–1003). Vertical axis: the true error (mean of six test runs) of the architecture a search returns. Lines with marks: means over 2000 searches with 95% intervals (most smaller than the marks), for random search (ink), regularized evolution (vermilion) and the oracle that selects by true error (grey). Hollow marks: 40 single random searches at each k, spread sideways. Dashed grey: the placebo; dotted: the best architecture; dashed vermilion: Theorem 5.6 for random search, from the space's mean and standard deviation and the proxy's correlation with the truth ($\rho = 0.66$ in the full space, 0.45 in the curated one). The shuffled-proxy search (not drawn) stays at the placebo level.

10, tournament 3, one-attribute mutations), the oracle (best of k draws by true error, a positive control) and random search with a shuffled proxy (a negative control). A search is scored by the true error of what it returns. 768 runs, about three minutes.

PREDICT FIRST. In the curated space the two-epoch proxy has Kendall $\tau = 0.32$ with the truth. What fraction of the oracle's gain will random search capture with 16 evaluations? Will evolution beat random search?

LOOK FOR. Where each curve flattens, the gap between random search and the oracle, and what the shuffled search reports about itself.

THE PROXY. In the curated space the two-epoch proxy has Kendall $\tau = 0.32$ with the truth (Pearson $\rho = 0.45$); one epoch gives 0.20, ten epochs 0.46, and the thirty-epoch validation error 0.56, close to the 0.52 at which two full test runs agree. Its bias shows in its favorite: averaged over its four runs, the two-epoch proxy prefers four layers of 128 tanh units with batch normalization, a fast learner that is 33rd of 96 by true error (2.81%), while the thirty-epoch validation error prefers three layers of 128 GELU units with batch normalization, which is second (2.04%). In the full space the proxies rank better ($\tau = 0.57$ at two epochs), because the narrow networks of the tail are bad early and late.

The search against its controls. In the full space (Figure 5.2) every search with an informative proxy beats the placebo by about two points, because a single random draw lands too often in the tail: with 16 evaluations random search returns architectures with true error 2.73 ± 0.02% (mean and 95% interval over 2000 searches) against the placebo's 4.64%. In the curated space the gain is about half a point, 2.64 ± 0.02% against 3.14%, the size of one run's seed noise, and single searches vary around that mean with a standard deviation of 0.47 points. In both spaces random search levels off near 2.6%, while the oracle goes on to 2.13% at $k = 16$ and 1.98% at $k = 64$ in the curated space (intervals below 0.01 points): the plateau of the biased-proxy heuristic. Evolution, the more capable optimizer, does not beat the sham. Its architectures are worse than those of random search by 0.03 ± 0.03, 0.06 ± 0.02 and 0.09 ± 0.02 points at $k = 16$, 32 and 64 in the curated space (0.02 ± 0.03, 0.06 ± 0.03 and 0.09 ± 0.02 in the full space): it finds the proxy's favorites faster, and they are fast learners. Given a proxy that ranks better, evolution wins by a little: with the ten- and thirty-epoch proxies it is ahead in seven of the eight comparisons we ran, by 0.02 to 0.07 points (with the thirty-epoch proxy and 64 evaluations, 2.14% against 2.21% in the curated space, a difference of 0.07 ± 0.01 points).

The theorem. In the curated space Theorem 5.6 predicts random search well while the budget is small: 2.98%, 2.84%, 2.73% and 2.63% for $k = 2$, 4, 8 and 16, against 2.96%, 2.85%, 2.71% and 2.64% observed. Beyond that the prediction keeps falling (2.46% at $k = 64$) and the observations do not: the smallest proxy values belong to the fast learners, and the straight regression line fails in the tail. In the full space the Gaussian model fails from the start, predicting 1.68% at $k = 8$, below the best architecture, against 2.82% observed: the long tail inflates the standard deviation to 3.18 points while the headroom is 2.76, and Remark 5.3 is the bound that holds. Figure 5.3 varies the proxy instead of the budget. In the curated space the gain of random search with 16 evaluations, divided by the perfect-proxy gain $\sigma_{\mathcal{A}} e_{16}$, follows the line ρ from the shuffled proxy ($\rho = 0$, gain 0.00 ± 0.03 points, a 95% interval over 2000 searches) through the one-epoch proxy ($\rho = 0.28$, 0.39 ± 0.02 points against a prediction of 0.32) to full training ($\rho = 0.71$, 0.82 ± 0.01 against 0.81); in the full space the points lie at 0.33 to 0.42 of $\sigma_{\mathcal{A}} e_{16}$ whatever the proxy.

What the search says about itself. With the thirty-epoch proxy, whose values are validation errors on the scale of the truth, random search with eight evaluations in the curated space reports a gain of 1.64 points over the average proxy score and achieves 0.68: a ratio of 2.39, somewhat more than the $1/\rho^2 = 2.01$ of Corollary 5.7. The negative control needs no model: with the proxy values shuffled among the architectures, random search with 16 evaluations achieves −0.03 ± 0.03 points (95% interval) and reports 2.09 (in the full space, −0.04 ± 0.14 and 3.44). A search's report is no evidence until it is measured against a control.

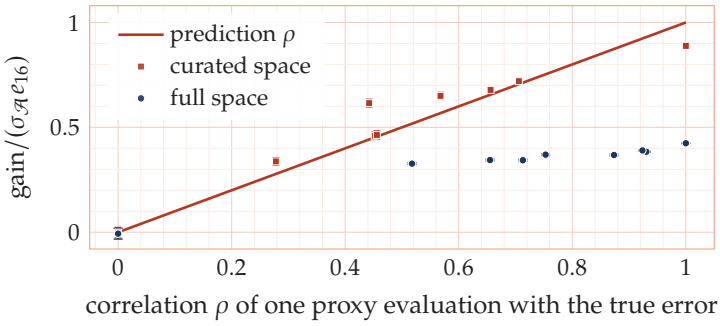


Figure 5.3. The proxy theorem across proxies. Each point is random search with $k = 16$ evaluations on the benchmark of Lab 5.2 (2000 searches, 95% interval): its gain over the placebo divided by $\sigma_{\mathcal{A}} e_{16}$ (1.14 points in the curated space, 5.61 in the full space), against the Pearson correlation of one proxy evaluation with the true error. From left to right: the shuffled proxy, the validation error after 1, 2, 3, 5, 10, 20 and 30 epochs, and the true error itself (the oracle). The line is Theorem 5.6.

Table 5.1. Lab 5.3: test error (%), mean and 95% t-interval; ten runs per architecture and pipeline. The placebo row averages 16 random curated architectures, two runs each, with an interval over the 16.

	old pipeline	new pipeline
textbook MLP (2×64, ReLU)	3.19 ± 0.23	2.00 ± 0.22
found (3×128, GELU, batch norm.)	2.04 ± 0.34	1.69 ± 0.22
placebo (random curated)	2.97 ± 0.29	2.32 ± 0.29

LAB 5.3 · A HISTORICAL CONTROL

SCRIPT
`05-controls-pipeline.py`

SETUP. A 2×2 factorial: the textbook MLP (two layers of 64 ReLU units) and the architecture found in Lab 5.2 (the curated architecture with the lowest validation error after full training), crossed with the old pipeline of Lab 5.1 and a new one (Adam with learning rate $2{\times}10^{-3}$, cosine schedule, 60 epochs, label smoothing 0.1); ten runs per cell, seeds 3000–3009. A placebo arm: 16 random curated architectures, two runs each, under both pipelines. 104 runs, under a minute.

PREDICT FIRST. A paper reports the found architecture with the new pipeline and copies the textbook MLP's number from an older paper that used the old one. How much of the reported improvement belongs to the search?

LOOK FOR. The pipeline effect on each architecture, and the placebo arm under the new pipeline.

THE HISTORICAL COMPARISON. The paper of the lab sheet would report an improvement of 1.50 ± 0.30 points (Table 5.1; 95% Welch intervals for all differences here). The new pipeline alone improves the textbook MLP by 1.19 ± 0.30 points; under the new pipeline the textbook MLP is better than

the average random architecture by $2.32 - 2.00 = 0.32$ points; and the found architecture beats the concurrent placebo by 0.63 ± 0.35 points. These are the three terms of (5.1), $1.19 - 0.32 + 0.63 = 1.50$: the search's own share is less than half of what the historical comparison credits to it. Pipeline and architecture also interact: the new pipeline helps the textbook MLP by 1.19 points but the found architecture by only 0.35 ± 0.38, so the contribution of the architecture depends on the pipeline at which it is measured (1.15 ± 0.38 points under the old one, 0.32 ± 0.29 under the new), the order dependence of **??**. In miniature this is the finding of Yang et al. (2020).

WHAT THE MINIATURE REPRODUCES AND WHAT IT DOES NOT. The benchmark reproduces the mechanism: a proxy that ranks poorly holds every search to a plateau, a better optimizer of a biased proxy does not help, a search's report overstates its gain, and a historical control mixes the pipeline into the search. It does not reproduce the near-zero margin over the placebo: our curated space is relatively wider than the DARTS space (headroom 40% of the mean error, against about 18% for the 214 cells of Yang et al. (2020)), and our short-training proxy ranks better than shared weights did, so random search captures about half of the oracle's gain. Weaker proxies move the gain toward the placebo along the line of Figure 5.3; we did not implement weight sharing.

5.7 Protocol

PROTOCOL 5.1 · CONTROLS FOR A SEARCH METHOD

- ☐ Fix the search space, its sampling distribution, the proxy, the budget and the final pipeline before searching, and use them in every arm.
- ☐ Run a placebo arm: train at least ten architectures drawn from the space with the final pipeline, and report their mean, standard deviation and best, the spread and headroom of the space.
- ☐ Run a sham arm: random search with the same proxy, budget and selection rule.
- ☐ Repeat every search with independent seeds, as often as Remark 5.10 requires for the gain you claim, and report the distribution of what the searches return, not the best search.
- ☐ Score every returned architecture by retraining it with fresh seeds on test data never used for selection; never report the winner's proxy score as its quality.
- ☐ Measure the proxy: report Kendall's τ between the proxy and full training on a sample of architectures, beside the τ between two seeds of full training.
- ☐ Run a positive control (an oracle or a planted better architecture, which must be detected) and a negative control (a shuffled proxy,

which must show no gain).

- ☐ Use published numbers only as context; claim gains only against concurrent arms, with and without training tricks.
- ☐ Release the space, the pipeline, the seeds and every sampled architecture with its scores.

The protocol measures a search within a space, not the space: a search with no placebo-adjusted gain in an excellent space may still be the right product, and a large gain over a poor placebo is worth little, since a designer can always widen a space with bad options, as our full space shows. Transfer is outside its reach: a cell searched on CIFAR-10 and deployed on ImageNet needs controls at the scale of deployment. And it is expensive when a search is: at a GPU-day per search, ten searches in each of three arms cost a month. Then the comparison of strategies belongs on a tabular or surrogate benchmark, and the full experiment can be reduced to the placebo arm, which has no search cost, the proxy correlation, and enough trainings of the final architecture to state its error with an interval.

Exercises

5.1 CONSOLIDATION. (Expected rank of the best of k.) In a space of S architectures ranked by true error, let v_k be the rank of the best of k uniform draws with replacement. (a) Show that $\mathbb{P}(v_k \geq r) = \big((S-r+1)/S\big)^k$ and $\mathbb{E}v_k = \sum_{j=1}^{S}(j/S)^k \leq 1 + S/(k+1)$, and compute $\mathbb{E}v_k$ for $S = 96$ and $k = 16$. (b) Compare with the true rank of what random search returned in Lab 5.2, and explain the difference with Lemma 5.5.

5.2 CONSOLIDATION. (What survives without normality.) (a) Show with Lemma 5.5 that if $\mathbb{E}[T \mid P] = \alpha + \beta P$, then every proxy-based selection among independent pairs has $\mathbb{E}T_J = \mathbb{E}T + \beta(\mathbb{E}P_J - \mathbb{E}P)$. (b) Let $P \sim \mathcal{N}(0,1)$ and $T = P^2$. Show that $\mathrm{Corr}(T,P) = 0$, that for $k \geq 3$ keeping the smallest P_i gives $\mathbb{E}T_J > \mathbb{E}T$ (equality for $k = 2$), and that keeping the smallest $|P_i|$ gives $\mathbb{E}T_J < \mathbb{E}T$. What does this say about summarizing a proxy by one correlation?

5.3 CONSOLIDATION. (The winner's curse in a null space.) In a space whose architectures are all equally good ($\sigma_T = 0$), a search with the additive proxy $P = T + \xi$, $\xi \sim \mathcal{N}(0, s^2)$, keeps the smallest of k proxy values. Show that it reports a gain of $s\,e_k$ and achieves none, compute the report for $s = 0.47$ points and $k = 16$, and relate it to the best-of-k reporting of Chapter 1.

5.4 CONSOLIDATION. ★ (Two seeds and one truth.) In the additive model $P_j = T + \xi_j$ with independent $\xi_1, \xi_2 \sim \mathcal{N}(0, s^2)$, show that $\mathrm{Corr}(P_1, P_2) = \mathrm{Corr}(T, P_1)^2$, and check it on the curated space, where two seeds give $\tau = 0.52$ and the thirty-epoch proxy has $\rho = 0.71$. *(Hint: the proxy is a*

validation error and the truth a six-run test mean, so expect agreement in order of magnitude only.)

5.5 LAB. (Make the proxy better or worse.) (a) In `05-controls-search.py` set `FID_MAIN` to 1, 5 and 30 and record the gains of random search and of evolution over the placebo at $k = 64$ in the curated space. At which fidelity does evolution start to win? Explain with the biased-proxy heuristic. (b) At equal compute, compare 60 two-epoch evaluations with 4 thirty-epoch ones in both spaces. Why does the answer differ between the spaces?

5.6 LAB. (A weaker proxy.) Add a proxy computed from one epoch on half the training images to Figure 5.3. Does it fall on the line?

5.7 LAB. (Widen the space.) Add width 8 and dropout 0.5 to the space of Lab 5.1. Predict, then measure, how the placebo, random search and the oracle move. Which control does the new tail affect, and which arm of a comparison does it flatter?

5.8 DESIGN. (Controls for data pruning.) A method keeps 30% of a training set, and its authors claim that training on the kept subset beats training on a random 30%. Design the placebo, the sham when there is one, a positive control and a negative control, with the number of runs per arm, for a budget of 200 training runs.

5.9 DESIGN. (A search experiment under a budget.) A search costs one GPU-day, a final training half a GPU-day, and you have 40 GPU-days. You expect a gain of 0.3 points over random sampling, with single trainings varying by 0.15 points and returned architectures by 0.2. Allocate the budget among searches, placebo draws and final trainings, and state the power of your comparison.

5.10 CRITIQUE. (A table without a random row.) A paper reports its search method's test error ($2.61 \pm 0.05\%$ over five trainings of its best cell) beside the published errors and search costs of six earlier methods, with no random architectures, and says the method "outperforms all baselines at a fraction of the cost". List what the table cannot show, and the fewest extra runs that would make the claim testable. (The description is invented.)

5.11 CRITIQUE. (A historical baseline.) A paper reports 1.8% error for a new architecture with its pipeline (cosine schedule, 300 epochs, augmentation) against 2.6% "reported for the baseline in its original paper" (step schedule, 90 epochs). Write the decomposition (5.1) for this comparison, say which terms the paper has measured, and design the 2×2 experiment that measures the others. (The description is invented.)

5.12 RESEARCH. (When can a search beat random by more than noise?) Theorem 5.6 describes random search; adaptive searches can exploit locality, the similarity of neighbors in the space, which Ying et al.

(2019) measured in NAS-Bench-101. Open: a model of a space with locality, a proxy with noise and a fixed bias, and an adaptive strategy, in which the advantage over random search with the same proxy can be computed, with conditions under which it exceeds the noise of one final training.

Notes and References

Architecture search grew out of hyperparameter optimization and neuroevolution; the survey of Elsken et al. (2019) organizes it, as this chapter does, by search space, search strategy and performance estimation, our proxy, and Zela et al. (2020) explained and repaired the failures of DARTS.

Lindauer and Hutter (2020) turned the lessons of the reality checks into a checklist, and the tabular benchmarks NAS-Bench-101 (Ying et al. 2019) and NAS-Bench-201 (Dong and Yang 2020) made the controls nearly free. Measuring the space itself is the idea of Radosavovic et al. (2019), who compare design spaces by the error distributions of models sampled from them: the placebo arm promoted from a baseline to an instrument.

The vocabulary of controls is borrowed from clinical trials (International Conference on Harmonisation 2000; Sacks et al. 1982) and epidemiology (Lipsitch et al. 2010), and selection on a noisy measurement is old in quantitative genetics (Falconer and Mackay 1996).

Translating the literature. What Lindauer and Hutter (2020) call random sampling and random search are our placebo and sham arms. The "random policy" of Yu et al. (2020) is our placebo repeated over seeds, and their $p(>\text{random})$ is the probability that a searched architecture beats one random architecture. The relative improvement $100(\text{Acc}_m - \text{Acc}_r)/\text{Acc}_r$ of Yang et al. (2020) is our gain G in accuracy, in percent of the placebo's accuracy. The test regret of Ying et al. (2019) is our $\mu(A_S) - \mu^\star$ in accuracy, and their epoch budget is our proxy fidelity f. In the breeder's equation $R = h^2 S$ of Falconer and Mackay (1996), R is our true gain, S the selection differential of the proxy ($\sigma_P e_k$ for the best of k) and h^2 our ρ^2 in the additive model; their selection intensity i is e_k when one of k is kept.

References

Bergstra, J. and Y. Bengio (2012). "Random Search for Hyper-Parameter Optimization". In: *Journal of Machine Learning Research* 13, pp. 281–305.

Dong, X. and Y. Yang (2020). "NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search". In: *International Conference on Learning Representations*.

Elsken, T., J. H. Metzen, and F. Hutter (2019). "Neural Architecture Search: A Survey". In: *Journal of Machine Learning Research* 20.55, pp. 1–21.

Falconer, D. S. and T. F. C. Mackay (1996). *Introduction to Quantitative Genetics*. 4th ed. Harlow: Longman.

International Conference on Harmonisation (2000). *Choice of Control Group and Related Issues in Clinical Trials (E10)*. ICH harmonised tripartite guideline, Step 4, July 2000.

Kendall, M. G. (1938). "A New Measure of Rank Correlation". In: *Biometrika* 30.1–2, pp. 81–93.

Li, L. and A. Talwalkar (2019). "Random Search and Reproducibility for Neural Architecture Search". In: *Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence (UAI 2019)*. Vol. 115. Proceedings of Machine Learning Research, pp. 367–377.

Lindauer, M. and F. Hutter (2020). "Best Practices for Scientific Research on Neural Architecture Search". In: *Journal of Machine Learning Research* 21.243, pp. 1–18.

Lipsitch, M., E. Tchetgen Tchetgen, and T. Cohen (2010). "Negative Controls: A Tool for Detecting Confounding and Bias in Observational Studies". In: *Epidemiology* 21.3, pp. 383–388.

Liu, H., K. Simonyan, and Y. Yang (2019). "DARTS: Differentiable Architecture Search". In: *International Conference on Learning Representations*.

Lucic, M., K. Kurach, M. Michalski, S. Gelly, and O. Bousquet (2018). "Are GANs Created Equal? A Large-Scale Study". In: *Advances in Neural Information Processing Systems 31*.

Melis, G., C. Dyer, and P. Blunsom (2018). "On the State of the Art of Evaluation in Neural Language Models". In: *International Conference on Learning Representations*.

Musgrave, K., S. Belongie, and S.-N. Lim (2020). "A Metric Learning Reality Check". In: *Computer Vision – ECCV 2020*. Vol. 12370. Lecture Notes in Computer Science. Springer, pp. 681–699.

Pham, H., M. Y. Guan, B. Zoph, Q. V. Le, and J. Dean (2018). "Efficient Neural Architecture Search via Parameter Sharing". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research, pp. 4095–4104.

Radosavovic, I., J. Johnson, S. Xie, W.-Y. Lo, and P. Dollár (2019). "On Network Design Spaces for Visual Recognition". In: *IEEE/CVF International Conference on Computer Vision*, pp. 1882–1890.

Real, E., A. Aggarwal, Y. Huang, and Q. V. Le (2019). "Regularized Evolution for Image Classifier Architecture Search". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33, pp. 4780–4789.

Sacks, H., T. C. Chalmers, and H. Smith (1982). "Randomized versus Historical Controls for Clinical Trials". In: *The American Journal of Medicine* 72.2, pp. 233–240.

Yang, A., P. M. Esperança, and F. M. Carlucci (2020). "NAS Evaluation is Frustratingly Hard". In: *International Conference on Learning Representations*.

Ying, C., A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter (2019). "NAS-Bench-101: Towards Reproducible Neural Architecture Search". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 7105–7114.

Yu, K., C. Sciuto, M. Jaggi, C. Musat, and M. Salzmann (2020). "Evaluating the Search Phase of Neural Architecture Search". In: *International Conference on Learning Representations*.

Zela, A., T. Elsken, T. Saikia, Y. Marrakchi, T. Brox, and F. Hutter (2020). "Understanding and Robustifying Differentiable Architecture Search". In: *International Conference on Learning Representations*.

Zoph, B. and Q. V. Le (2017). "Neural Architecture Search with Reinforcement Learning". In: *International Conference on Learning Representations*.

Zoph, B., V. Vasudevan, J. Shlens, and Q. V. Le (2018). "Learning Transferable Architectures for Scalable Image Recognition". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8697–8710.

Part III

Causes

Knowing that a component helps is not knowing why. An ablation removes a part and watches what changes, a mechanism claim names what the part does, and a correlation across many runs suggests a cause without isolating it. Each has a design that makes it informative and a way of going wrong.

Part IV

Scale

Scaling laws turned the planning of large training runs into an exercise in fitting and extrapolation. A fitted curve is a measurement with a protocol, an uncertainty and a domain of validity, and each of the three has been the source of a famous error.

Part V

Measurement

A measurement is a number produced by an instrument. Before a curve can be read as a fact about models, the reader has to know what the instrument does to it: how the metric transforms the quantity beneath it, how many items it rests on, and how its settings move it.

A Statistical Toolkit

A

The statistics the experiments share

IN THIS CHAPTER. A handful of statistical tools recur across the chapters of this book: intervals for a mean and for a difference of means; the bootstrap and the permutation test; paired comparisons of two classifiers on the same test items; corrections for many comparisons; the distribution of the best of several draws; the delta method; nonlinear least squares; variance components and factorial designs; sample sizes; rank statistics; and intervals for proportions near zero. This appendix states what the chapters use and proves what can be proved in a page; for the rest it gives a sketch and a reference.

The appendix is a reference, not a chapter to be read in order. Intervals and tests for means (Section A.1) are used everywhere; resampling (Section A.2) in Chapters 1, 3 and 4 and **??**; multiple comparisons (Section A.3) in Chapter 3; order statistics (Section A.4) in Chapters 1, 4 and 5; the delta method and nonlinear least squares (Sections A.5 and A.6) in **??????**; variance components and factorial designs (Section A.7) in Chapter 1 and **??????**; sample sizes (Section A.8) in Chapter 1; rank statistics (Section A.9) in Chapters 3 and 5 and **??**; and binomial intervals (Section A.10) in **??**.

Throughout, $Y_1, \dots, Y_n$ are outcomes of runs, $\bar{Y}$ is their mean and $S^2 = \frac{1}{n-1}\sum_i (Y_i - \bar{Y})^2$ their sample variance. A p-value for a true null hypothesis is *valid* if $\mathbb{P}(p \le u) \le u$ for every $u \in [0,1]$.

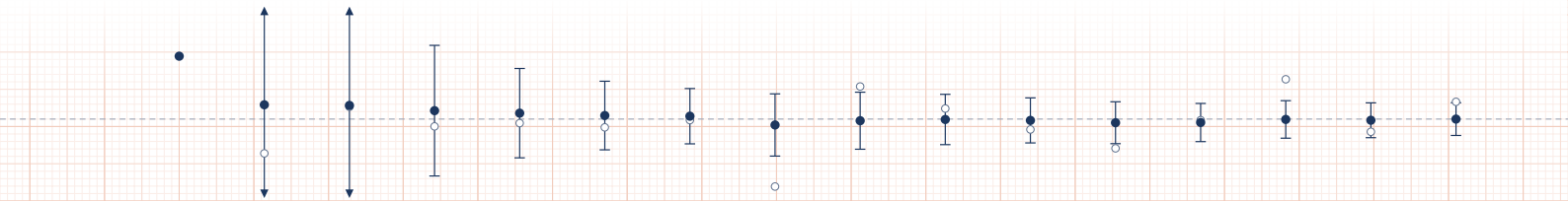
A.1 Intervals for means

If $Y_1, \dots, Y_n$ are independent with mean μ and variance σ^2, then $\mathbb{E}\bar{Y} = \mu$, $\operatorname{Var}\bar{Y} = \sigma^2/n$ and $\mathbb{E}S^2 = \sigma^2$, so that $S/\sqrt{n}$, the *standard error*, estimates the standard deviation of $\bar{Y}$.

Theorem A.1 (t-intervals and Welch's test). *(i) If the Y_i are normal, then $(\bar{Y} - \mu)/(S/\sqrt{n})$ has Student's t distribution with $n-1$ degrees of freedom, and the interval $\bar{Y} \pm t_{n-1,1-\alpha/2}\, S/\sqrt{n}$ contains μ with probability exactly $1-\alpha$.*

(ii) Without normality, if $\mathbb{E}Y_1^2 < \infty$, the coverage of the same interval tends to $1-\alpha$ as $n \to \infty$.

(iii) For two independent samples of sizes n_1, n_2, means μ_1, μ_2 and variances



σ_1^2, σ_2^2, *Welch's statistic*

$$W = \frac{\bar{Y}_1 - \bar{Y}_2 - (\mu_1 - \mu_2)}{\sqrt{S_1^2/n_1 + S_2^2/n_2}}$$

is asymptotically standard normal as $n_1, n_2 \to \infty$, *and for normal samples it is approximately* t*-distributed with* $\nu = \left(S_1^2/n_1 + S_2^2/n_2\right)^2 \big/ \left[(S_1^2/n_1)^2/(n_1-1) + (S_2^2/n_2)^2/(n_2-1)\right]$ *degrees of freedom.*

Proof sketch. Part (i) rests on two facts about normal samples: $\bar{Y}$ and S^2 are independent, and $(n-1)S^2/\sigma^2$ has the χ^2 distribution with $n-1$ degrees of freedom (Casella and Berger 2002, Section 5.3). A standard normal divided by the square root of an independent χ^2_ν/ν has, by definition, Student's distribution with ν degrees of freedom (Student 1908). Part (ii) follows from the central limit theorem for $\sqrt{n}(\bar{Y} - \mu)/\sigma$, the convergence of S to σ in probability, Slutsky's lemma, and $t_{n-1,1-\alpha/2} \to z_{1-\alpha/2}$. Part (iii) is proved the same way for the asymptotic statement; the degrees of freedom come from matching the first two moments of $S_1^2/n_1 + S_2^2/n_2$ to those of a scaled χ^2 variable (Satterthwaite 1946; Welch 1947), which makes the t distribution an approximation rather than an exact law. ■

When the runs of two arms come in pairs that share a seed, a data order or a set of test items, the comparison should use the differences $D_i = Y_{1i} - Y_{2i}$ and the one-sample interval of part (i) for $\mathbb{E}D$. Since $\operatorname{Var} D = \sigma_1^2 + \sigma_2^2 - 2\operatorname{Cov}(Y_1, Y_2)$, pairing narrows the interval whenever the paired outcomes are positively correlated, which is the usual case.

A.2 Resampling

Definition A.2 (Bootstrap intervals). Let $\hat{\theta} = \hat{\theta}(Y_1, \ldots, Y_n)$ be a statistic. A *bootstrap sample* draws n values uniformly with replacement from $Y_1, \ldots, Y_n$; computing $\hat{\theta}$ on B independent bootstrap samples gives $\hat{\theta}_1^*, \ldots, \hat{\theta}_B^*$. The *percentile interval* at level $1-\alpha$ runs from the $\alpha/2$-quantile to the $(1-\alpha/2)$-quantile of the $\hat{\theta}_b^*$. When the data fall into strata, for instance tasks with several runs each, the *stratified bootstrap* resamples with replacement within each stratum and keeps the strata fixed.

Theorem A.3 (Consistency of the bootstrap). *Let* $Y_1, \ldots, Y_n$ *be independent and identically distributed with finite variance, and let* $\hat{\theta} = g(\bar{Y})$ *for a function* g *continuously differentiable at* μ *with* $g'(\mu) \neq 0$. *Then, conditionally on the data and for almost every sequence of data,* $\sqrt{n}(\hat{\theta}^* - \hat{\theta})$ *has the same normal limit as* $\sqrt{n}(\hat{\theta} - g(\mu))$, *and the percentile interval has asymptotic coverage* $1-\alpha$.

Proof sketch. Bickel and Freedman (1981) prove the conditional central limit theorem for the bootstrap mean; the delta method (Theorem A.10) carries it to $g(\bar{Y})$; the symmetry of the normal limit then gives the coverage of the percentile interval (Vaart 1998, Chapter 23). With few runs the percentile interval tends to be too narrow; the bias-corrected and accelerated interval of Efron (1987) corrects for bias and skewness, and Efron and Tibshirani (1993) is the standard introduction. ∎

A permutation test needs no asymptotics. Its null hypothesis is a symmetry: for two arms of runs, that the arm labels are exchangeable, as they are when the two arms have the same run distribution and the runs are independent.

Theorem A.4 (Permutation tests are exact). *Let G be a finite group acting on the data, and suppose that under the null hypothesis gY has the same distribution as Y for every $g \in G$. For a statistic T put $p(Y) = |G|^{-1}\,\#\{h \in G : T(hY) \geq T(Y)\}$. Then, under the null hypothesis, $\mathbb{P}(p(Y) \leq u) \leq u$ for every $u \in [0,1]$.*

Proof. The plan is to show that in every orbit of the group at most a fraction u of the points have a p-value at most u, and then to use the symmetry to turn this count into a probability.

Fix a data set y and let $A = \{g \in G : p(gy) \leq u\}$. Since $h \mapsto hg$ is a bijection of G, the p-value of gy is computed from the same values as that of y:

$$p(gy) = |G|^{-1}\,\#\{h \in G : T(hgy) \geq T(gy)\} = |G|^{-1}\,\#\{h \in G : T(hy) \geq T(gy)\}.$$

If A is empty there is nothing to prove. Otherwise choose $g^\star \in A$ for which $T(g^\star y)$ is smallest among the elements of A. Every $g \in A$ satisfies $T(gy) \geq T(g^\star y)$, so A is contained in $\{h : T(hy) \geq T(g^\star y)\}$, a set with $|G|\,p(g^\star y) \leq u|G|$ elements. Hence $|A| \leq u|G|$ for every y.

Now let the data be random. By the null hypothesis $p(gY)$ has the distribution of $p(Y)$ for each g, so

$$\mathbb{P}(p(Y) \leq u) = \frac{1}{|G|}\sum_{g \in G} \mathbb{P}(p(gY) \leq u) = \mathbb{E}\Big[\frac{\#\{g : p(gY) \leq u\}}{|G|}\Big] \leq u. \qquad \blacksquare$$

For two arms of n runs, G is the group of permutations of the $2n$ pooled outcomes, acting by reassigning arm labels, and T is for instance the difference of the arm means. When $|G|$ is too large to enumerate, M random permutations give the valid p-value $(1 + \#\{b : T_b \geq T_{\mathrm{obs}}\})/(M+1)$; dropping the two ones, as is often done, can produce a p-value of zero and is not valid (Phipson and Smyth 2010).

Two classifiers evaluated on the same test items are a paired design. Its analysis goes back to McNemar (1947); Dietterich (1998) recommended McNemar's test for comparing two learning algorithms when each can be trained only once.

Proposition A.5 (Two classifiers on the same items). *Two classifiers are evaluated on m test items drawn independently. Let b count the items that the first classifies correctly and the second incorrectly, and c the items with the reverse, and let π_b, π_c be the probabilities of these two discordances.*

(i) If $\pi_b = \pi_c$, then conditionally on $b + c = s$ the count b has the binomial distribution $\mathrm{Bin}(s, 1/2)$.

(ii) The difference of the two accuracies is $(b - c)/m$, with variance $\big(\pi_b + \pi_c - (\pi_b - \pi_c)^2\big)/m$.

Proof. Each item falls, independently of the others, into one of four categories: both correct, only the first correct, only the second correct, both wrong. Conditionally on which items are discordant, each discordant item is of the first kind with probability $\pi_b/(\pi_b + \pi_c)$, independently of the others; under $\pi_b = \pi_c$ this probability is $1/2$, which proves (i). For (ii), write the difference of accuracies as $\frac{1}{m}\sum_i X_i$ with $X_i = 1$ for a discordance of the first kind, -1 for one of the second kind and 0 otherwise. Then $\mathbb{E}X_i = \pi_b - \pi_c$ and $\mathbb{E}X_i^2 = \pi_b + \pi_c$, so $\mathrm{Var}\, X_i = \pi_b + \pi_c - (\pi_b - \pi_c)^2$, and the mean of m independent copies has $1/m$ of this variance. ∎

Two classifiers of similar accuracy usually agree on most items, so $\pi_b + \pi_c$ is small and the paired variance in (ii) is far below the unpaired $\big(p_1(1 - p_1) + p_2(1 - p_2)\big)/m$.

A.3 Many comparisons

Let $H_1, \dots, H_m$ be null hypotheses with valid p-values $p_1, \dots, p_m$, let I_0 be the set of true nulls and $m_0 = |I_0|$. A procedure rejects some of the hypotheses; let R be the number of rejections and V the number of true nulls among them. The *family-wise error rate* is $\mathbb{P}(V \geq 1)$, and the *false discovery rate* is $\mathbb{E}[V/\max(R, 1)]$.

Proposition A.6 (Bonferroni). *Rejecting H_i when $p_i \leq \alpha/m$ gives a family-wise error rate at most $m_0\alpha/m \leq \alpha$, whatever the dependence between the p-values.*

Proof. By the union bound and validity, $\mathbb{P}(V \geq 1) \leq \sum_{i \in I_0} \mathbb{P}(p_i \leq \alpha/m) \leq m_0\alpha/m$. ∎

Theorem A.7 (Holm's procedure). *Order the p-values as $p_{(1)} \leq \cdots \leq p_{(m)}$. Let j be the smallest index with $p_{(j)} > \alpha/(m - j + 1)$, reject the hypotheses with the $j - 1$ smallest p-values, and reject all of them if there is no such index. Then the family-wise error rate is at most α, whatever the dependence between the p-values.*

Proof. If $m_0 = 0$ there is no true null to reject. Otherwise consider the event E that every true null has $p_i > \alpha/m_0$; by the union bound its probability is at least $1 - m_0 \cdot \alpha/m_0 = 1 - \alpha$. On E, let j_0 be the first position in the

ordering that holds a true null. The positions before it hold false nulls only, so $j_0 \le m - m_0 + 1$, that is $m - j_0 + 1 \ge m_0$, and therefore

$$p_{(j_0)} > \alpha/m_0 \ge \alpha/(m - j_0 + 1).$$

The procedure stops at or before position j_0 and rejects no true null. The family-wise error rate is at most $\mathbb{P}(E^c) \le \alpha$. ∎

Theorem A.8 (Benjamini and Hochberg). *Let $R = \max\{k \ge 1 : p_{(k)} \le \alpha k/m\}$, or $R = 0$ if no such k exists, and reject the R hypotheses with the smallest p-values. If each true-null p-value is valid and independent of all the other p-values, then the false discovery rate is at most $m_0\alpha/m \le \alpha$.*

Proof. The plan is to write the false discovery rate as a sum over true nulls, and to decouple each true null's p-value from the number of rejections by setting it to zero.

Write $N(k) = \#\{j : p_j \le \alpha k/m\}$. Since $p_{(k)} \le \alpha k/m$ exactly when $N(k) \ge k$, we have $R = \max\{k : N(k) \ge k\}$. A hypothesis H_i is rejected exactly when $p_i \le \alpha R/m$: the rejected p-values are at most $p_{(R)} \le \alpha R/m$, and a p-value $p_i \le \alpha R/m$ outside the R smallest would give $p_{(R+1)} \le \alpha(R+1)/m$, against the maximality of R. Hence

$$\mathbb{E}\Big[\frac{V}{\max(R,1)}\Big] = \sum_{i \in I_0} \sum_{k=1}^{m} \frac{1}{k}\, \mathbb{P}\big(p_i \le \alpha k/m,\ R = k\big). \tag{A.1}$$

Fix $i \in I_0$ and let $R^{(i)}$ and $N^{(i)}$ be computed with p_i replaced by 0; then $N^{(i)}(k) = N(k) + \mathbf{1}\{p_i > \alpha k/m\}$, and $R^{(i)} \ge 1$ always. We claim that on the event $\{p_i \le \alpha k/m\}$ the equalities $R = k$ and $R^{(i)} = k$ are equivalent. On that event $N^{(i)}(k') = N(k')$ for every $k' \ge k$. If $R = k$, then $N^{(i)}(k) = N(k) \ge k$, and for $k' > k$, $N^{(i)}(k') = N(k') < k'$, so $R^{(i)} = k$. If $R^{(i)} = k$, then $N(k) = N^{(i)}(k) \ge k$, and for $k' > k$, $N(k') \le N^{(i)}(k') < k'$, so $R = k$.

The variable $R^{(i)}$ depends only on the p-values other than p_i, which are independent of p_i. By the claim and validity,

$$\mathbb{P}\big(p_i \le \alpha k/m,\ R = k\big) = \mathbb{P}\big(p_i \le \alpha k/m\big)\, \mathbb{P}\big(R^{(i)} = k\big) \le \frac{\alpha k}{m}\, \mathbb{P}\big(R^{(i)} = k\big).$$

Substituting into (A.1), the inner sum is at most $\frac{\alpha}{m} \sum_k \mathbb{P}(R^{(i)} = k) = \alpha/m$, and the outer sum over the m_0 true nulls gives $m_0\alpha/m$. ∎

Under arbitrary dependence the same procedure run at level $\alpha / \sum_{k=1}^{m} k^{-1}$ controls the false discovery rate at α (Benjamini and Yekutieli 2001). The original papers are Holm (1979) and Benjamini and Hochberg (1995).

A.4 The best of several draws

Proposition A.9 (Order statistics). *Let $Y_1, \dots, Y_k$ be independent with distribution function F and $\mathbb{E}|Y_1| < \infty$, and let $M_k = \max_i Y_i$.*

(i) $\mathbb{P}(M_k \le x) = F(x)^k$; when F is continuous, the q-quantile of M_k is

the $q^{1/k}$-quantile of F.

(ii) $\mathbb{E}M_k = \int_0^\infty \bigl(1 - F(x)^k\bigr)\,\mathrm{d}x - \int_{-\infty}^0 F(x)^k\,\mathrm{d}x$.

(iii) For standard normal draws, $e_k = \mathbb{E}M_k$ satisfies $e_k \le \sqrt{2\ln k}$ for every $k \ge 1$.

Proof. Part (i) is independence: $\{M_k \le x\}$ is the intersection of the k events $\{Y_i \le x\}$. If F is continuous, then $F(x)^k = q$ exactly when $F(x) = q^{1/k}$. Part (ii) is the identity $\mathbb{E}Z = \int_0^\infty \mathbb{P}(Z > x)\,\mathrm{d}x - \int_{-\infty}^0 \mathbb{P}(Z \le x)\,\mathrm{d}x$, valid for every integrable Z, applied to $Z = M_k$, which is integrable because $|M_k| \le \sum_i |Y_i|$. For (iii) the case $k = 1$ is $e_1 = 0$. For $k \ge 2$ and $t > 0$, Jensen's inequality and the bound of a maximum by a sum give

$$e^{te_k} \le \mathbb{E}e^{tM_k} \le \sum_{i=1}^{k} \mathbb{E}e^{tY_i} = k\,e^{t^2/2},$$

so $e_k \le (\ln k)/t + t/2$; the choice $t = \sqrt{2\ln k}$ gives $e_k \le \sqrt{2\ln k}$. ∎

The bound is tight to leading order: $e_k/\sqrt{2\ln k} \to 1$, and after centering by b_k and scaling by $a_k = 1/\sqrt{2\ln k}$ the maximum of k standard normals converges to the Gumbel distribution $\exp(-e^{-x})$, with $b_k = \sqrt{2\ln k} - (\ln\ln k + \ln 4\pi)/(2\sqrt{2\ln k})$ (Leadbetter et al. 1983, Theorem 1.5.3). The convergence is slow, and for the small k of practice the exact values of e_k should be computed from (ii) by numerical integration; David and Nagaraja (2003) treat order statistics in general.

A.5 The delta method

Theorem A.10 (Delta method). *Let $\hat\theta_n$ be estimators of $\theta \in \mathbb{R}^p$ with $\sqrt{n}(\hat\theta_n - \theta)$ converging in distribution to $\mathcal{N}(0, \Sigma)$, and let $g : \mathbb{R}^p \to \mathbb{R}$ be differentiable at θ. Then $\sqrt{n}\bigl(g(\hat\theta_n) - g(\theta)\bigr)$ converges in distribution to $\mathcal{N}\bigl(0, \nabla g(\theta)^{\top} \Sigma\, \nabla g(\theta)\bigr)$.*

Proof. The plan is a first-order expansion whose remainder the factor $\sqrt{n}$ cannot rescue. Differentiability at θ means that $g(\theta + h) = g(\theta) + \nabla g(\theta)^{\top} h + \|h\|\,\rho(h)$ for a function ρ with $\rho(0) = 0$ that is continuous at 0. With $h_n = \hat\theta_n - \theta$,

$$\sqrt{n}\bigl(g(\hat\theta_n) - g(\theta)\bigr) = \nabla g(\theta)^{\top} \sqrt{n}\, h_n + \|\sqrt{n}\, h_n\|\,\rho(h_n).$$

The first term converges in distribution to $\mathcal{N}(0, \nabla g^{\top}\Sigma \nabla g)$ by the continuous mapping theorem. In the second, $\|\sqrt{n}h_n\|$ is bounded in probability because $\sqrt{n}h_n$ converges in distribution, and $\rho(h_n) \to 0$ in probability because $h_n = (\sqrt{n}h_n)/\sqrt{n} \to 0$ in probability and ρ is continuous at 0. Their product tends to 0 in probability, and Slutsky's lemma completes the proof. ∎

For example, the compute-optimal allocation exponent of **??** is $a = \alpha_D/(\alpha_N + \alpha_D)$, with gradient $(-\alpha_D, \alpha_N)/(\alpha_N + \alpha_D)^2$ in (α_N, α_D). If the fitted exponents have covariance Σ, the delta method gives

$$\operatorname{Var} \hat{a} \approx \frac{\alpha_D^2 \Sigma_{NN} - 2\alpha_N \alpha_D \Sigma_{ND} + \alpha_N^2 \Sigma_{DD}}{(\alpha_N + \alpha_D)^4},$$

so a positive correlation between the two fitted exponents makes $\hat{a}$ more precise than either exponent alone.

A.6 Nonlinear least squares

In a nonlinear regression $y_i = f(x_i; \beta) + \varepsilon_i$, $i = 1, \dots, n$, with parameters $\beta \in \mathbb{R}^p$ and independent errors of mean zero and variance σ^2, the least-squares estimate $\hat{\beta}$ minimizes $\sum_i (y_i - f(x_i;\beta))^2$. Let J be the $n \times p$ Jacobian with entries $\partial f(x_i;\beta)/\partial \beta_j$.

> **Theorem A.11** (Asymptotics of nonlinear least squares). *Suppose that β is identifiable, that f is twice continuously differentiable in β with suitably bounded derivatives, and that $J^\top J/n$ converges to a positive definite matrix V. Then $\hat{\beta}$ is consistent and $\sqrt{n}(\hat{\beta} - \beta)$ converges in distribution to $\mathcal{N}(0, \sigma^2 V^{-1})$. In practice the covariance of $\hat{\beta}$ is estimated by $\hat{\sigma}^2 (\hat{J}^\top \hat{J})^{-1}$, with $\hat{\sigma}^2$ the residual sum of squares divided by $n - p$ and $\hat{J}$ the Jacobian at $\hat{\beta}$.*

Proof sketch. Expanding the normal equations $\hat{J}^\top (y - f(\hat{\beta})) = 0$ to first order around β gives $\hat{\beta} - \beta \approx (J^\top J)^{-1} J^\top \varepsilon$, a linear function of the errors to which the central limit theorem applies; the regularity conditions control the remainder. Jennrich (1969) gives the proof, and Seber and Wild (1989) the practice. ∎

Three practical consequences are used in **??**. First, if $J^\top J$ is nearly singular, some combination of parameters is barely identified and its variance is large; the condition number of $J^\top J$ measures this. Second, the objective is not convex, so a fit should be started from a grid of initial values and checked for convergence. Third, with a robust loss such as Huber's (Huber 1964), which is quadratic for small residuals and linear for large ones, the covariance becomes the sandwich $A^{-1} B A^{-1}$, with A the expected Hessian of the loss and B the covariance of its gradient (Vaart 1998, Chapter 5), and the formula above no longer applies. When in doubt, bootstrap the runs (Section A.2).

A.7 Variance components and designed experiments

In the *one-way random-effects model*, $Y_{ij} = \mu + a_i + e_{ij}$ for I groups and J runs per group, with all a_i and e_{ij} independent, the a_i of mean zero and variance σ_a^2, and the e_{ij} of mean zero and variance σ_e^2. A group may be an initialization seed with J data orders, or a hardware configuration with J

seeds. The mean squares are

$$\mathrm{MS}_A = \frac{J}{I-1}\sum_{i=1}^{I}(\bar{Y}_{i\cdot}-\bar{Y}_{\cdot\cdot})^2, \qquad \mathrm{MS}_E = \frac{1}{I(J-1)}\sum_{i=1}^{I}\sum_{j=1}^{J}(Y_{ij}-\bar{Y}_{i\cdot})^2.$$

Theorem A.12 (Variance components). *In the one-way random-effects model with $I, J \geq 2$, $\mathbb{E}\,\mathrm{MS}_E = \sigma_e^2$ and $\mathbb{E}\,\mathrm{MS}_A = \sigma_e^2 + J\sigma_a^2$. Hence $\hat{\sigma}_e^2 = \mathrm{MS}_E$ and $\hat{\sigma}_a^2 = (\mathrm{MS}_A - \mathrm{MS}_E)/J$ are unbiased.*

Proof. Both statements follow from one fact: for independent $Z_1, \ldots, Z_r$ with common variance v, $\mathbb{E}\sum_l (Z_l - \bar{Z})^2 = (r-1)v$. Within group i, $Y_{ij} - \bar{Y}_{i\cdot} = e_{ij} - \bar{e}_{i\cdot}$, so the inner sum in MS_E is the sum of squared deviations of J independent errors and has expectation $(J-1)\sigma_e^2$; summing over the I groups and dividing by $I(J-1)$ gives σ_e^2. The group means $\bar{Y}_{i\cdot} = \mu + a_i + \bar{e}_{i\cdot}$ are independent with common variance $\sigma_a^2 + \sigma_e^2/J$, so $\mathbb{E}\sum_i (\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot})^2 = (I-1)(\sigma_a^2 + \sigma_e^2/J)$, and multiplying by $J/(I-1)$ gives $J\sigma_a^2 + \sigma_e^2$. ∎

The estimate $\hat{\sigma}_a^2$ can be negative when the true component is small; it is then reported as zero, or the components are estimated by restricted maximum likelihood (Searle et al. 1992). The crossed two-way design used for seeds in Chapter 1 is analyzed in the same way.

In a *two-level factorial design*, each of k factors is set to a low level, coded -1, or a high level, coded $+1$, every one of the 2^k combinations is run r times, and $N = r2^k$. For a nonempty set S of factors and a run with levels $x_1, \ldots, x_k$, put $c_S = \prod_{j \in S} x_j$. The *effect* of S is estimated by the mean outcome over the runs with $c_S = +1$ minus the mean over the runs with $c_S = -1$: a main effect when S has one factor, an interaction when it has more (Box et al. 2005).

Proposition A.13 (Factorial contrasts). *If the runs of a full two-level factorial design are independent with common variance σ^2, the $2^k - 1$ effect estimates are uncorrelated and each has variance $4\sigma^2/N$.*

Proof. Each sign c_S takes the value $+1$ on half of the runs, so the estimate of the effect of S is $\hat{\theta}_S = \frac{2}{N}\sum_n c_S(n) Y_n$, where n runs over all runs. Its variance is $\frac{4}{N^2}\sum_n c_S(n)^2\sigma^2 = 4\sigma^2/N$. For two different sets S and T, $c_S c_T = c_{S \triangle T}$ because $x_j^2 = 1$, and the symmetric difference $S \triangle T$ is nonempty; in a full factorial design every nonempty sign takes each value on exactly half of the runs, so

$$\mathrm{Cov}(\hat{\theta}_S, \hat{\theta}_T) = \frac{4\sigma^2}{N^2}\sum_n c_{S\triangle T}(n) = 0.$$ ∎

Two further principles of design, due to Fisher (1935), complete the toolkit. *Randomize* the order and placement of runs (which machine, which time, which batch of the queue), so that drifts and hidden nuisance variables cannot line up with the treatment. *Block* on known nuisance variables, comparing treatments within blocks: running both arms with the same seeds, or evaluating both on the same test items, is blocking, and its analysis is the paired one of Section A.1. Montgomery (2017) is a standard textbook.

A.8 Sample sizes and power

Proposition A.14 (Runs needed for a comparison). *Two arms have independent normal outcomes with common known standard deviation σ and n runs each. The two-sided level-α test that rejects $\Delta = 0$ when $|Z| > z_{1-\alpha/2}$, with $Z = (\bar{Y}_1 - \bar{Y}_2)/(\sigma\sqrt{2/n})$, has power*

$$\Phi(\delta - z_{1-\alpha/2}) + \Phi(-\delta - z_{1-\alpha/2}), \qquad \delta = |\Delta|\sqrt{n/2}/\sigma,$$

at the effect Δ. In particular, power at least π is guaranteed by $n \geq 2\sigma^2(z_{1-\alpha/2} + z_\pi)^2/\Delta^2$.

Proof. The statistic Z is normal with variance 1 and mean $\Delta\sqrt{n/2}/\sigma$, so $\mathbb{P}(|Z| > z_{1-\alpha/2}) = \Phi(\delta - z_{1-\alpha/2}) + \Phi(-\delta - z_{1-\alpha/2})$, an expression that depends on Δ only through δ. The second term is nonnegative, so the power is at least $\Phi(\delta - z_{1-\alpha/2})$, which is at least π when $\delta \geq z_{1-\alpha/2} + z_\pi$; solving for n gives the stated bound. ∎

For example, to detect a gain of $\Delta = 0.3$ points with a run-to-run standard deviation of $\sigma = 0.2$ points at level 0.05 with power 0.8 ($z_{0.975} \approx 1.960$, $z_{0.8} \approx 0.842$), $n \geq 2 \cdot 0.04 \cdot 2.802^2/0.09 \approx 6.98$, so seven runs per arm suffice. With σ estimated from the runs, the t quantiles replace the normal ones and the answer rises slightly; with paired runs, $2\sigma^2$ is replaced by $\mathrm{Var}(Y_1 - Y_2)$. Cohen (1988) is the classic reference on power.

A.9 Rank statistics

For n pairs (X_i, Y_i) without ties, a pair of indices $i < j$ is *concordant* if $(X_i - X_j)(Y_i - Y_j) > 0$ and discordant otherwise. With C concordant and D discordant pairs, *Kendall's* τ is $\hat{\tau} = (C - D)/\binom{n}{2} \in [-1, 1]$. It depends on the data only through their ranks, so it is unchanged by monotone transformations of either variable.

Proposition A.15 (Kendall's τ under independence). *If X and Y are independent with continuous distributions, then $\mathbb{E}\hat{\tau} = 0$ and $\mathrm{Var}\,\hat{\tau} = 2(2n + 5)/(9n(n - 1))$.*

Proof sketch. For $i \neq j$ the differences $X_i - X_j$ and $Y_i - Y_j$ are independent, continuous and symmetric about zero, so each pair is concordant with probability $1/2$ and $\mathbb{E}\hat{\tau} = 0$. The variance is a computation with the dependence between pairs that share an index; see Kendall (1938) and Kendall (1970). ∎

When the runs of a sweep fall into strata, a correlation computed within each stratum and then averaged, the *granulated* coefficient of Jiang et al. (2020), removes the part of the pooled correlation that the stratifying variable creates (**??**).

The *probability of improvement* of one arm over another is $\theta = \mathbb{P}(X > Y) + \frac{1}{2}\mathbb{P}(X = Y)$ for independent outcomes X and Y of the two arms.

Proposition A.16 (Mann–Whitney estimate). *For independent samples $X_1, \dots, X_{n_1}$ and $Y_1, \dots, Y_{n_2}$ of the two arms,*

$$\hat{\theta} = \frac{1}{n_1 n_2} \sum_{i=1}^{n_1} \sum_{j=1}^{n_2} \Big(\mathbf{1}\{X_i > Y_j\} + \tfrac{1}{2}\mathbf{1}\{X_i = Y_j\} \Big)$$

is unbiased for θ. If the two arms have the same continuous distribution, then $\theta = 1/2$ and $\operatorname{Var} \hat{\theta} = (n_1 + n_2 + 1)/(12 n_1 n_2)$.

Proof. Each pair (X_i, Y_j) has the distribution of (X, Y), so each summand has expectation θ, and so has their average. Under a common continuous distribution, X and Y are exchangeable and tie with probability zero, so $\theta = 1/2$. The variance is that of the Mann–Whitney statistic $U = n_1 n_2 \hat{\theta}$, computed by Mann and Whitney (1947). ∎

A.10 Proportions near zero

Let $X \sim \operatorname{Bin}(m, p)$ count the successes on m independent test items. For $x < m$ let $p_U(x)$ be the solution q of $\mathbb{P}_q(X \le x) = \alpha/2$, and put $p_U(m) = 1$; for $x > 0$ let $p_L(x)$ be the solution q of $\mathbb{P}_q(X \ge x) = \alpha/2$, and put $p_L(0) = 0$.

Proposition A.17 (Binomial intervals near zero). *(i) The interval $[p_L(X), p_U(X)]$ of Clopper and Pearson (1934) contains p with probability at least $1 - \alpha$, for every p.*

(ii) If $X = 0$, then the one-sided upper confidence bound at level $1 - \alpha$, which is the solution q of $(1 - q)^m = \alpha$, equals $1 - \alpha^{1/m}$; it is at most $\ln(1/\alpha)/m$, about $3/m$ at $\alpha = 0.05$.

Proof. For (i), fix p. For $x < m$ the probability $\mathbb{P}_q(X \le x)$ is continuous and strictly decreasing in q, so $p_U(x) < p$ holds exactly when $\mathbb{P}_p(X \le x) < \alpha/2$. Let $x^\star$ be the largest x with $\mathbb{P}_p(X \le x) < \alpha/2$, or -1 if there is none; the cumulative probabilities increase in x, so the event $\{p_U(X) < p\}$ is contained in $\{X \le x^\star\}$, whose probability is less than $\alpha/2$. The same argument with $\mathbb{P}_q(X \ge x)$, which increases in q for $x > 0$, gives $\mathbb{P}_p(p_L(X) > p) < \alpha/2$, and the union bound gives the coverage. For (ii), $(1 - q)^m$ decreases from 1 to 0 on $[0, 1]$ and equals α at $q = 1 - \alpha^{1/m} = 1 - e^{-\ln(1/\alpha)/m}$, which is at most $\ln(1/\alpha)/m$ because $1 - e^{-t} \le t$; and $\ln 20 \approx 3.0$. ∎

The rule of three is the subject of a short paper by Hanley and Lippman-Hand (1983). The Clopper–Pearson interval is conservative; the interval of Wilson (1927) is shorter and has close to nominal coverage, and the usual normal-approximation interval $\hat{p} \pm z_{1-\alpha/2}\sqrt{\hat{p}(1 - \hat{p})/m}$ fails badly near 0 and 1, where it can even have zero width (Brown et al. 2001).

Notes and References

The tools of this appendix are classical, and each has a standard source. For the t distribution and the theory of tests, Casella and Berger (2002) and Lehmann and Romano (2005); for the bootstrap, Efron (1979) and Efron and Tibshirani (1993); for asymptotics, including the delta method and M-estimators, Vaart (1998); for designed experiments, Fisher (1935), Box et al. (2005) and Montgomery (2017); for variance components, Searle et al. (1992); for extremes, Leadbetter et al. (1983).

TRANSLATING THE LITERATURE. Statistics texts write $\bar{X}$, s and n where this book writes $\bar{Y}$, S and n, and call the arms of an experiment treatments or groups. Machine learning papers often report "mean ± standard deviation"; the standard deviation describes the spread of runs, while the half-width of an interval describes the uncertainty of the mean, and the two differ by a factor of about $\sqrt{n}$ for large n. What Dodge et al. (2019) call expected validation performance is the expected maximum $\mathbb{E}M_k$ of Proposition A.9.

References

Benjamini, Y. and Y. Hochberg (1995). "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing". In: *Journal of the Royal Statistical Society: Series B* 57.1, pp. 289–300.

Benjamini, Y. and D. Yekutieli (2001). "The Control of the False Discovery Rate in Multiple Testing under Dependency". In: *The Annals of Statistics* 29.4, pp. 1165–1188.

Bickel, P. J. and D. A. Freedman (1981). "Some Asymptotic Theory for the Bootstrap". In: *The Annals of Statistics* 9.6, pp. 1196–1217.

Box, G. E. P., J. S. Hunter, and W. G. Hunter (2005). *Statistics for Experimenters: Design, Innovation, and Discovery*. 2nd ed. Wiley.

Brown, L. D., T. T. Cai, and A. DasGupta (2001). "Interval Estimation for a Binomial Proportion". In: *Statistical Science* 16.2, pp. 101–133.

Casella, G. and R. L. Berger (2002). *Statistical Inference*. 2nd ed. Duxbury.

Clopper, C. J. and E. S. Pearson (1934). "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial". In: *Biometrika* 26.4, pp. 404–413.

Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates.

David, H. A. and H. N. Nagaraja (2003). *Order Statistics*. 3rd ed. Hoboken, NJ: Wiley.

Dietterich, T. G. (1998). "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms". In: *Neural Computation* 10.7, pp. 1895–1923.

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2019). "Show Your Work: Improved Reporting of Experimental Results". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 2185–2194.

Efron, B. (1979). "Bootstrap Methods: Another Look at the Jackknife". In: *The Annals of Statistics* 7.1, pp. 1–26.

Efron, B. (1987). "Better Bootstrap Confidence Intervals". In: *Journal of the American Statistical Association* 82.397, pp. 171–185.

Efron, B. and R. J. Tibshirani (1993). *An Introduction to the Bootstrap*. New York: Chapman & Hall.

Fisher, R. A. (1935). *The Design of Experiments*. Edinburgh: Oliver and Boyd.

Hanley, J. A. and A. Lippman-Hand (1983). "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators". In: *JAMA* 249.13, pp. 1743–1745.

Holm, S. (1979). "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2, pp. 65–70.

Huber, P. J. (1964). "Robust Estimation of a Location Parameter". In: *The Annals of Mathematical Statistics* 35.1, pp. 73–101.

Jennrich, R. I. (1969). "Asymptotic Properties of Non-Linear Least Squares Estimators". In: *The Annals of Mathematical Statistics* 40.2, pp. 633–643.

Jiang, Y., B. Neyshabur, H. Mobahi, D. Krishnan, and S. Bengio (2020). "Fantastic Generalization Measures and Where to Find Them". In: *International Conference on Learning Representations*.

Kendall, M. G. (1938). "A New Measure of Rank Correlation". In: *Biometrika* 30.1/2, pp. 81–93.

Kendall, M. G. (1970). *Rank Correlation Methods*. 4th ed. London: Griffin.

Leadbetter, M. R., G. Lindgren, and H. Rootzén (1983). *Extremes and Related Properties of Random Sequences and Processes*. New York: Springer.

Lehmann, E. L. and J. P. Romano (2005). *Testing Statistical Hypotheses*. 3rd ed. New York: Springer.

Mann, H. B. and D. R. Whitney (1947). "On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other". In: *The Annals of Mathematical Statistics* 18.1, pp. 50–60.

McNemar, Q. (1947). "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages". In: *Psychometrika* 12.2, pp. 153–157.

Montgomery, D. C. (2017). *Design and Analysis of Experiments*. 9th ed. Wiley.

Phipson, B. and G. K. Smyth (2010). "Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn". In: *Statistical Applications in Genetics and Molecular Biology* 9.1.

Satterthwaite, F. E. (1946). "An Approximate Distribution of Estimates of Variance Components". In: *Biometrics Bulletin* 2.6, pp. 110–114.

Searle, S. R., G. Casella, and C. E. McCulloch (1992). *Variance Components*. New York: Wiley.

Seber, G. A. F. and C. J. Wild (1989). *Nonlinear Regression*. New York: Wiley.

Student (1908). "The Probable Error of a Mean". In: *Biometrika* 6.1, pp. 1–25.

Vaart, A. W. van der (1998). *Asymptotic Statistics*. Cambridge: Cambridge University Press.

Welch, B. L. (1947). "The Generalization of "Student's" Problem when Several Different Population Variances are Involved". In: *Biometrika* 34.1/2, pp. 28–35.

Wilson, E. B. (1927). "Probable Inference, the Law of Succession, and Statistical Inference". In: *Journal of the American Statistical Association* 22.158, pp. 209–212.

Index

adaptive gradient methods, 72
assay sensitivity, 97
Atari 100k, 51

batch invariance, 33
Benjamini–Hochberg procedure, 119
best of k, 17
bootstrap, 116
    percentile interval, 116
    stratified, 58, 116
budget, 4
budget curve, 77

churn, 40
Clopper–Pearson interval, 124
concomitant, 81
control
    historical, 97
    negative, 97
    placebo, 97
    positive, 97
    sham, 97
crossed design, 18

delta method, 120
divergence time, 39

effective number of tasks, 55
environment of a run, 35
expected maximum, 59, 119
expected validation performance, 82
experimental unit, 2

factor, 4
factorial design, 122
failed run, 18
false discovery rate, 59, 118
family-wise error rate, 59, 118
floating-point arithmetic, 35

garden of forking paths, 60

held-fixed variable, 4
Holm's procedure, 118
human-normalized score, 50

inclusion of optimizers, 75
instability of training, 33
interquartile mean, 55
item churn, 16
item difficulty, 16

Kendall's tau, 123
Kendall's τ, 101

Lyapunov exponent, 39

McNemar's test, 118
momentum (heavy ball), 73
multiple comparisons, 59

neural architecture search, 92
nonlinear least squares, 121
nuisance variable, 4

optimality gap, 55
order statistics, 119
outcome, 4

p-value
    valid, 115
pairing, 21
performance profile, 55
permutation test, 117
population-level inference, 54
power, 21, 123
probability of improvement, 55, 123
protocol, 4
proxy, 97
pseudoreplication, 2

random effects, 121
random search, 77, 97
reproducibility
    bitwise (repeatability), 35
    distributional, 35
    of a conclusion, 35
rule of three, 124
run, 4
run distribution, 4, 14
running mean, 15

sample size, 123

search space, 77
    headroom, 98
    spread, 98
seed, 14
standard error, 15, 115
suite-level inference, 54
summation
    pairwise, 36
    recursive, 36
    tree, 36

t-interval, 115
tabular benchmark, 102
training recipe, 14
treatment, 4
trial, 77
true error, 98
tuning budget, 77
tuning protocol, 77
twin run, 37

U-statistic, 82
unit in the last place (ulp), 35
unit roundoff, 35

variance components, 19, 121

weight sharing, 92
Welch's test, 115
winner's curse, 101

How This Book Was Made

Colophon

Roles

Deep Learning as an Experimental Science was written by Claude Opus 5.5, an AI model made by Anthropic, in October 2026. Xiaoyu Li conceived the series *The Missing Textbooks* and proposed this volume: its subject, its gap (an experimental science whose method is taught by word of mouth), its format (each chapter opening with a famous wrong conclusion and performing its post-mortem), its first examples, and its position relative to existing books. Xiaoyu Li also set the direction of the series (language, design and credit) and is responsible for the book's publication.

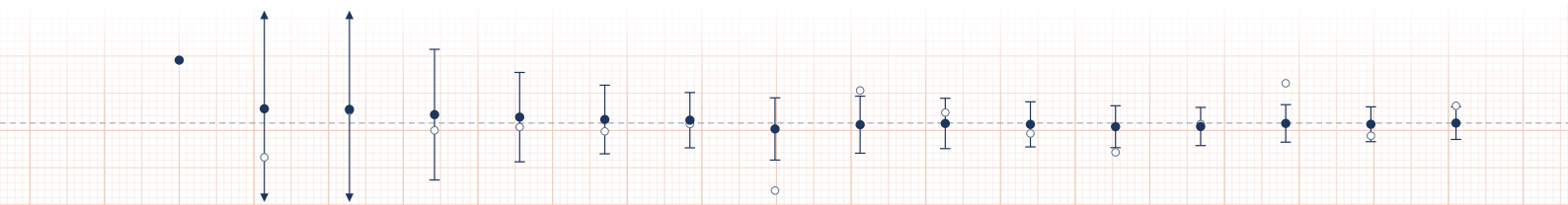
Process

A written plan fixed the thesis, the fifteen famous conclusions, the verdict vocabulary, the global notation, the conventions of the laboratory, and a specification for each chapter. A style guide fixed the anatomy of a chapter, the voice, a proof style that follows Halmos, and a rule of fairness: each claim is stated as its authors stated it before it is examined. A coordinating instance of the model designed the volume's additions to the series typography, wrote the shared laboratory kit, the prologue and the front matter, and integrated the whole. Separate instances of the same model drafted the chapters, at most five at a time, each verifying its references with web search and running its own experiments.

Limits

No human has yet checked every proof or every attribution. Web search was available while the book was written, but the full texts of many papers could not be opened, so some claims about what a paper reports were checked against abstracts, tables and secondary accounts; the chapters say so where it matters. The laboratory experiments are deliberately small, and a miniature that reproduces a failure does not prove that the failure explains the original result. Results from 2024 to 2026 are cited from preprints where no refereed version could be found. Corrections are welcome and will be recorded in the repository of *The Missing Textbooks*.

Typeset with pdfLaTeX in `missingbook`, the series class built on KOMA-Script, with the volume style `laboratory`; Palatino (newpx) for text and mathematics, Source Sans and Source Code Pro for labels and code, biblatex and biber for references, Ti*k*Z and pgfplots for figures, whose data were computed with Python, NumPy, SciPy, scikit-learn and PyTorch. The title page is log-log paper. On it, a scaling law with an irreducible term is fitted, by least squares in the logarithm of the loss, to six measured scales of five runs each (synthetic, with five percent run-to-run noise); the filled marks are means with 95 percent intervals, the open ones single runs,



and the band is a bootstrap 95 percent band for the fitted curve, which is dashed where it extrapolates. The foot of each opening page is millimetre paper carrying sixteen draws from a standard normal distribution, made with NumPy's generator and the seed 3407, one more draw for each chapter, with their running mean and a 95 percent t-interval; the first draw lies 2.8 standard deviations above the true mean and the sixteenth running mean lies 0.001 below it. Both are computed by `00-emblem.py`.